TÉCNICO
UNIVERSIDADE
DE LISBOA

Advanced Network Security and Architectures

METI

Report Module 2 [EN]

Authors:
José Oliveira (103771) jose.novais.oliveira@tecnico.ulisboa.pt
Afonso Mendes (116024) afonsofmendes@tecnico.ulisboa.pt
Tiago Videira (116069) tiago.g.videira@tecnico.ulisboa.pt

Group 3

2025/2026 – 2nd Semester, P4

Contents

1 Introduction

This report presents the experimental work carried out during Module 2 of the Advanced Network Security and Architectures (SAAR) course, covering the design, configuration, and analysis of secure network communications and network virtualization technologies.

The module is organized into three laboratory sessions. Lab 2.1 addresses IPSec and DMVPN, covering GRE tunneling, IPSec with IKEv1 (pre-shared keys and digital signatures), IPSec with IKEv2, and DMVPN Phase 3. Lab 2.2 focuses on VRFs and BGP-MPLS Layer 3 VPNs, exploring how virtual routing instances and multi-protocol BGP cooperate to provide isolated connectivity across a shared provider backbone. Lab 2.3 covers the networking of virtual containers, progressing from Linux network namespaces and IPIP tunneling through Docker networking (macvlan with VLAN isolation, VXLAN with multicast routing) to Docker Swarm service orchestration.

All experiments were performed in GNS3, using Cisco 7200 and 3725 routers for the IPSec and BGP-MPLS scenarios, and Ubuntu Cloud Guest nodes with Docker for the container networking exercises. Protocol behaviour was observed and verified using Wireshark captures, Cisco IOS `show` commands, and Linux networking tools throughout.

The report follows the sequence of the lab guides. Each section identifies the experimental setup, presents the relevant evidence, and explains the observed behaviour in terms of the underlying protocols and mechanisms. Configuration details that would interrupt the flow of the discussion are moved to appendices where appropriate.

2 IKEv1 with Pre-Shared Keys

2.1 Introduction

In this experiment, IPSec tunnels using IKEv1 with Pre-Shared Keys (PSK) were analyzed. Both AH and ESP protocols were tested in order to observe the establishment of IPSec Security Associations (SAs), OSPF traffic protection, and the exchanged ISAKMP messages during tunnel negotiation.

2.2 AH Tunnel Operation

Initially, the IPSec tunnel was configured using the Authentication Header (AH) protocol. ICMP traffic exchanged between PC1 and PC2 was captured using Wireshark.

Figure 1 shows an AH-protected ICMP packet. The AH header includes fields such as the Security Parameters Index (SPI), Sequence Number, and Integrity Check Value (ICV). The SPI identifies the Security Association being used, the sequence number prevents replay attacks, and the ICV guarantees packet integrity and authentication.

Unlike ESP, AH does not encrypt the payload. Therefore, the original ICMP packet remains visible after the AH header.

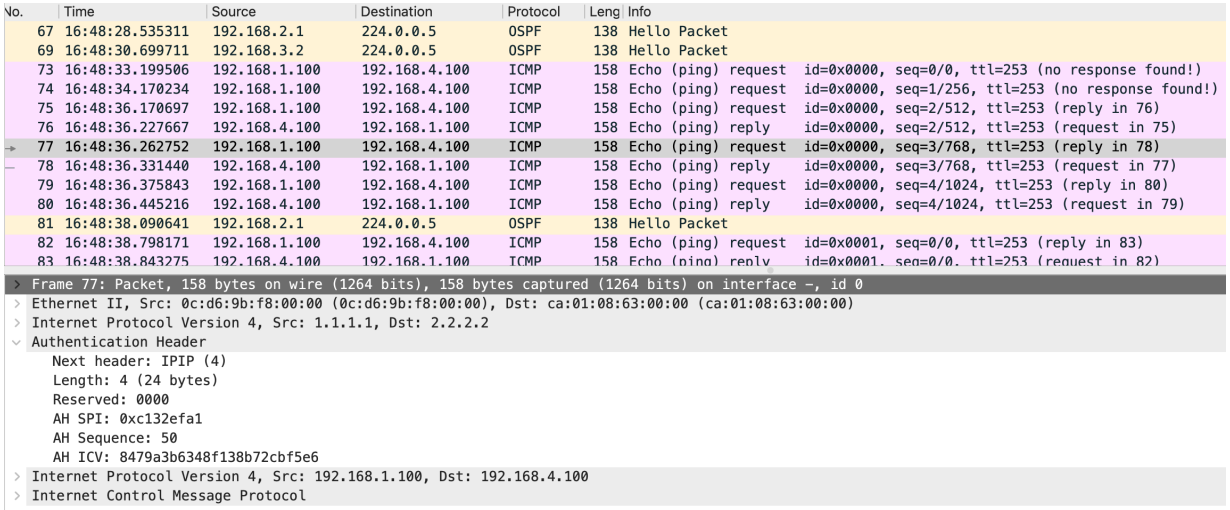


Figure 1: AH protecting ICMP traffic between PC1 and PC2

2.3 OSPF Traffic Analysis

OSPF traffic exchanged through the tunnel was also analyzed.

Figure 2 shows a normal OSPF Hello packet before IPSec protection. The packet includes the Router ID, Area ID, Hello Interval, Dead Interval, and neighbor information required for adjacency establishment.

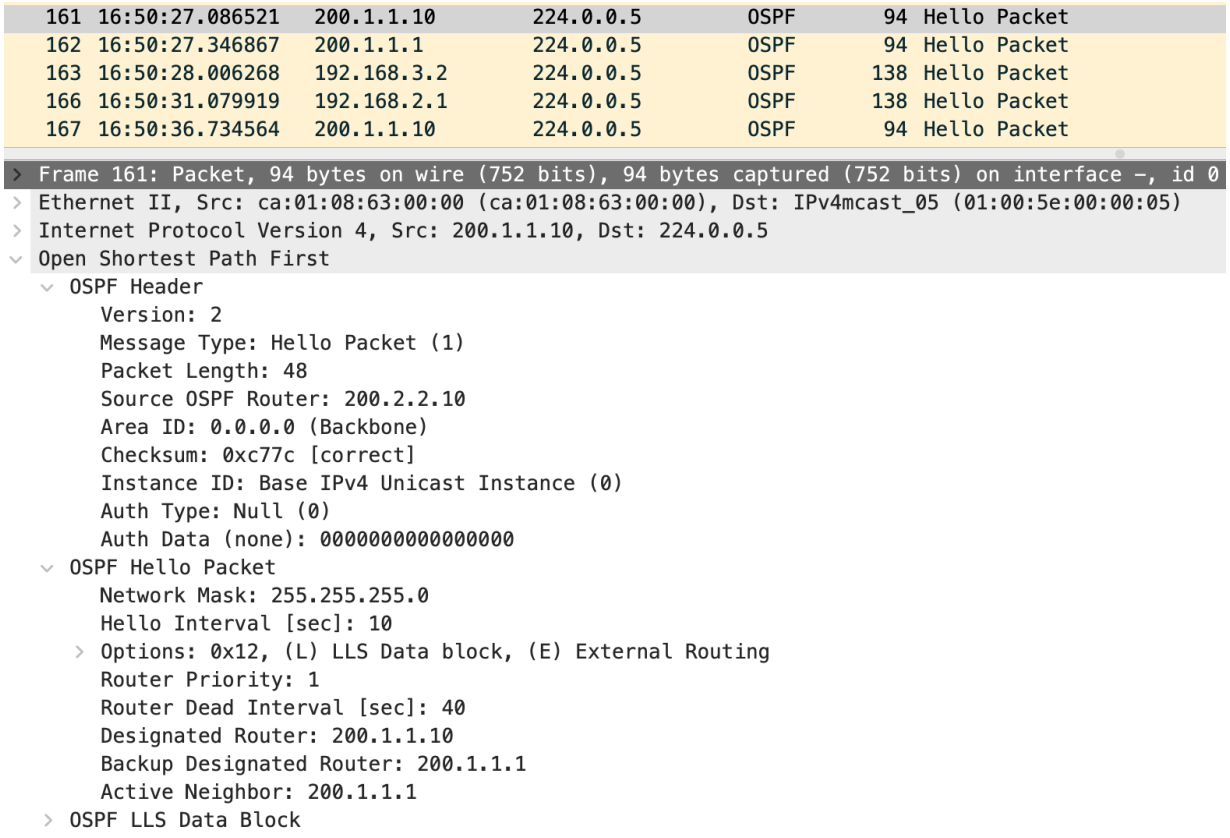


Figure 2: OSPF Hello packet without IPSec protection

Figure 3 shows the same OSPF traffic protected using AH. The Authentication Header appears before the OSPF packet, proving that the packet integrity and authentication are protected. However, the OSPF payload itself remains visible because AH does not provide confidentiality.

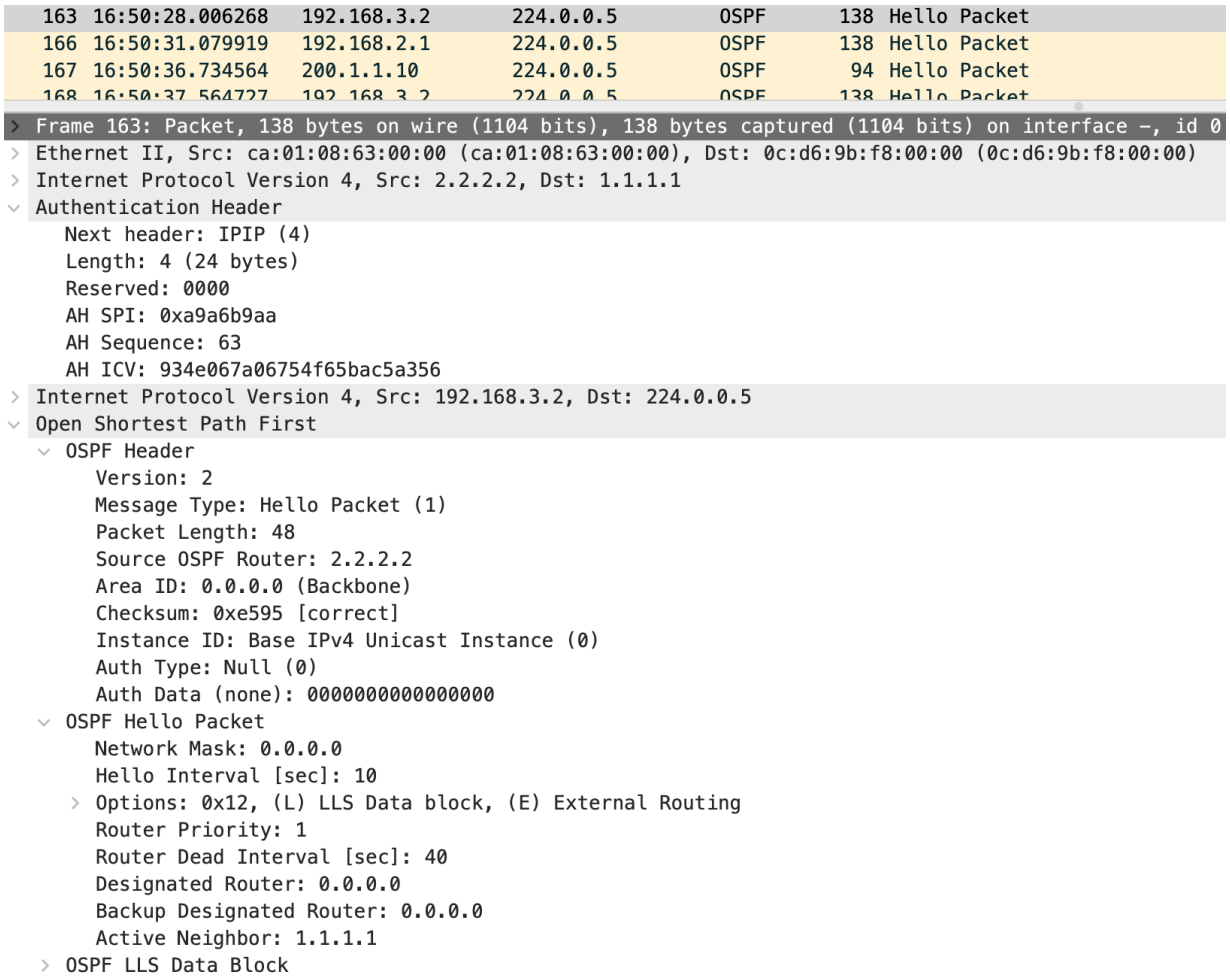


Figure 3: OSPF Hello packet protected with AH

2.4 IKEv1 and ISAKMP Negotiation

The IPSec Security Associations were cleared using the command:

```
clear crypto session
```

This operation forced the routers to delete the existing IPSec tunnel and establish a new one. During this process, Wireshark captured the exchanged ISAKMP packets.

2.4.1 ISAKMP Informational Messages

Before re-establishing the tunnel, ISAKMP Informational messages were exchanged between peers. These messages are used to maintain or delete Security Associations.

Figure 4 shows an Informational exchange containing encrypted data.

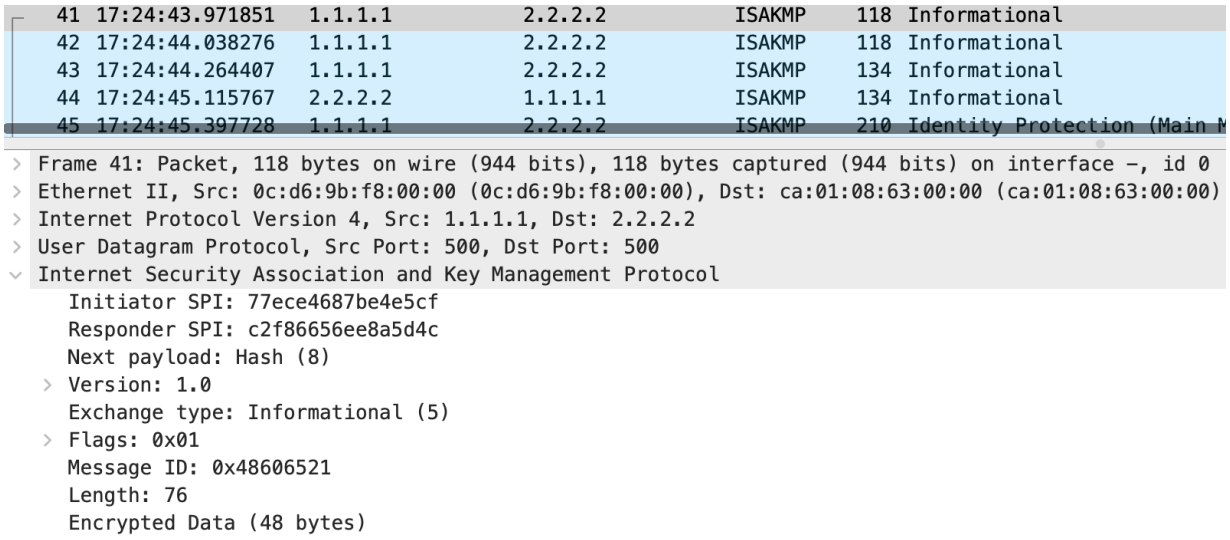


Figure 4: ISAKMP Informational message

2.4.2 IKE Phase 1 – Main Mode

IKE Phase 1 establishes the ISAKMP Security Association used to securely negotiate IPSec parameters.

Figure 5 shows the initial Main Mode exchange. The Security Association payload contains the proposed cryptographic algorithms and authentication methods supported by the initiator.

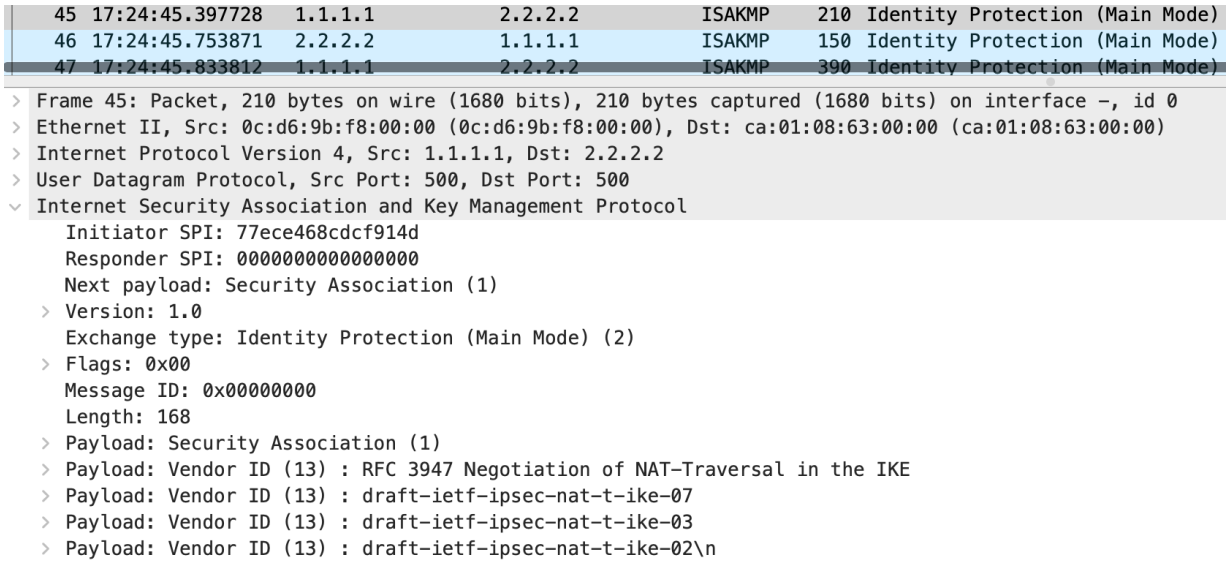


Figure 5: IKEv1 Main Mode Security Association negotiation

Figure 6 shows the Diffie-Hellman exchange phase. The Key Exchange payload carries the public Diffie-Hellman values required to generate the shared secret key between both peers. The Nonce payload adds randomness to the generated keys, increasing security.

The Key Exchange payload is therefore responsible for securely generating a common secret without transmitting the secret itself over the network.

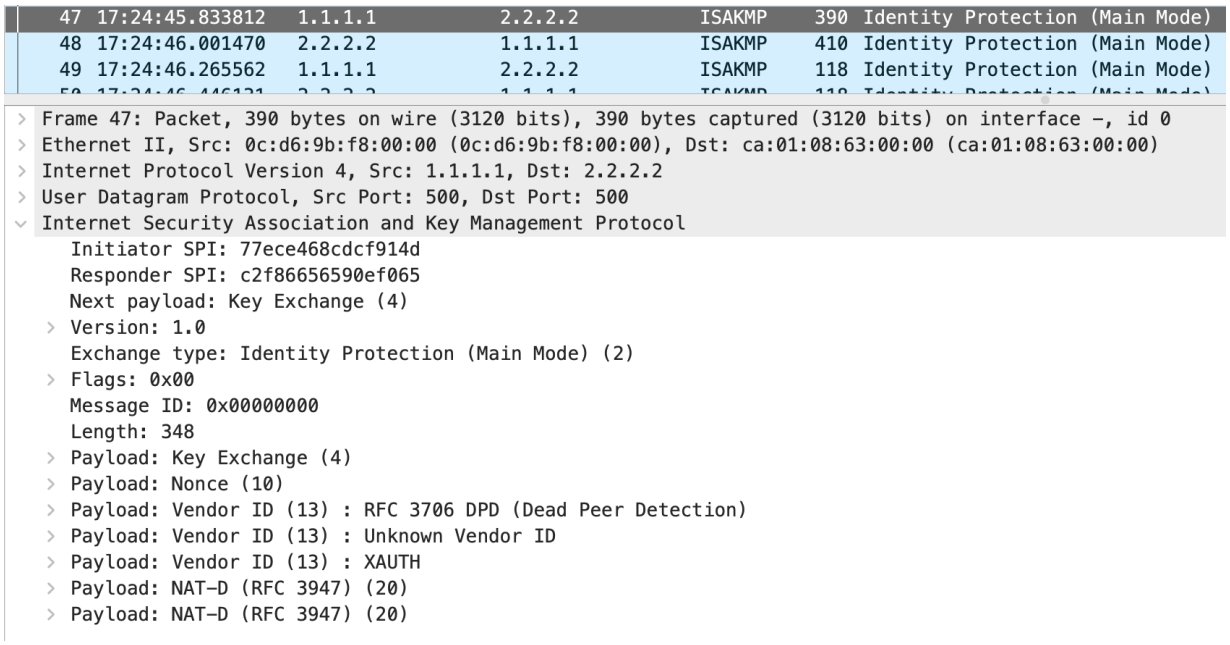


Figure 6: Diffie-Hellman Key Exchange during IKE Phase 1

The Nonce payload (also visible in Figure 6) contributes a fresh random value to every exchange. Combined with the Diffie-Hellman output, it ensures that even if the same DH parameters were reused across sessions, the derived keying material would differ — protecting against key reuse and replay attacks.

Figure 7 shows the final Main Mode exchanges where identity information and authentication data are transmitted in encrypted form. At this stage, the ISAKMP SA has already been established.

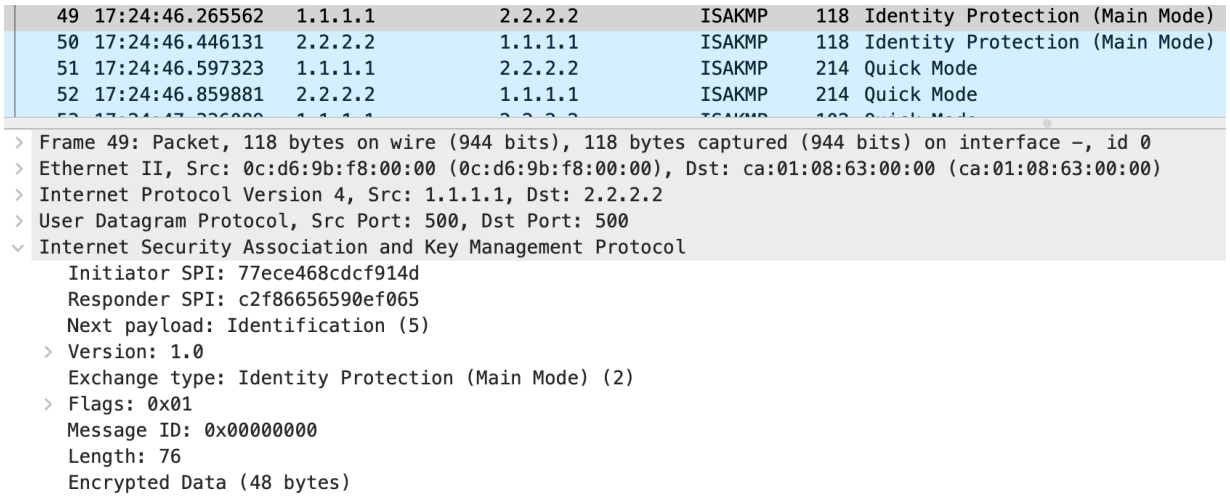


Figure 7: Identity protection during IKE Phase 1

2.4.3 IKE Phase 2 – Quick Mode

IKE Phase 2 negotiates the IPSec Security Associations used to protect data traffic.

Figure 8 shows the Quick Mode exchange responsible for negotiating IPSec parameters such as AH/ESP protocols, encryption algorithms, integrity algorithms, and Security Associations.

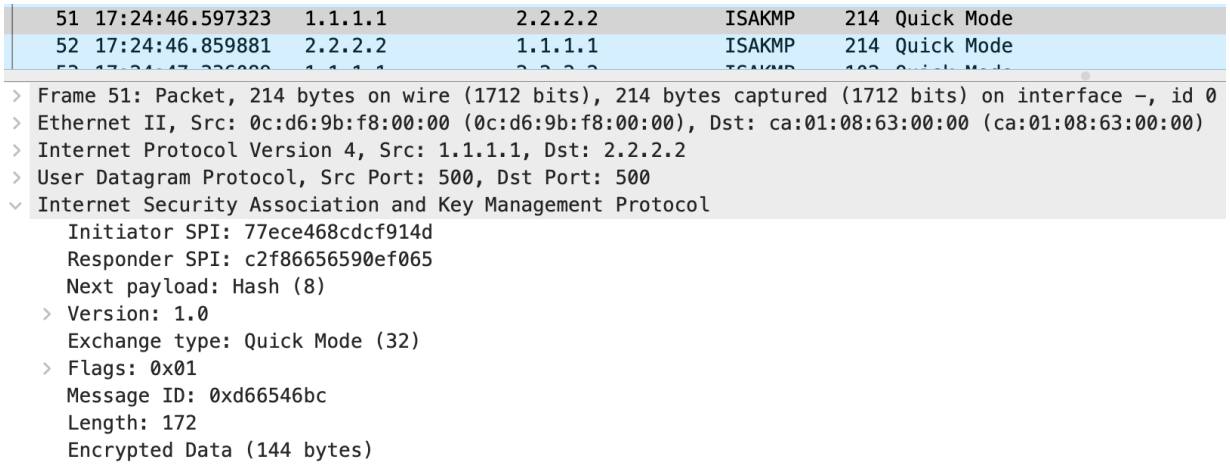


Figure 8: IKE Phase 2 Quick Mode negotiation

The router outputs confirmed the successful establishment of the tunnel. The ISAKMP state reached `QM_IDLE`, indicating that Phase 1 and Phase 2 negotiations completed successfully.

```
Router#show crypto isakmp sa

dst             src             state          conn-id status
1.1.1.1         2.2.2.2         QM_IDLE           1002 ACTIVE
2.2.2.2         1.1.1.1         QM_IDLE           1003 ACTIVE
```

The IPSec Security Associations were also verified using:

```
show crypto ipsec sa
```

The output confirmed active inbound and outbound AH Security Associations.

2.5 Negotiation Failure Analysis

To trigger a negotiation failure, the IPSec protocol on R2 was changed from AH to ESP while R1 retained the AH configuration, creating an incompatible transform set. The resulting repeated Main Mode and Quick Mode exchanges are shown in Figure 9.

```
151 12:09:26.242043   1.1.1.1          2.2.2.2          ISAKMP    118 Informational
152 12:09:26.288903   1.1.1.1          2.2.2.2          ISAKMP    118 Informational
153 12:09:26.493217   1.1.1.1          2.2.2.2          ISAKMP    134 Informational
154 12:09:27.316934   1.1.1.1          2.2.2.2          ISAKMP    210 Identity Protection (Main Mode)
155 12:09:27.443088   2.2.2.2          1.1.1.1          ISAKMP    150 Identity Protection (Main Mode)
156 12:09:27.652128   1.1.1.1          2.2.2.2          ISAKMP    390 Identity Protection (Main Mode)
157 12:09:27.886886   2.2.2.2          1.1.1.1          ISAKMP    410 Identity Protection (Main Mode)
158 12:09:28.100810   1.1.1.1          2.2.2.2          ISAKMP    150 Identity Protection (Main Mode)
160 12:09:28.255179   2.2.2.2          1.1.1.1          ISAKMP    118 Identity Protection (Main Mode)
162 12:09:28.672144   1.1.1.1          2.2.2.2          ISAKMP    214 Quick Mode
163 12:09:28.926799   2.2.2.2          1.1.1.1          ISAKMP    134 Informational
164 12:09:29.855434   2.2.2.2          1.1.1.1          ISAKMP    210 Identity Protection (Main Mode)
166 12:09:30.042589   1.1.1.1          2.2.2.2          ISAKMP    150 Identity Protection (Main Mode)
167 12:09:30.176062   2.2.2.2          1.1.1.1          ISAKMP    390 Identity Protection (Main Mode)
169 12:09:30.341166   1.1.1.1          2.2.2.2          ISAKMP    410 Identity Protection (Main Mode)
170 12:09:30.892237   2.2.2.2          1.1.1.1          ISAKMP    118 Identity Protection (Main Mode)
171 12:09:31.046912   1.1.1.1          2.2.2.2          ISAKMP    118 Identity Protection (Main Mode)
172 12:09:31.297021   2.2.2.2          1.1.1.1          ISAKMP    230 Quick Mode
173 12:09:31.401622   1.1.1.1          2.2.2.2          ISAKMP    134 Informational
185 12:10:00.474533   2.2.2.2          1.1.1.1          ISAKMP    230 Quick Mode
186 12:10:00.568552   1.1.1.1          2.2.2.2          ISAKMP    134 Informational
191 12:10:13.892447   1.1.1.1          2.2.2.2          ISAKMP    214 Quick Mode
192 12:10:13.959083   2.2.2.2          1.1.1.1          ISAKMP    134 Informational
211 12:10:58.901471   2.2.2.2          1.1.1.1          ISAKMP    230 Quick Mode
212 12:10:59.105788   1.1.1.1          2.2.2.2          ISAKMP    134 Informational
213 12:10:59.466884   1.1.1.1          2.2.2.2          ISAKMP    214 Quick Mode
214 12:10:59.621211   2.2.2.2          1.1.1.1          ISAKMP    134 Informational
```

Figure 9: Failed IPSec negotiation due to parameter mismatch

2.6 ESP Tunnel Operation

The transform set was later changed to ESP using the command:

```
crypto ipsec transform-set myTSet esp-aes esp-sha-hmac
```

ESP provides confidentiality in addition to authentication and integrity.

Figure 10 shows an ESP-protected packet. Unlike AH, only the ESP header remains visible, while the original payload is encrypted and cannot be inspected in Wireshark.

```
No.   Time          Source     Destination   Protocol  Leng Info
    2 12:30:08.264215 2.2.2.2    1.1.1.1       ESP        166 ESP (SPI=0x6e0b4bc2)

> Frame 2: Packet, 166 bytes on wire (1328 bits), 166 bytes captured (1328 bits) on interface -, id 0
> Ethernet II, Src: ca:01:08:63:00:00 (ca:01:08:63:00:00), Dst: 0c:d6:9b:f8:00:00 (0c:d6:9b:f8:00:00)
> Internet Protocol Version 4, Src: 2.2.2.2, Dst: 1.1.1.1
v Encapsulating Security Payload
    ESP SPI: 0x6e0b4bc2 (1846234050)
    ESP Sequence: 21
```

Figure 10: ESP protected packet

Figure 11 shows ESP packets exchanged together with OSPF Hello packets during tunnel operation.

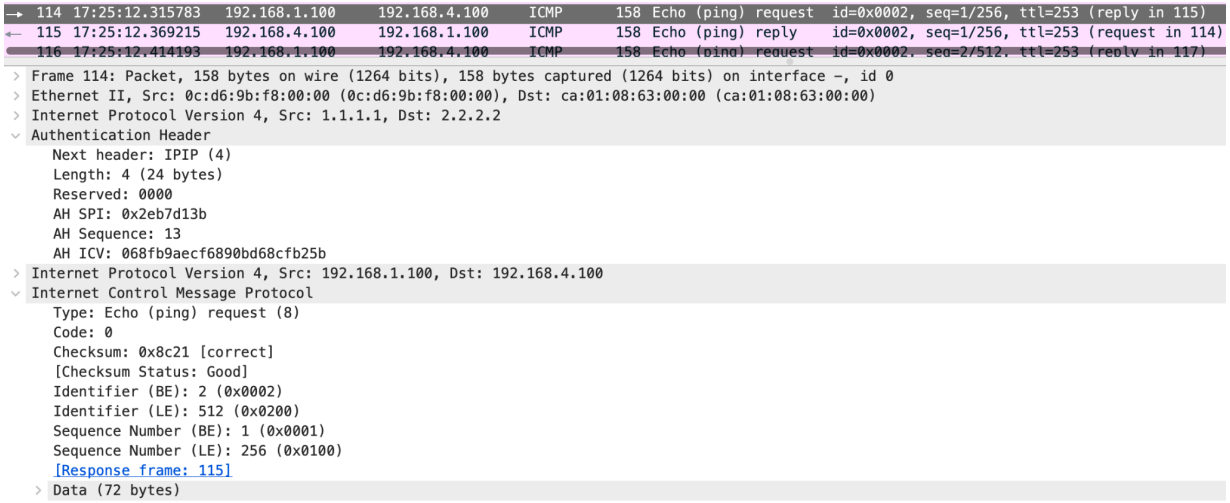


Figure 11: ESP traffic and OSPF exchanges

Compared to AH, ESP provides confidentiality by encrypting the payload, while AH only guarantees integrity and authentication. In AH mode, the original payload remains visible in Wireshark, whereas in ESP mode only the ESP header can be inspected and the payload contents remain encrypted.

2.7 Firewall Requirements for IPSec

In order for an IPSec tunnel to be successfully established through a firewall, the firewall must allow the protocols and ports used by IKE and IPSec.

The following traffic must be permitted:

- UDP port 500 (ISAKMP/IKE)
- ESP protocol (IP protocol 50)
- AH protocol (IP protocol 51), when AH is used

If NAT traversal is used, UDP port 4500 must also be allowed.

Blocking any of these protocols prevents successful establishment of the IPSec tunnel.

3 IKEv1 with Digital Signatures

3.1 Introduction

This section describes the configuration and analysis of IPSec VPN using IKEv1 with digital certificates (PKI-based authentication using SCEP). The objective is to replace pre-shared keys with RSA signatures issued by a Certification Authority (CA).

3.2 Topology Overview

RA operates as the Certification Authority (CA) and NTP server. R1 and R2 establish an IPSec tunnel authenticated using RSA signatures and certificates obtained through SCEP enrollment.

3.3 PKI Installation Verification

The Certification Authority (CA) and router certificates were verified using IOS show commands.

3.3.1 CA Configuration and Certificate (RA)

The CA is configured on router RA and operates in automatic enrollment mode.

```
RA#show crypto pki server
Certificate Server myCA:
    Status: enabled
    State: enabled
    Server's configuration is locked  (enter "shut" to unlock it)
    Issuer name: cn=saarCA
    CA cert fingerprint: 46CB9C10 CC370311 8CFCB545 E4DD2F58
    Granting mode is: auto
    Last certificate issued serial number (hex): 1
    CA certificate expiration timer: 16:17:23 UTC May 14 2029
    CRL NextUpdate timer: 22:17:23 UTC May 15 2026
    Current primary storage dir: nvram:
    Database Level: Minimum - no cert data written to storage
```

The CA certificate installed on RA confirms that RA is acting as a trusted root authority:

```
RA#show crypto ca certificate
CA Certificate
  Status: Available
  Certificate Serial Number (hex): 01
  Certificate Usage: Signature
  Issuer:
    cn=saarCA
  Subject:
```

```
    cn=saarCA
  Validity Date:
    start date: 16:17:23 UTC May 15 2026
    end   date: 16:17:23 UTC May 14 2029
  Associated Trustpoints: myCA
  Storage: nvram:saarCA#1CA.cer
```

The detailed certificate view confirms RSA key usage and X.509 extensions:

```
RA#show crypto ca certificate verbose
CA Certificate
  Status: Available
  Version: 3
  Certificate Serial Number (hex): 01
  Certificate Usage: Signature
  Issuer:
    cn=saarCA
  Subject:
    cn=saarCA
  Validity Date:
    start date: 16:17:23 UTC May 15 2026
    end   date: 16:17:23 UTC May 14 2029
  Subject Key Info:
    Public Key Algorithm: rsaEncryption
    RSA Public Key: (1024 bit)
  Signature Algorithm: MD5 with RSA Encryption
  Fingerprint MD5: 46CB9C10 CC370311 8CFCB545 E4DD2F58
  Fingerprint SHA1: 0B6BF47E F0932FC1 54A33FA8 4C1F48D1 80083958
  X509v3 extensions:
    X509v3 Key Usage: 86000000
      Digital Signature
      Key Cert Sign
      CRL Signature
    X509v3 Subject Key ID: 42BD5391 0D07D957 6CC26630 AABA09E4 A93162E2
    X509v3 Basic Constraints:
        CA: TRUE
    X509v3 Authority Key ID: 42BD5391 0D07D957 6CC26630 AABA09E4 A93162E2
    Authority Info Access:
  Associated Trustpoints: myCA
  Storage: nvram:saarCA#1CA.cer
```

3.3.2 Router Certificates (R1)

R1 successfully enrolled with the CA and received both the CA certificate and its own identity certificate:

```
R1#show crypto pki certificates
```

```
Certificate
  Status: Available
  Certificate Serial Number (hex): 03
  Certificate Usage: General Purpose
  Issuer:
    cn=saarCA
  Subject:
    Name: R1
    Serial Number: 9SO8DYF8WAKSOLHEG3ZVL
    hostname=R1+serialNumber=9SO8DYF8WAKSOLHEG3ZVL
  Validity Date:
    start date: 16:33:16 UTC May 15 2026
    end   date: 16:33:16 UTC May 15 2027
  Associated Trustpoints: myTrustpoint
```

The CA certificate is also stored locally to validate the trust chain:

```
CA Certificate
  Status: Available
  Certificate Serial Number (hex): 01
  Certificate Usage: Signature
  Issuer:
    cn=saarCA
  Subject:
    cn=saarCA
  Validity Date:
    start date: 16:17:23 UTC May 15 2026
    end   date: 16:17:23 UTC May 14 2029
  Associated Trustpoints: myTrustpoint
```

3.3.3 RSA Public Keys (R1 and R2)

The RSA public keys generated on R1 and R2 are extracted using the following IOS command:

```
show crypto key mypub rsa
```

R2 RSA Public Key:

```
R2#show crypto key mypub rsa
% Key pair was generated at: 16:32:05 UTC May 15 2026
Key name: R2
Key type: RSA KEYS
 Storage Device: private-config
 Usage: General Purpose Key
 Key is not exportable.
 Key Data:
```

```
  30819F30 0D06092A 864886F7 0D010101 05000381 8D003081 89028181 00C337FE
  B0DAEE4D 444B0AF7 A19A73B3 2D2FCD5E 27E09095 3017E393 2F27F7AA 581894E6
  5E7F50B3 0454ACF2 FD1E8E24 F91125DD 6D611D93 611D70AD 36C59F6F EAEE003C
  7704B6DB 41D9A165 27252864 17170DA2 F38A68D4 46C91133 36DBCBC5 B09AD90E
  0F05531B F7ABB4A5 0B954C0F 389C2195 3DE377FF 7FE63FB3 B02DF189 35020301
  0001
% Key pair was generated at: 16:57:46 UTC May 16 2026
Key name: R2.server
Key type: RSA KEYS
Temporary key
 Usage: Encryption Key
 Key is not exportable.
 Key Data:
  307C300D 06092A86 4886F70D 01010105 00036B00 30680261 00A38B5F B33F29E4
  E5E30AFB 512084C9 0A89E015 BCA1569A C10C686B B79BD8EC C3C25CAD 6EE74C8F
  C891E5FF 1B042503 710EC940 65712523 377F2776 77CDC4C0 D7D61754 57C88063
  19BF191B F48130CC 327EF733 1F877D95 5CB74450 AEEF7E99 45020301 0001
```

R1 RSA Public Key:

```
R1#show crypto key mypub rsa
% Key pair was generated at: 16:31:43 UTC May 15 2026
Key name: R1
Key type: RSA KEYS
 Storage Device: private-config
 Usage: General Purpose Key
 Key is not exportable.
 Key Data:
  30819F30 0D06092A 864886F7 0D010101 05000381 8D003081 89028181 00DF87BB
  11BD33EE C4E9DF4A C2FF2CCB 2ECCEFF9 713AAA45 633ADE4A C188109E 88680D67
  FB0A2F20 67257007 79DB3616 75D5459A 4898DE69 E74E80BA 77C993E8 A4B70D41
  00D1D2FB E37BF0E3 89C8BEEE 9830712E 8A4C13DC E30C5C72 D72FEDF2 A221F6DB
  9A1A1B8F 9A69BC61 3700CA1B B66DB044 72F24609 E0C1FE8D 24D1B3DA DD020301
  0001
% Key pair was generated at: 16:57:42 UTC May 16 2026
Key name: R1.server
Key type: RSA KEYS
Temporary key
 Usage: Encryption Key
 Key is not exportable.
 Key Data:
  307C300D 06092A86 4886F70D 01010105 00036B00 30680261 0095CC14 73EE9BA8
  1B0A40A6 E3EE90C1 BC2FAADF 2D532270 160D9039 C19C40CA 6A4E54B5 BAB8892C
  D266AE9A 53210F58 876755F6 381C44B6 5D71F57F 35705BC5 3E8D9746 61833A6A
  DB5E740B 8A22A486 83EE7AFA ACAB3798 27D03FC2 7C6EA875 FD020301 0001
```

Both routers generate 1024-bit RSA key pairs used for digital signature authentication during IKEv1 Phase 1.

3.4 SCEP Enrollment Analysis

SCEP (Simple Certificate Enrollment Protocol) is used to automate certificate distribution between routers and the CA using HTTP.

The enrollment process consists of:

- `CA certificate request`
- `CA certificate response`
- `Router certificate request`
- `Router certificate issuance`

3.4.1 CA Certificate Retrieval

Initially, R1 requests the CA certificate using the SCEP `GetCACert` operation over HTTP.

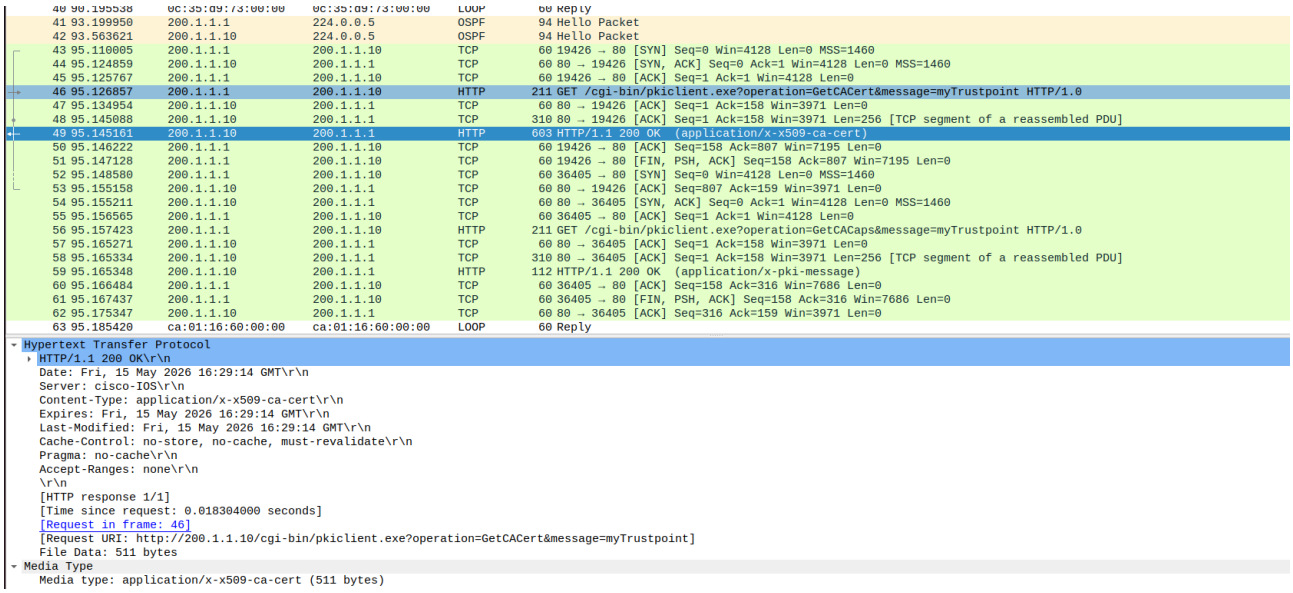


Figure 12: SCEP GetCACert request and CA certificate response

The CA replies with an HTTP `200 OK` message containing an X.509 CA certificate with media type `application/x-x509-ca-cert`. This certificate allows R1 to establish trust with the Certification Authority before certificate enrollment.

3.4.2 Certificate Enrollment Request

After establishing trust with the CA, R1 performs certificate enrollment using the SCEP `PKIOperation` request.

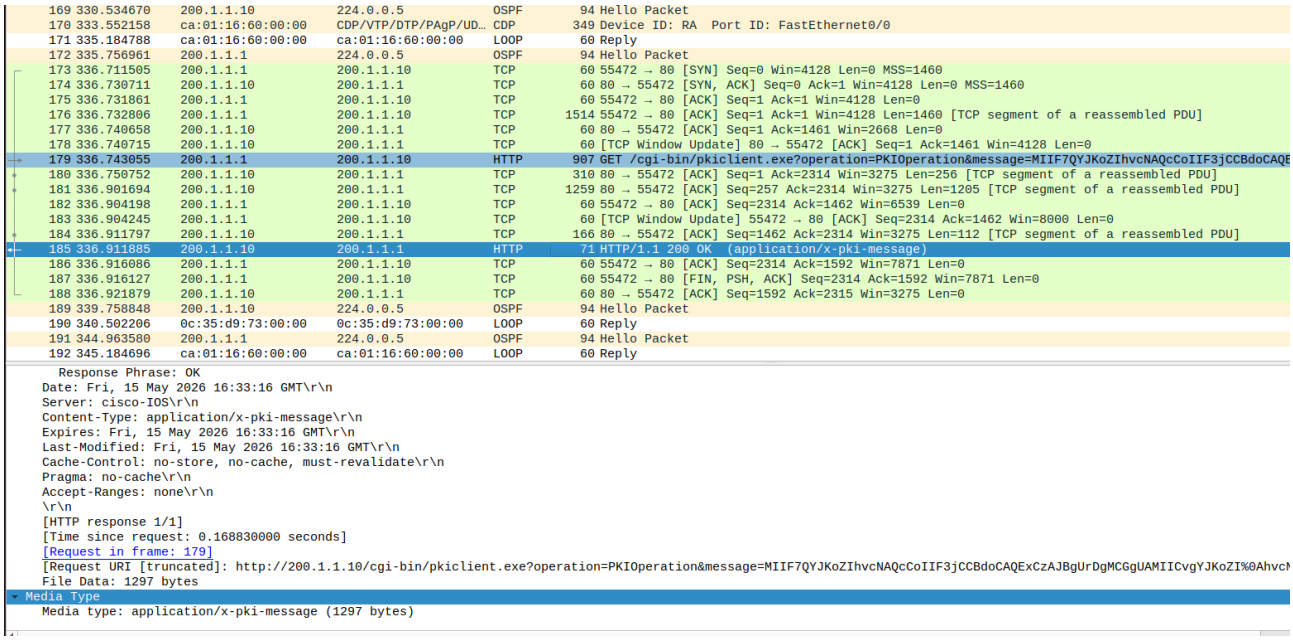
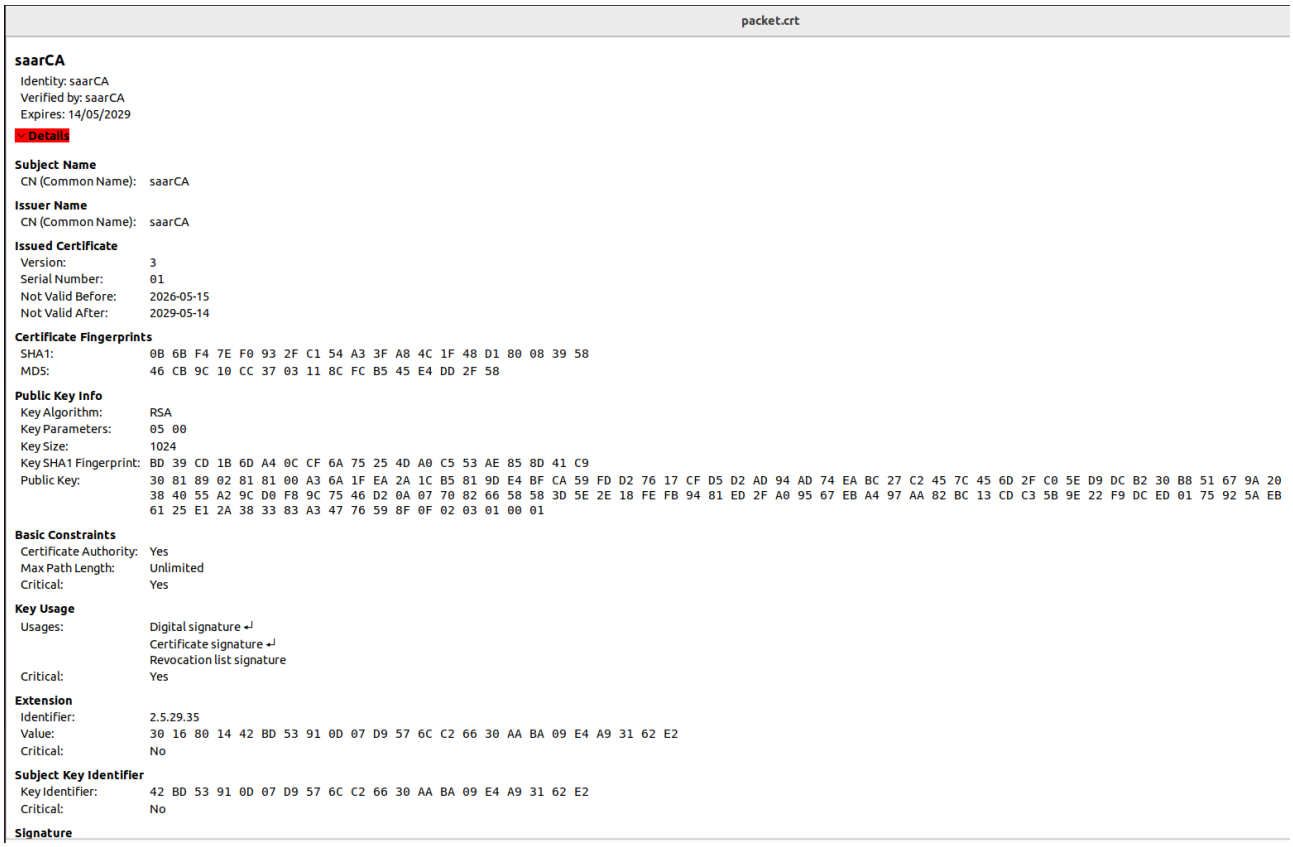


Figure 13: SCEP PKIOperation certificate enrollment request

The HTTP request contains a PKCS#10 certificate signing request (CSR) encoded in the message field. The CSR includes the router identity and its RSA public key, allowing the CA to generate a signed X.509 certificate for R1.

The captured HTTP messages show certificate exchange between R1 and the CA. The content-type field indicates certificate payload transfer (.crt).

Using Wireshark, the packet containing the issued certificate was identified by filtering HTTP traffic and locating the response message with a `Media Type` indicating a certificate object. The packet was then selected, and the certificate data was extracted by right-clicking on the relevant field and choosing *Export Packet Bytes*. The file was saved with a `.crt` extension, which allows it to be decoded as an X.509 certificate.



packet.crt

saarCA

Identity: saarCA
Verified by: saarCA
Expires: 14/05/2029

Details

Subject Name
CN (Common Name): saarCA

Issuer Name
CN (Common Name): saarCA

Issued Certificate
Version: 3
Serial Number: 01
Not Valid Before: 2026-05-15
Not Valid After: 2029-05-14

Certificate Fingerprints
SHA1: 0B 6B F4 7E F0 93 2F C1 54 A3 3F A8 4C 1F 48 D1 80 08 39 58
MD5: 46 CB 9C 10 CC 37 03 11 8C FC B5 45 E4 DD 2F 58

Public Key Info
Key Algorithm: RSA
Key Parameters: 05 00
Key Size: 1024
Key SHA1 Fingerprint: BD 39 CD 1B 6D A4 0C CF 6A 75 25 4D A0 C5 53 AE 85 8D 41 C9
Public Key: 30 81 89 02 81 81 00 A3 6A 1F EA 2A 1C B5 81 9D E4 BF CA 59 FD D2 76 17 CF D5 D2 AD 94 AD 74 EA BC 27 C2 45 7C 45 6D 2F C0 5E D9 DC B2 30 B8 51 67 9A 20 38 40 55 A2 9C D0 F8 9C 75 46 D2 0A 07 70 82 66 58 58 3D 5E 2E 18 FE FB 94 81 ED 2F A0 95 67 EB A4 97 AA 82 BC 13 CD C3 5B 9E 22 F9 DC ED 01 75 92 5A EB 61 25 E1 2A 38 33 83 A3 47 76 59 8F 0F 02 03 01 00 01

Basic Constraints
Certificate Authority: Yes
Max Path Length: Unlimited
Critical: Yes

Key Usage
Usages: Digital signature
Certificate signature
Revocation list signature
Critical: Yes

Extension
Identifier: 2.5.29.35
Value: 30 16 80 14 42 BD 53 91 0D 07 D9 57 6C C2 66 30 AA BA 09 E4 A9 31 62 E2
Critical: No

Subject Key Identifier
Key Identifier: 42 BD 53 91 0D 07 D9 57 6C C2 66 30 AA BA 09 E4 A9 31 62 E2
Critical: No

Signature

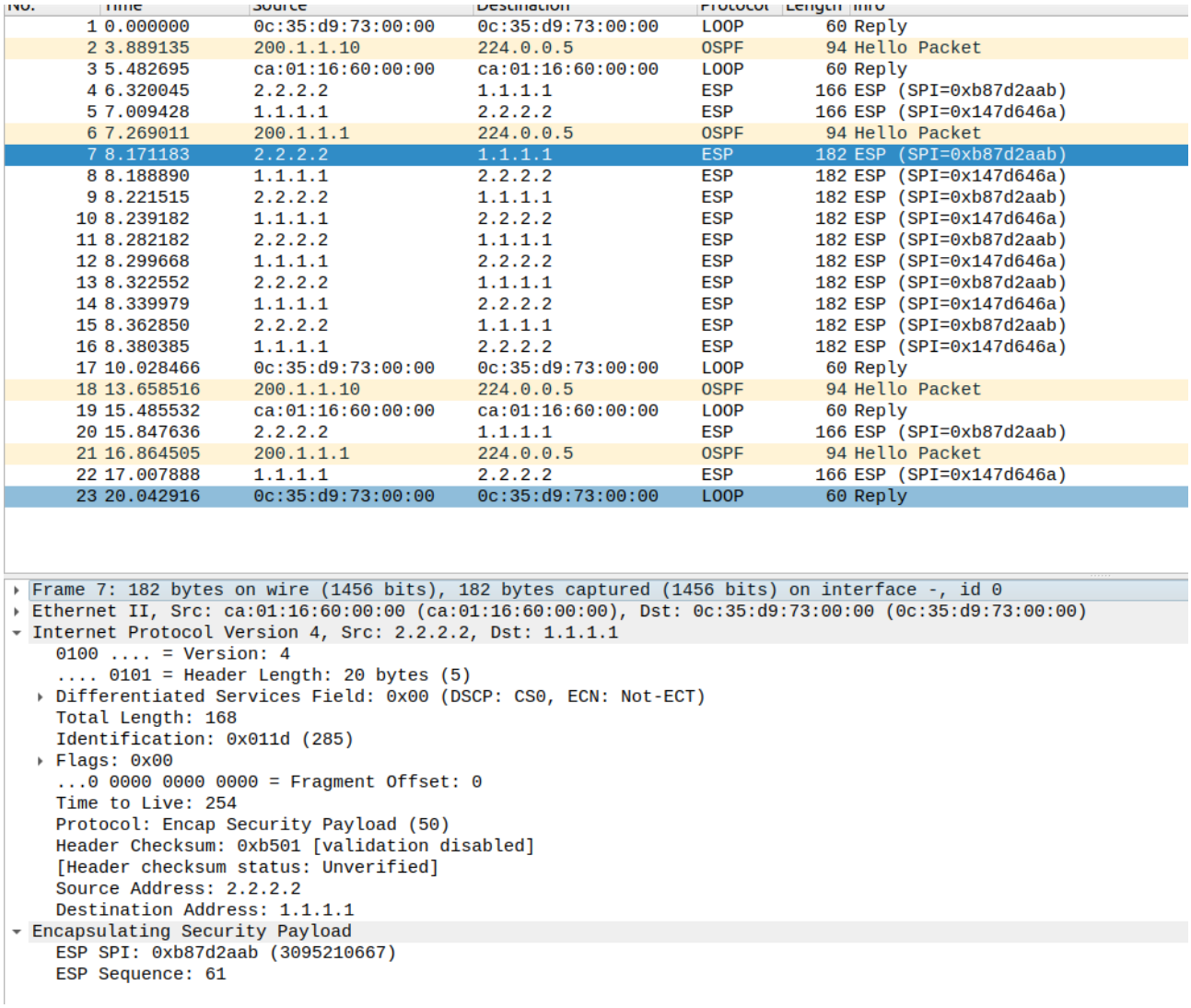
Figure 14: HTTP packet carrying the issued X.509 certificate (.crt file export in Wireshark)

The exported certificate represents the CA-signed identity certificate for R1. It contains the subject information, the public key corresponding to R1, and the digital signature generated by the CA, which validates its authenticity within the PKI infrastructure.

3.5 Encrypted ICMP Traffic Analysis

After configuring IPSec with RSA-signature authentication, connectivity between PC1 and PC2 was successfully verified using ICMP echo requests (ping).

Traffic was captured on the public network between R1 and R2 while the ping operation was performed.



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	0c:35:d9:73:00:00	0c:35:d9:73:00:00	LOOP	60	Reply
2	3.889135	200.1.1.10	224.0.0.5	OSPF	94	Hello Packet
3	5.482695	ca:01:16:60:00:00	ca:01:16:60:00:00	LOOP	60	Reply
4	6.320045	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0xb87d2aab)
5	7.009428	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0x147d646a)
6	7.269011	200.1.1.1	224.0.0.5	OSPF	94	Hello Packet
7	8.171183	2.2.2.2	1.1.1.1	ESP	182	ESP (SPI=0xb87d2aab)
8	8.188890	1.1.1.1	2.2.2.2	ESP	182	ESP (SPI=0x147d646a)
9	8.221515	2.2.2.2	1.1.1.1	ESP	182	ESP (SPI=0xb87d2aab)
10	8.239182	1.1.1.1	2.2.2.2	ESP	182	ESP (SPI=0x147d646a)
11	8.282182	2.2.2.2	1.1.1.1	ESP	182	ESP (SPI=0xb87d2aab)
12	8.299668	1.1.1.1	2.2.2.2	ESP	182	ESP (SPI=0x147d646a)
13	8.322552	2.2.2.2	1.1.1.1	ESP	182	ESP (SPI=0xb87d2aab)
14	8.339979	1.1.1.1	2.2.2.2	ESP	182	ESP (SPI=0x147d646a)
15	8.362850	2.2.2.2	1.1.1.1	ESP	182	ESP (SPI=0xb87d2aab)
16	8.380385	1.1.1.1	2.2.2.2	ESP	182	ESP (SPI=0x147d646a)
17	10.028466	0c:35:d9:73:00:00	0c:35:d9:73:00:00	LOOP	60	Reply
18	13.658516	200.1.1.10	224.0.0.5	OSPF	94	Hello Packet
19	15.485532	ca:01:16:60:00:00	ca:01:16:60:00:00	LOOP	60	Reply
20	15.847636	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0xb87d2aab)
21	16.864505	200.1.1.1	224.0.0.5	OSPF	94	Hello Packet
22	17.007888	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0x147d646a)
23	20.042916	0c:35:d9:73:00:00	0c:35:d9:73:00:00	LOOP	60	Reply

Figure 15: Encrypted IPSec traffic observed between R1 and R2

The Wireshark capture shows that the ICMP packets exchanged between PC1 and PC2 are not visible in plaintext on the public network. Instead, the packets appear encapsulated using ESP (Encapsulating Security Payload).

The ESP packets contain encrypted payload data, preventing direct inspection of the original ICMP echo request and echo reply contents. Only the external IP header remains visible, showing communication between the tunnel endpoints (R1 and R2).

This confirms that IPSec is correctly providing:

- **Confidentiality** through payload encryption
- **Integrity protection** through ESP authentication
- **Secure tunnel encapsulation** between the two sites

The observed ESP traffic corresponds to the IPSec transform-set configuration:

```
crypto ipsec transform-set myTSet esp-aes esp-sha-hmac
```

AES is used for encryption while SHA-HMAC is used for integrity verification.

3.6 ISAKMP and SA Re-establishment Analysis

To analyze tunnel re-establishment, the IPSec Security Associations (SAs) were manually cleared using:

```
clear crypto session
```

While capturing traffic on the public network, new ISAKMP exchanges were observed as the routers negotiated fresh IPSec Security Associations.

1	0.000000	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0x37bfac13)
2	1.087267	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0x3aa0a0ef)
3	1.968939	200.1.1.10	224.0.0.5	OSPF	94	Hello Packet
4	4.571026	200.1.1.1	224.0.0.5	OSPF	94	Hello Packet
5	5.111299	0c:35:d9:73:00:00	0c:35:d9:73:00:00	LOOP	60	Reply
6	5.470172	ca:01:16:60:00:00	ca:01:16:60:00:00	LOOP	60	Reply
7	7.109808	ca:01:16:60:00:00	CDP/VTP/DTP/PAgP/UD…	CDP	349	Device ID: RA Port ID: FastEthernet0/0
8	8.201714	1.1.1.1	2.2.2.2	ISAKMP	118	Informational
9	8.203333	1.1.1.1	2.2.2.2	ISAKMP	134	Informational
10	8.217971	2.2.2.2	1.1.1.1	ISAKMP	134	Informational
11	8.888812	1.1.1.1	2.2.2.2	ISAKMP	210	Identity Protection (Main Mode)
12	8.901438	2.2.2.2	1.1.1.1	ISAKMP	150	Identity Protection (Main Mode)
13	8.907525	1.1.1.1	2.2.2.2	ISAKMP	414	Identity Protection (Main Mode)
14	8.931954	2.2.2.2	1.1.1.1	ISAKMP	434	Identity Protection (Main Mode)
15	8.968150	1.1.1.1	2.2.2.2	ISAKMP	742	Identity Protection (Main Mode)
16	9.012585	2.2.2.2	1.1.1.1	ISAKMP	742	Identity Protection (Main Mode)
17	9.031615	1.1.1.1	2.2.2.2	ISAKMP	230	Quick Mode
18	9.054747	2.2.2.2	1.1.1.1	ISAKMP	230	Quick Mode
19	9.072084	1.1.1.1	2.2.2.2	ISAKMP	102	Quick Mode
20	9.073561	1.1.1.1	2.2.2.2	ESP	150	ESP (SPI=0x59df71ad)
21	9.084887	2.2.2.2	1.1.1.1	ESP	150	ESP (SPI=0x08155835)
22	9.087434	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0x59df71ad)
23	9.105426	2.2.2.2	1.1.1.1	ESP	150	ESP (SPI=0x08155835)
24	9.105619	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0x08155835)
25	9.109976	1.1.1.1	2.2.2.2	ESP	150	ESP (SPI=0x59df71ad)
26	9.110573	1.1.1.1	2.2.2.2	ESP	262	ESP (SPI=0x59df71ad)
27	9.125696	2.2.2.2	1.1.1.1	ESP	182	ESP (SPI=0x08155835)
28	9.127906	1.1.1.1	2.2.2.2	ESP	134	ESP (SPI=0x59df71ad)
29	9.128484	1.1.1.1	2.2.2.2	ESP	150	ESP (SPI=0x59df71ad)
30	9.145928	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0x08155835)
31	9.145991	2.2.2.2	1.1.1.1	ESP	134	ESP (SPI=0x08155835)
32	9.148997	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0x59df71ad)
33	11.649739	200.1.1.10	224.0.0.5	OSPF	94	Hello Packet
34	11.661379	1.1.1.1	2.2.2.2	ESP	150	ESP (SPI=0x59df71ad)
35	11.680534	2.2.2.2	1.1.1.1	ESP	150	ESP (SPI=0x08155835)
36	13.733941	1.1.1.1	2.2.2.2	ESP	182	ESP (SPI=0x59df71ad)
37	13.754607	2.2.2.2	1.1.1.1	ESP	182	ESP (SPI=0x08155835)
38	14.254637	200.1.1.1	224.0.0.5	OSPF	94	Hello Packet
39	15.160283	0c:35:d9:73:00:00	0c:35:d9:73:00:00	LOOP	60	Reply
40	15.468087	ca:01:16:60:00:00	ca:01:16:60:00:00	LOOP	60	Reply
41	16.248445	2.2.2.2	1.1.1.1	ESP	150	ESP (SPI=0x08155835)
42	16.264702	1.1.1.1	2.2.2.2	ESP	150	ESP (SPI=0x59df71ad)
43	18.238626	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0x59df71ad)
44	18.409561	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0x08155835)
45	20.827957	200.1.1.10	224.0.0.5	OSPF	94	Hello Packet

▸ Frame 1: 166 bytes on wire (1328 bits), 166 bytes captured (1328 bits) on interface -, id 0

Figure 16: ISAKMP exchanges after clearing crypto sessions

After the existing SAs were deleted, R1 and R2 initiated a new IKEv1 Phase 1 negotiation using ISAKMP messages over UDP port 500.

The captured exchange shows the following operations:

- ISAKMP SA proposal negotiation
- Exchange of nonces and Diffie-Hellman values
- Certificate exchange between peers

- RSA digital signature authentication
- Establishment of a new ISAKMP Security Association
- Negotiation of IPSec Phase 2 parameters
- Creation of new IPSec SAs for ESP traffic

The certificate-based authentication process relies on the X.509 certificates previously obtained from the CA through SCEP enrollment.

During authentication, each router validates:

- The peer certificate authenticity
- The CA signature
- The RSA signature generated during IKE authentication

Successful validation allows the tunnel to be securely re-established without the use of pre-shared keys.

The re-established IPSec tunnel was confirmed by successful encrypted ICMP communication between PC1 and PC2 after the negotiation process completed.

3.7 Peer Public Key Storage Verification

After successful IKEv1 authentication and IPSec tunnel establishment, each router stores the public key information of the remote peer.

At R1, the public key of R2 can be viewed using:

```
show crypto key pubkey-chain rsa name R2
```

The output confirms that R1 has learned and stored the RSA public key associated with R2's certificate.

```
R1#sh crypto key pubkey-chain rsa name R2
 Key name: R2
 Subject name(X.500 DN name): hostname=R2+serialNumber=9C92ZG7WMXGLWHR7XWAK8
 Key id: 11
 Serial number: 02
 Usage: Signature Key
 Source: Certificate
 Data:
  30819F30 0D06092A 864886F7 0D010101 05000381 8D003081 89028181 00C337FE
  B0DAEE4D 444B0AF7 A19A73B3 2D2FCD5E 27E09095 3017E393 2F27F7AA 581894E6
  5E7F50B3 0454ACF2 FD1E8E24 F91125DD 6D611D93 611D70AD 36C59F6F EAEE003C
  7704B6DB 41D9A165 27252864 17170DA2 F38A68D4 46C91133 36DBCBC5 B09AD90E
  0F05531B F7ABB4A5 0B954C0F 389C2195 3DE377FF 7FE63FB3 B02DF189 35020301
  0001
```

This public key is extracted from the X.509 certificate presented by R2 during the IKEv1 authentication process.

The stored public key allows R1 to:

- Verify digital signatures generated by R2
- Validate the identity of the remote peer
- Establish trusted IPSec communications

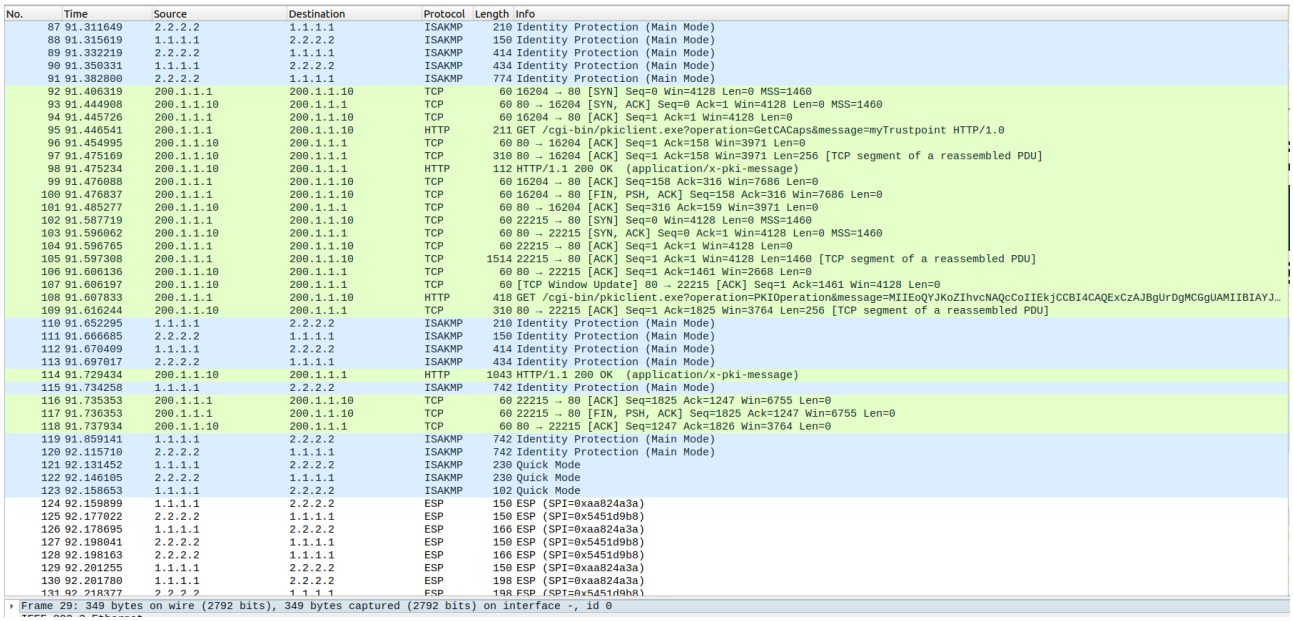
Similarly, R2 stores the public key of R1 after successful authentication.

The existence of the peer public key chain confirms that certificate-based authentication using RSA signatures was successfully completed and that both routers trust certificates issued by the Certification Authority.

3.8 SCEP Messages Observed After Router Restart

After stopping and restarting all routers, Wireshark captures were collected on the public network shortly after boot.

During startup, HTTP messages related to SCEP were again exchanged between the routers and the CA.



No.	Time	Source	Destination	Protocol	Length	Info
87	91.311649	2.2.2.2	1.1.1.1	ISAKMP	210	Identity Protection (Main Mode)
88	91.315619	1.1.1.1	2.2.2.2	ISAKMP	150	Identity Protection (Main Mode)
89	91.332219	2.2.2.2	1.1.1.1	ISAKMP	414	Identity Protection (Main Mode)
90	91.350331	1.1.1.1	2.2.2.2	ISAKMP	434	Identity Protection (Main Mode)
91	91.382800	2.2.2.2	1.1.1.1	ISAKMP	774	Identity Protection (Main Mode)
92	91.406319	200.1.1.1	200.1.1.10	TCP	60	16204 → 80 [SYN] Seq=0 Win=4128 Len=0 MSS=1460
93	91.444908	200.1.1.10	200.1.1.1	TCP	60	80 → 16204 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=1460
94	91.445726	200.1.1.1	200.1.1.10	TCP	60	16204 → 80 [ACK] Seq=1 Ack=1 Win=4128 Len=0
95	91.446541	200.1.1.1	200.1.1.10	HTTP	211	GET /cgi-bin/pkiclient.exe?operation=GetCACaps&message=myTrustpoint HTTP/1.0
96	91.454995	200.1.1.10	200.1.1.1	TCP	60	80 → 16204 [ACK] Seq=1 Ack=158 Win=3971 Len=0
97	91.475169	200.1.1.10	200.1.1.1	TCP	310	80 → 16204 [ACK] Seq=1 Ack=158 Win=3971 Len=256 [TCP segment of a reassembled PDU]
98	91.475234	200.1.1.10	200.1.1.1	HTTP	112	HTTP/1.1 200 OK (application/x-pki-message)
99	91.476088	200.1.1.1	200.1.1.10	TCP	60	16204 → 80 [ACK] Seq=158 Ack=316 Win=7686 Len=0
100	91.476837	200.1.1.1	200.1.1.10	TCP	60	16204 → 80 [FIN, PSH, ACK] Seq=158 Ack=316 Win=7686 Len=0
101	91.485277	200.1.1.10	200.1.1.1	TCP	60	80 → 16204 [ACK] Seq=316 Ack=159 Win=3971 Len=0
102	91.587719	200.1.1.1	200.1.1.10	TCP	60	22215 → 80 [SYN] Seq=0 Win=4128 Len=0 MSS=1460
103	91.596062	200.1.1.10	200.1.1.1	TCP	60	80 → 22215 [SYN, ACK] Seq=0 Ack=1 Win=4128 Len=0 MSS=1460
104	91.596765	200.1.1.1	200.1.1.10	TCP	60	22215 → 80 [ACK] Seq=1 Ack=1 Win=4128 Len=0
105	91.597308	200.1.1.1	200.1.1.10	TCP	1514	22215 → 80 [ACK] Seq=1 Ack=1 Win=4128 Len=1460 [TCP segment of a reassembled PDU]
106	91.606136	200.1.1.10	200.1.1.1	TCP	60	80 → 22215 [ACK] Seq=1 Ack=1461 Win=2668 Len=0
107	91.606197	200.1.1.10	200.1.1.1	TCP	60	[TCP Window Update] 80 → 22215 [ACK] Seq=1 Ack=1461 Win=4128 Len=0
108	91.607833	200.1.1.1	200.1.1.10	HTTP	418	GET /cgi-bin/pkiclient.exe?operation=PKIOperation&message=MIIEoQYJKoZIhvcNAQcCoIIEkjCCBI4CAQExCzAJBgUrDgMCGgUAMIIBIAYJ…
109	91.616244	200.1.1.10	200.1.1.1	TCP	310	80 → 22215 [ACK] Seq=1 Ack=1825 Win=3764 Len=256 [TCP segment of a reassembled PDU]
110	91.652295	1.1.1.1	2.2.2.2	ISAKMP	210	Identity Protection (Main Mode)
111	91.666685	2.2.2.2	1.1.1.1	ISAKMP	150	Identity Protection (Main Mode)
112	91.670409	1.1.1.1	2.2.2.2	ISAKMP	414	Identity Protection (Main Mode)
113	91.697017	2.2.2.2	1.1.1.1	ISAKMP	434	Identity Protection (Main Mode)
114	91.729434	200.1.1.10	200.1.1.1	HTTP	1043	HTTP/1.1 200 OK (application/x-pki-message)
115	91.734258	1.1.1.1	2.2.2.2	ISAKMP	742	Identity Protection (Main Mode)
116	91.735353	200.1.1.1	200.1.1.10	TCP	60	22215 → 80 [ACK] Seq=1825 Ack=1247 Win=6755 Len=0
117	91.736353	200.1.1.1	200.1.1.10	TCP	60	22215 → 80 [FIN, PSH, ACK] Seq=1825 Ack=1247 Win=6755 Len=0
118	91.737934	200.1.1.10	200.1.1.1	TCP	60	80 → 22215 [ACK] Seq=1247 Ack=1826 Win=3764 Len=0
119	91.859141	1.1.1.1	2.2.2.2	ISAKMP	742	Identity Protection (Main Mode)
120	92.115710	2.2.2.2	1.1.1.1	ISAKMP	742	Identity Protection (Main Mode)
121	92.131452	1.1.1.1	2.2.2.2	ISAKMP	230	Quick Mode
122	92.146105	2.2.2.2	1.1.1.1	ISAKMP	230	Quick Mode
123	92.158653	1.1.1.1	2.2.2.2	ISAKMP	102	Quick Mode
124	92.159899	1.1.1.1	2.2.2.2	ESP	150	ESP (SPI=0xaa824a3a)
125	92.177022	2.2.2.2	1.1.1.1	ESP	150	ESP (SPI=0x5451d9b8)
126	92.178695	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0xaa824a3a)
127	92.198041	2.2.2.2	1.1.1.1	ESP	150	ESP (SPI=0x5451d9b8)
128	92.198163	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0x5451d9b8)
129	92.201255	1.1.1.1	2.2.2.2	ESP	150	ESP (SPI=0xaa824a3a)
130	92.201780	1.1.1.1	2.2.2.2	ESP	198	ESP (SPI=0xaa824a3a)
131	92.218377	2.2.2.2	1.1.1.1	ESP	198	ESP (SPI=0x5451d9b8)

Frame 29: 349 bytes on wire (2792 bits), 349 bytes captured (2792 bits) on interface -, id 0

Figure 17: SCEP-related HTTP traffic observed after router restart

The captured traffic includes HTTP requests and responses associated with certificate validation and certificate status retrieval operations.

These SCEP exchanges occur because the routers must re-establish trust information after reboot and verify the validity of the installed certificates before using them for IPSec authentication.

The observed messages serve several purposes:

- Verification that the CA is reachable
- Retrieval or validation of CA information
- Validation of certificate status
- Synchronization of trust-related information

This mechanism ensures that routers do not use invalid, expired, or revoked certificates during IKEv1 authentication.

The SCEP communication therefore plays an essential role in maintaining the integrity and trustworthiness of the PKI infrastructure used for IPSec VPN authentication.

3.9 Certificate Revocation and Authentication Failure

The certificates issued to R1 and R2 were revoked at the Certification Authority using:

```
RA#crypto pki server myCA revoke 02
RA#crypto pki server myCA revoke 03
```

After revocation, all routers were restarted and Wireshark captures were collected on the links between the routers and the CA.

When attempting communication between PC1 and PC2, the ping operation failed and the IPSec tunnel could not be established.

3.9.1 SCEP and Certificate Validation Traffic

After reboot, HTTP/SCEP-related messages were observed between the routers and the CA.

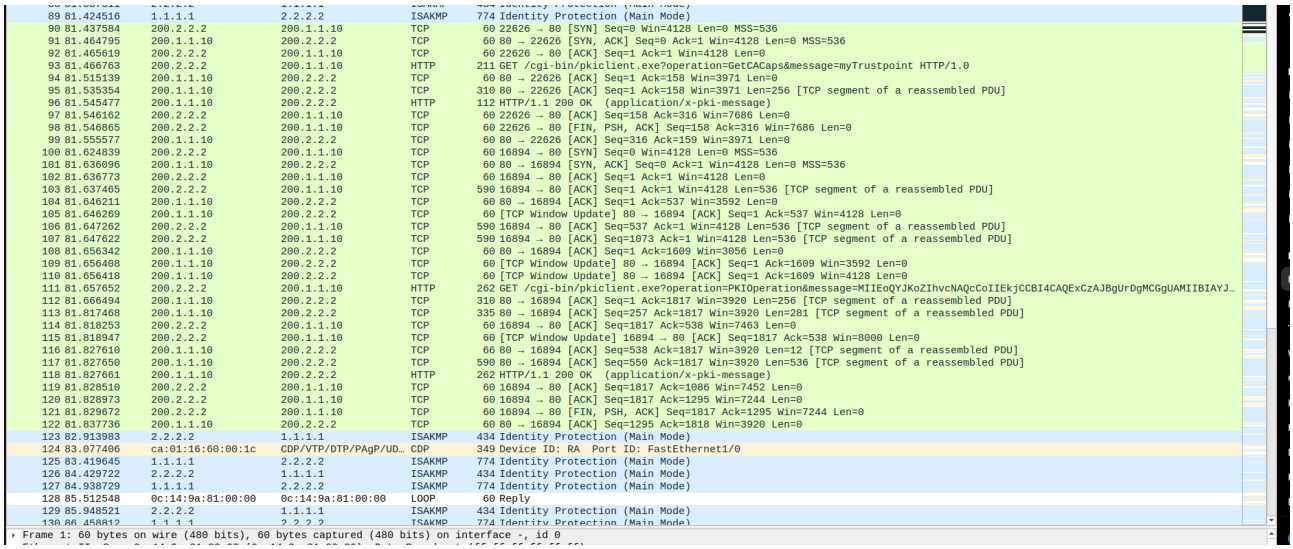


Figure 18: SCEP-related traffic observed after certificate revocation

These messages are associated with certificate validation and trust verification operations performed before IPSec authentication.

The routers contact the CA to verify the validity of certificates and retrieve updated revocation information.

3.9.2 Failed ISAKMP Negotiation

While the routers attempted to establish the IPSec tunnel, repeated ISAKMP exchanges were observed.

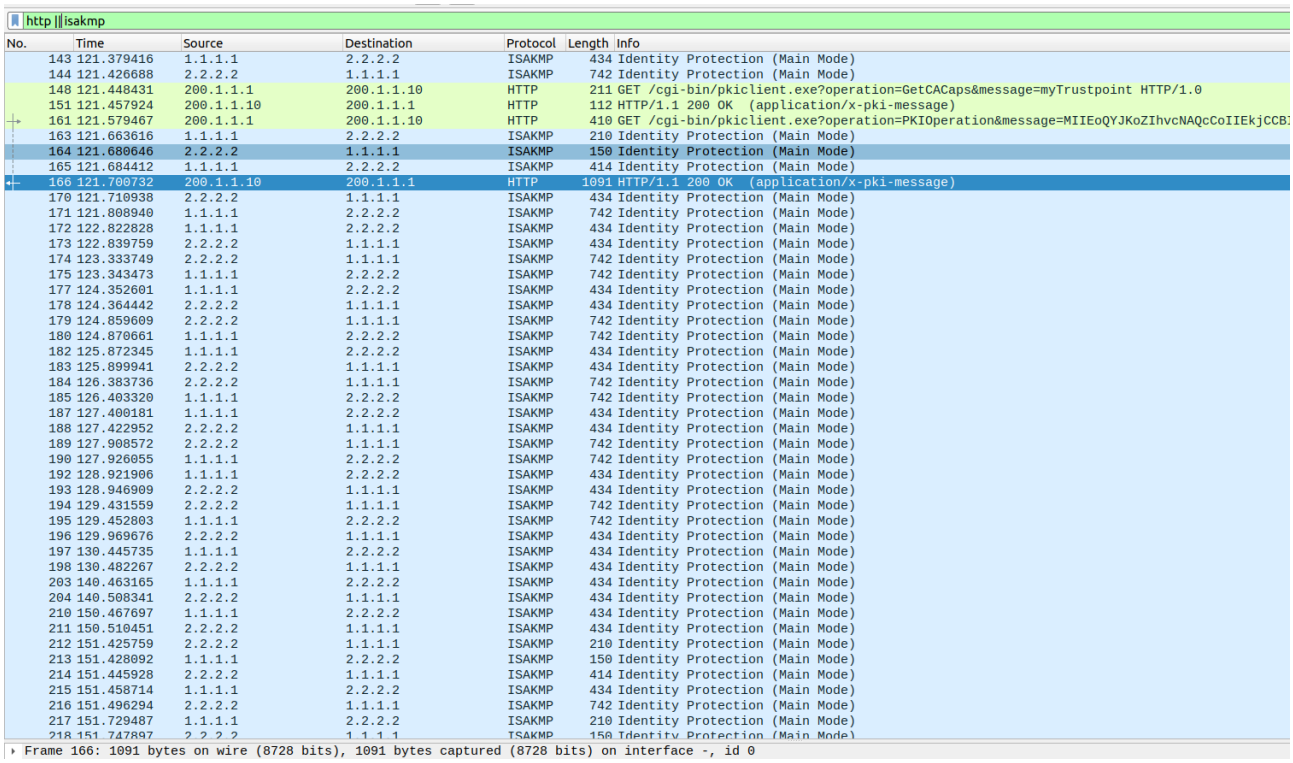


Figure 19: Repeated ISAKMP negotiation attempts after certificate revocation

The repeated ISAKMP packets indicate that the routers continuously attempt to establish an IKEv1 Security Association but fail during the authentication phase.

This behavior was confirmed using IOS verification commands.

```
R1#sh crypto isakmp sa
IPv4 Crypto ISAKMP SA
dst src state conn-id status
1.1.1.1 2.2.2.2 MM_KEY_EXCH 1040 ACTIVE
1.1.1.1 2.2.2.2 MM_KEY_EXCH 1038 ACTIVE
1.1.1.1 2.2.2.2 MM_NO_STATE 1036 ACTIVE (deleted)
1.1.1.1 2.2.2.2 MM_NO_STATE 1034 ACTIVE (deleted)
2.2.2.2 1.1.1.1 MM_KEY_EXCH 1041 ACTIVE
2.2.2.2 1.1.1.1 MM_NO_STATE 1039 ACTIVE (deleted)
2.2.2.2 1.1.1.1 MM_NO_STATE 1037 ACTIVE (deleted)

IPv6 Crypto ISAKMP SA
```

The ISAKMP state remains in `MM_KEY_EXCH`, indicating that Main Mode negotiation does not complete successfully.

A successfully established tunnel would normally reach the `QM_IDLE` state.

3.9.3 Absence of IPSec Security Associations

The IPSec SA database confirms that no operational IPSec tunnel was created.

```
R1#sh crypto ipsec sa

interface: Tunnel0
  Crypto map tag: Tunnel0-head-0, local addr 1.1.1.1

  protected vrf: (none)
  local ident (addr/mask/prot/port): (0.0.0.0/0.0.0.0/0/0)
  remote ident (addr/mask/prot/port): (0.0.0.0/0.0.0.0/0/0)
  current_peer 2.2.2.2 port 500
   PERMIT, flags={origin_is_acl,} #pkts encaps: 0,
  #pkts encrypt: 0, #pkts digest: 0 #pkts decaps: 0,
  #pkts decrypt: 0, #pkts verify: 0 #pkts compressed: 0,
  #pkts decompressed: 0 #pkts not compressed: 0,
  #pkts compr. failed: 0 #pkts not decompressed: 0,
  #pkts decompress failed: 0
  #send errors 0, #recv errors 0

  local crypto endpt.: 1.1.1.1, remote crypto endpt.: 2.2.2.2
  plaintext mtu 1500, path mtu 1500, ip mtu 1500, ip mtu idb
GigabitEthernet0/0
  current outbound spi: 0x0(0)
  PFS (Y/N): N, DH group: none

  inbound esp sas:

  inbound ah sas:

  inbound pcp sas:

  outbound esp sas:

  outbound ah sas:

  outbound pcp sas:

R1#
```

The counters remain at zero because IPSec encryption was never activated.

Additionally, no ESP Security Associations were present:

```
inbound esp sas:
outbound esp sas:
```

This confirms that Phase 2 IPSec negotiation could not be completed.

3.9.4 Explanation of Communication Failure

Communication fails because the certificates used by R1 and R2 were revoked by the Certification Authority.

After reboot, the routers verify certificate validity during IKEv1 authentication. The CA indicates that the peer certificates are no longer trusted because they are listed in the Certificate Revocation List (CRL).

As a result:

- RSA signature authentication fails
- IKEv1 Main Mode negotiation cannot complete
- ISAKMP Security Associations are not successfully established
- IPSec Security Associations are never created
- ESP encrypted traffic cannot be generated
- ICMP communication between PC1 and PC2 fails

The experiment demonstrates the importance of certificate revocation in PKI-based IPSec infrastructures, ensuring that compromised or invalid certificates cannot be used to establish trusted VPN connections.

4 IPSec with IKEv2

4.1 Introduction

This section describes the configuration and analysis of an IPSec VPN using IKEv2 authentication with pre-shared keys. The objective is to establish a protected tunnel between routers R1 and R2 through the intermediate router RA and analyze the establishment and operation of IKEv2 and IPSec Security Associations (SAs).

4.2 IKEv2 and IPSec Configuration Verification

The IKEv2 and IPSec configuration was verified on both R1 and R2 using IOS verification commands.

The following elements were validated:

- IKEv2 proposals defining AES-CBC-256 encryption, SHA512 integrity, SHA512 PRF, and Diffie-Hellman Group 16;

- IKEv2 policies associated with the configured proposals;
- IPSec transform sets and profiles used for tunnel protection;
- active IPSec Security Associations and encapsulation parameters.

Figures 20, 21, 22, and 23 present the verification outputs collected from both routers.

```
R1#show crypto ikev2 proposal
 IKEv2 proposal: default
     Encryption : AES-CBC-256 AES-CBC-192 AES-CBC-128
     Integrity  : SHA512 SHA384 SHA256 SHA96 MD596
     PRF        : SHA512 SHA384 SHA256 SHA1 MD5
     DH Group   : DH_GROUP_1536_MODP/Group 5 DH_GROUP_1024_MODP/Group 2
 IKEv2 proposal: myProposal
     Encryption : AES-CBC-256
     Integrity  : SHA512
     PRF        : SHA512
     DH Group   : DH_GROUP_4096_MODP/Group 16
R1#
```

```
R2#show crypto ikev2 proposal
 IKEv2 proposal: default
     Encryption : AES-CBC-256 AES-CBC-192 AES-CBC-128
     Integrity  : SHA512 SHA384 SHA256 SHA96 MD596
     PRF        : SHA512 SHA384 SHA256 SHA1 MD5
     DH Group   : DH_GROUP_1536_MODP/Group 5 DH_GROUP_1024_MODP/Group 2
 IKEv2 proposal: myProposal
     Encryption : AES-CBC-256
     Integrity  : SHA512
     PRF        : SHA512
     DH Group   : DH_GROUP_4096_MODP/Group 16
R2#
```

R1 R2

Figure 20: IKEv2 proposal verification

```
R1#sh crypto ikev2 policy

 IKEv2 policy : default
      Match fvrf : any
      Match address local : any
      Proposal    : default

 IKEv2 policy : myIKEv2Policy
      Match fvrf  : global
      Match address local : any
      Proposal    : myProposal
R1#
```

```
R2#sh crypto ikev2 policy

 IKEv2 policy : default
      Match fvrf : any
      Match address local : any
      Proposal    : default

 IKEv2 policy : myIKEv2Policy
      Match fvrf  : global
      Match address local : any
      Proposal    : myProposal
R2#
```

R1 R2

Figure 21: IKEv2 policy verification

```
R1#show crypto ipsec profile
IPSEC profile default
        Security association lifetime: 4608000 kilobytes/3600 seconds
        Responder-Only (Y/N): N
        PFS (Y/N): N
        Mixed-mode : Disabled
        Transform sets={
                default:  { esp-aes esp-sha-hmac  } ,
        }

IPSEC profile myIPSecProfile
        IKEv2 Profile: myIKEv2Profile
        Security association lifetime: 4608000 kilobytes/3600 seconds
        Responder-Only (Y/N): N
        PFS (Y/N): N
        Mixed-mode : Disabled
        Transform sets={
                myTSet:  { esp-aes esp-sha-hmac  } ,
        }

R1#
```

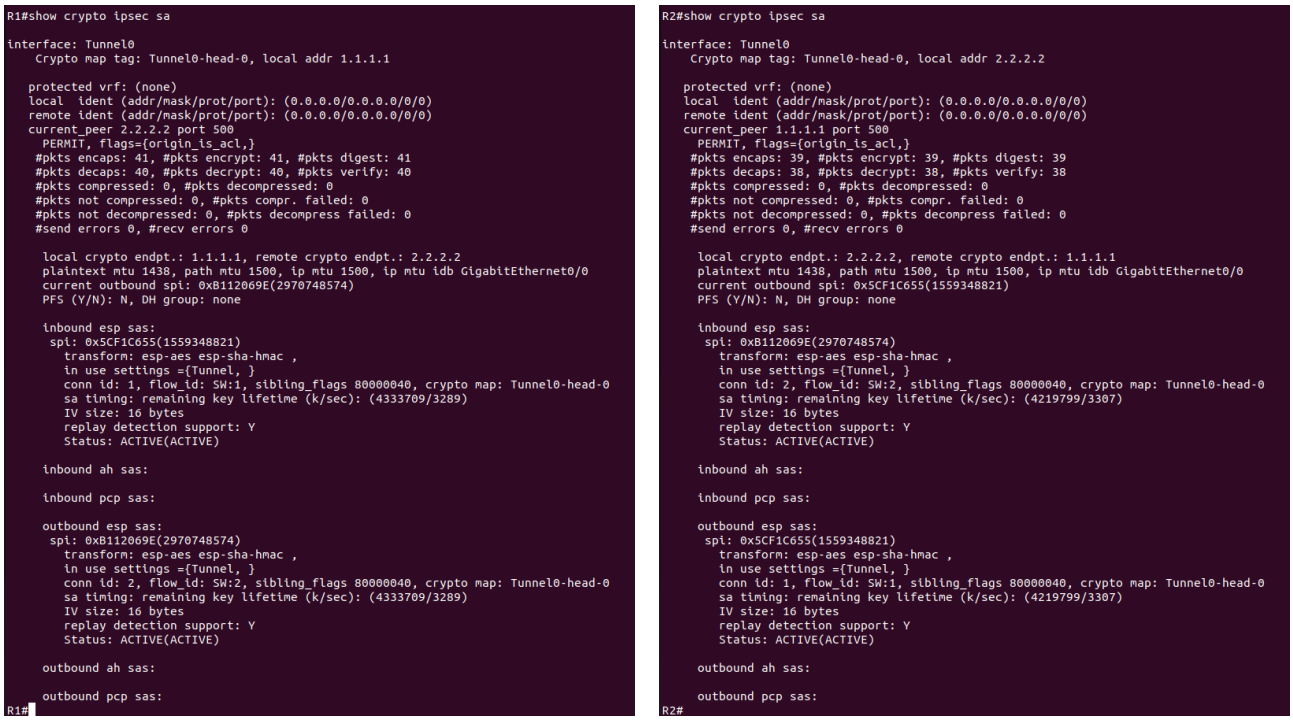
```
R2#show crypto ipsec profile
IPSEC profile default
        Security association lifetime: 4608000 kilobytes/3600 seconds
        Responder-Only (Y/N): N
        PFS (Y/N): N
        Mixed-mode : Disabled
        Transform sets={
                default:  { esp-aes esp-sha-hmac  } ,
        }

IPSEC profile myIPSecProfile
        IKEv2 Profile: myIKEv2Profile
        Security association lifetime: 4608000 kilobytes/3600 seconds
        Responder-Only (Y/N): N
        PFS (Y/N): N
        Mixed-mode : Disabled
        Transform sets={
                myTSet:  { esp-aes esp-sha-hmac  } ,
        }

R2#
```

R1 R2

Figure 22: IPSec profile verification



```
R1#show crypto ipsec sa

interface: Tunnel0
    Crypto map tag: Tunnel0-head-0, local addr 1.1.1.1

   protected vrf: (none)
   local  ident (addr/mask/prot/port): (0.0.0.0/0.0.0.0/0/0)
   remote ident (addr/mask/prot/port): (0.0.0.0/0.0.0.0/0/0)
   current_peer 2.2.2.2 port 500
     PERMIT, flags={origin_is_acl,}
    #pkts encaps: 41, #pkts encrypt: 41, #pkts digest: 41
    #pkts decaps: 40, #pkts decrypt: 40, #pkts verify: 40
    #pkts compressed: 0, #pkts decompressed: 0
    #pkts not compressed: 0, #pkts compr. failed: 0
    #pkts not decompressed: 0, #pkts decompress failed: 0
    #send errors 0, #recv errors 0

     local crypto endpt.: 1.1.1.1, remote crypto endpt.: 2.2.2.2
     plaintext mtu 1438, path mtu 1500, ip mtu 1500, ip mtu idb GigabitEthernet0/0
     current outbound spi: 0xB112069E(2970748574)
     PFS (Y/N): N, DH group: none

     inbound esp sas:
      spi: 0x5CF1C655(1559348821)
        transform: esp-aes esp-sha-hmac ,
        in use settings ={Tunnel, }
        conn id: 1, flow_id: SW:1, sibling_flags 80000040, crypto map: Tunnel0-head-0
        sa timing: remaining key lifetime (k/sec): (4333709/3289)
        IV size: 16 bytes
        replay detection support: Y
        Status: ACTIVE(ACTIVE)

     inbound ah sas:

     inbound pcp sas:

     outbound esp sas:
      spi: 0xB112069E(2970748574)
        transform: esp-aes esp-sha-hmac ,
        in use settings ={Tunnel, }
        conn id: 2, flow_id: SW:2, sibling_flags 80000040, crypto map: Tunnel0-head-0
        sa timing: remaining key lifetime (k/sec): (4333709/3289)
        IV size: 16 bytes
        replay detection support: Y
        Status: ACTIVE(ACTIVE)

     outbound ah sas:

     outbound pcp sas:
R1#
```

```
R2#show crypto ipsec sa

interface: Tunnel0
    Crypto map tag: Tunnel0-head-0, local addr 2.2.2.2

   protected vrf: (none)
   local  ident (addr/mask/prot/port): (0.0.0.0/0.0.0.0/0/0)
   remote ident (addr/mask/prot/port): (0.0.0.0/0.0.0.0/0/0)
   current_peer 1.1.1.1 port 500
     PERMIT, flags={origin_is_acl,}
    #pkts encaps: 39, #pkts encrypt: 39, #pkts digest: 39
    #pkts decaps: 38, #pkts decrypt: 38, #pkts verify: 38
    #pkts compressed: 0, #pkts decompressed: 0
    #pkts not compressed: 0, #pkts compr. failed: 0
    #pkts not decompressed: 0, #pkts decompress failed: 0
    #send errors 0, #recv errors 0

     local crypto endpt.: 2.2.2.2, remote crypto endpt.: 1.1.1.1
     plaintext mtu 1438, path mtu 1500, ip mtu 1500, ip mtu idb GigabitEthernet0/0
     current outbound spi: 0x5CF1C655(1559348821)
     PFS (Y/N): N, DH group: none

     inbound esp sas:
      spi: 0xB112069E(2970748574)
        transform: esp-aes esp-sha-hmac ,
        in use settings ={Tunnel, }
        conn id: 2, flow_id: SW:2, sibling_flags 80000040, crypto map: Tunnel0-head-0
        sa timing: remaining key lifetime (k/sec): (4219799/3307)
        IV size: 16 bytes
        replay detection support: Y
        Status: ACTIVE(ACTIVE)

     inbound ah sas:

     inbound pcp sas:

     outbound esp sas:
      spi: 0x5CF1C655(1559348821)
        transform: esp-aes esp-sha-hmac ,
        in use settings ={Tunnel, }
        conn id: 1, flow_id: SW:1, sibling_flags 80000040, crypto map: Tunnel0-head-0
        sa timing: remaining key lifetime (k/sec): (4219799/3307)
        IV size: 16 bytes
        replay detection support: Y
        Status: ACTIVE(ACTIVE)

     outbound ah sas:

     outbound pcp sas:
R2#
```

R1 R2

Figure 23: Active IPSec Security Associations

The verification outputs confirm that the tunnel was successfully established and that encrypted traffic was being encapsulated and decapsulated through ESP Security Associations.

4.3 Tunnel Establishment and Connectivity Test

After configuration, end-to-end connectivity between PC1 and PC2 was verified using ICMP echo requests.

Successful communication confirmed that:

- the IPSec tunnel was operational;
- encrypted traffic was correctly forwarded between endpoints;
- routing across the protected tunnel was functional.

Figure 24 shows successful ICMP communication between the end hosts through the IPSec tunnel.

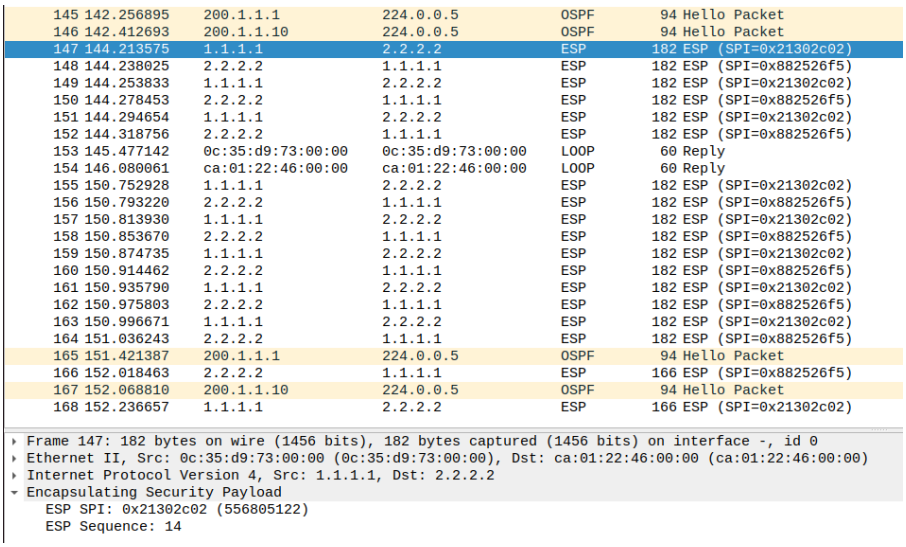


Figure 24: Successful ICMP connectivity between PC1 and PC2

4.4 Analysis of IKE_SA_INIT Exchange

Traffic captures were obtained on interface G0/0 of R1 during tunnel establishment. The IKE_SA_INIT exchange is responsible for:

- negotiation of cryptographic algorithms;
- exchange of Diffie-Hellman public values;
- generation of shared secret material;
- exchange of nonces;
- establishment of the initial IKE Security Association.

Figure **??** presents the captured IKEv2 exchange messages.

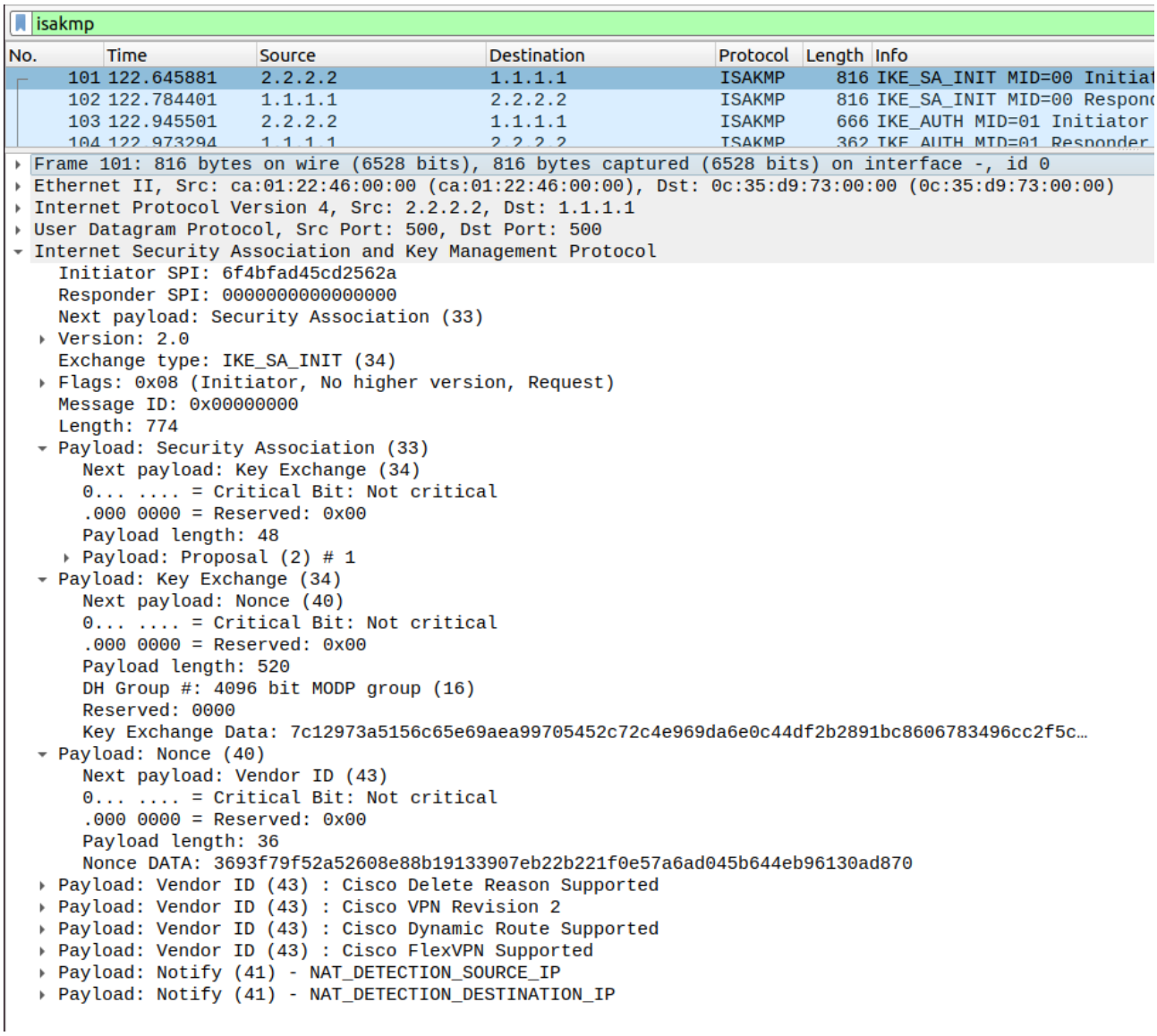


Figure 25: Contents of the IKE_SA_INIT exchange

```
Payload length: 40
▾ Payload: Proposal (2) # 1
    Next payload: NONE / No Next Payload  (0)
    Reserved: 00
    Payload length: 44
    Proposal number: 1
    Protocol ID: IKE (1)
    SPI Size: 0
    Proposal transforms: 4
  ▾ Payload: Transform (3)
      Next payload: Transform (3)
      Reserved: 00
      Payload length: 12
      Transform Type: Encryption Algorithm (ENCR) (1)
      Reserved: 00
      Transform ID (ENCR): ENCR_AES_CBC (12)
    ▸ Transform Attribute (t=14,l=2): Key Length: 256
  ▾ Payload: Transform (3)
      Next payload: Transform (3)
      Reserved: 00
      Payload length: 8
      Transform Type: Pseudo-random Function (PRF) (2)
      Reserved: 00
      Transform ID (PRF): PRF_HMAC_SHA2_512 (7)
  ▾ Payload: Transform (3)
      Next payload: Transform (3)
      Reserved: 00
      Payload length: 8
      Transform Type: Integrity Algorithm (INTEG) (3)
      Reserved: 00
      Transform ID (INTEG): AUTH_HMAC_SHA2_512_256 (14)
  ▾ Payload: Transform (3)
      Next payload: NONE / No Next Payload  (0)
      Reserved: 00
      Payload length: 8
      Transform Type: Diffie-Hellman Group (D-H) (4)
      Reserved: 00
      Transform ID (D-H): 4096 bit MODP group (16)
Payload: Key Exchange (34)
```

Figure 26: Content of the payload: Proposal in the IKE_SA_INIT packet

The capture shows the Security Parameter Indexes (SPIs), Security Association proposals, Diffie-Hellman key exchange payloads, and nonce payloads exchanged between both peers.

The negotiated proposal includes:

- AES-CBC-256 encryption;
- SHA2-512 integrity protection;
- SHA2-512 pseudo-random function (PRF);
- Diffie-Hellman Group 16.

The nonce payloads contribute entropy to the key generation process and ensure freshness of the exchange, protecting against replay attacks.

4.5 SA Deletion and Re-establishment

The command `clear crypto session` was executed to remove the active Security Associations and force a new negotiation process.

Wireshark captures show the deletion of the existing SAs followed by the establishment of new IKEv2 and IPSec Security Associations.

1	0.000000	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0x37bfac13)
2	1.087267	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0x3aa0a0ef)
3	1.968939	200.1.1.10	224.0.0.5	OSPF	94	Hello Packet
4	4.571026	200.1.1.1	224.0.0.5	OSPF	94	Hello Packet
5	5.111299	0c:35:d9:73:00:00	0c:35:d9:73:00:00	LOOP	60	Reply
6	5.470172	ca:01:16:60:00:00	ca:01:16:60:00:00	LOOP	60	Reply
7	7.109808	ca:01:16:60:00:00	CDP/VTP/DTP/PAgP/UD…	CDP	349	Device ID: RA Port ID: FastEthernet0/0
8	8.201714	1.1.1.1	2.2.2.2	ISAKMP	118	Informational
9	8.203333	1.1.1.1	2.2.2.2	ISAKMP	134	Informational
10	8.217971	2.2.2.2	1.1.1.1	ISAKMP	134	Informational
11	8.888812	1.1.1.1	2.2.2.2	ISAKMP	210	Identity Protection (Main Mode)
12	8.901438	2.2.2.2	1.1.1.1	ISAKMP	150	Identity Protection (Main Mode)
13	8.907525	1.1.1.1	2.2.2.2	ISAKMP	414	Identity Protection (Main Mode)
14	8.931954	2.2.2.2	1.1.1.1	ISAKMP	434	Identity Protection (Main Mode)
15	8.968150	1.1.1.1	2.2.2.2	ISAKMP	742	Identity Protection (Main Mode)
16	9.012585	2.2.2.2	1.1.1.1	ISAKMP	742	Identity Protection (Main Mode)
17	9.031615	1.1.1.1	2.2.2.2	ISAKMP	230	Quick Mode
18	9.054747	2.2.2.2	1.1.1.1	ISAKMP	230	Quick Mode
19	9.072084	1.1.1.1	2.2.2.2	ISAKMP	102	Quick Mode
20	9.073561	1.1.1.1	2.2.2.2	ESP	150	ESP (SPI=0x59df71ad)
21	9.084887	2.2.2.2	1.1.1.1	ESP	150	ESP (SPI=0x08155835)
22	9.087434	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0x59df71ad)
23	9.105426	2.2.2.2	1.1.1.1	ESP	150	ESP (SPI=0x08155835)
24	9.105619	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0x08155835)
25	9.109976	1.1.1.1	2.2.2.2	ESP	150	ESP (SPI=0x59df71ad)
26	9.110573	1.1.1.1	2.2.2.2	ESP	262	ESP (SPI=0x59df71ad)
27	9.125696	2.2.2.2	1.1.1.1	ESP	182	ESP (SPI=0x08155835)
28	9.127906	1.1.1.1	2.2.2.2	ESP	134	ESP (SPI=0x59df71ad)
29	9.128484	1.1.1.1	2.2.2.2	ESP	150	ESP (SPI=0x59df71ad)
30	9.145928	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0x08155835)
31	9.145991	2.2.2.2	1.1.1.1	ESP	134	ESP (SPI=0x08155835)
32	9.148997	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0x59df71ad)
33	11.649739	200.1.1.10	224.0.0.5	OSPF	94	Hello Packet
34	11.661379	1.1.1.1	2.2.2.2	ESP	150	ESP (SPI=0x59df71ad)
35	11.680534	2.2.2.2	1.1.1.1	ESP	150	ESP (SPI=0x08155835)
36	13.733941	1.1.1.1	2.2.2.2	ESP	182	ESP (SPI=0x59df71ad)
37	13.754607	2.2.2.2	1.1.1.1	ESP	182	ESP (SPI=0x08155835)
38	14.254637	200.1.1.1	224.0.0.5	OSPF	94	Hello Packet
39	15.160283	0c:35:d9:73:00:00	0c:35:d9:73:00:00	LOOP	60	Reply
40	15.468087	ca:01:16:60:00:00	ca:01:16:60:00:00	LOOP	60	Reply
41	16.248445	2.2.2.2	1.1.1.1	ESP	150	ESP (SPI=0x08155835)
42	16.264702	1.1.1.1	2.2.2.2	ESP	150	ESP (SPI=0x59df71ad)
43	18.238626	1.1.1.1	2.2.2.2	ESP	166	ESP (SPI=0x59df71ad)
44	18.409561	2.2.2.2	1.1.1.1	ESP	166	ESP (SPI=0x08155835)
45	20.827957	200.1.1.10	224.0.0.5	OSPF	94	Hello Packet

▸ Frame 1: 166 bytes on wire (1328 bits), 166 bytes captured (1328 bits) on interface -, id 0

Figure 27: Deletion and re-establishment of IKEv2 Security Associations after clearing the crypto session, showing Informational deletion followed by new IKE_SA_INIT and IKE_AUTH exchanges

After the previous SAs were removed, new IKE_SA_INIT and IKE_AUTH exchanges were observed, resulting in the creation of new SPIs and new IPSec Security Associations.

The results confirm that IPSec tunnel establishment using IKEv2 was successfully achieved between R1 and R2.

The analysis of Wireshark captures demonstrated:

- negotiation of cryptographic algorithms during IKE_SA_INIT;
- exchange of Diffie-Hellman values and nonces;
- encrypted peer authentication during IKE_AUTH;

- successful creation of IPSec Security Associations;
- successful deletion and recreation of SAs after clearing the crypto session.

Cisco IOS verification commands further confirmed the operational state of the tunnel, the establishment of OSPF adjacency through Tunnel0, and the correct operation of encrypted ESP traffic.

4.6 IKEv2 Security Association Rekeying

To analyze the IKEv2 rekeying process, the lifetime of the IKEv2 Security Association was reduced to 180 seconds on R2 using the following configuration:

```
crypto ikev2 profile myIKEv2Profile
 lifetime 180
```

After applying the configuration, the existing Security Associations were cleared using the `clear crypto session` command, allowing a new IKEv2 SA to be established with the updated lifetime.

Wireshark captures were obtained during the rekeying process. Figure 28 presents the exchanged messages associated with IKEv2 rekeying.

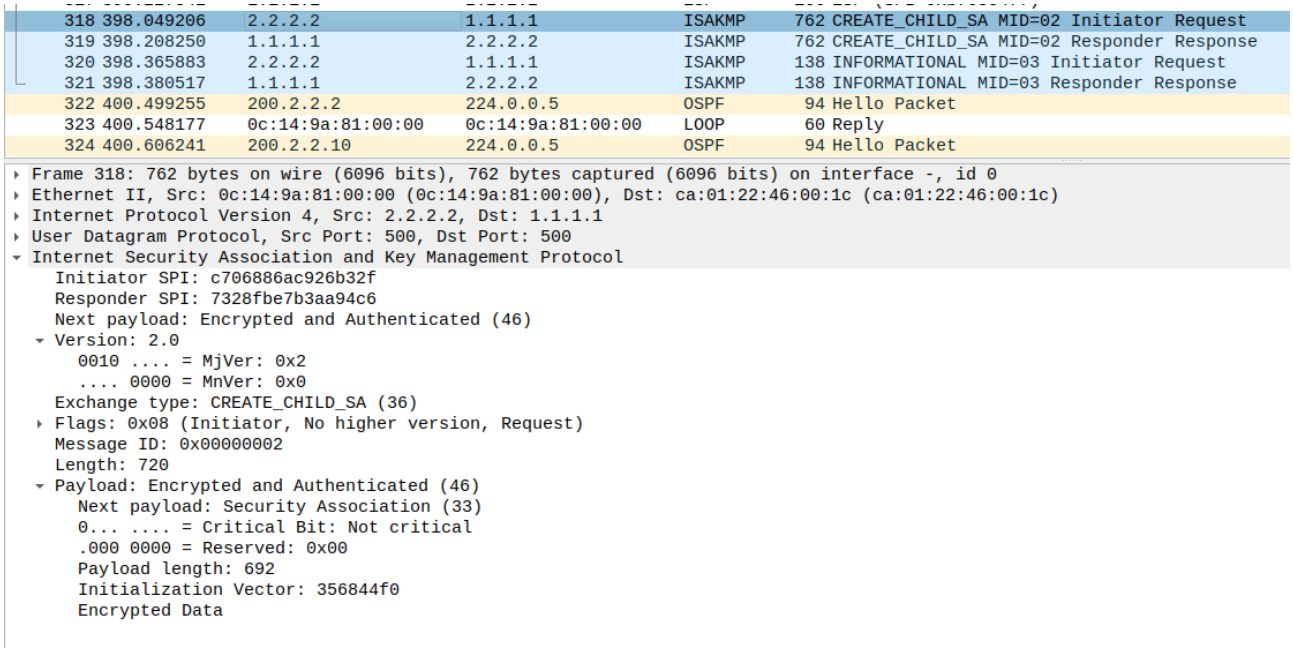


Figure 28: IKEv2 rekeying messages captured in Wireshark

The captures show that the rekeying procedure does not restart the tunnel establishment from scratch. No new IKE_SA_INIT exchange was observed during rekeying. Instead, the process occurs through protected exchanges within the existing secure channel.

The CREATE_CHILD_SA exchange is responsible for generating fresh key material and establishing a new Security Association while the previous one is still active. This allows rekeying to occur without interrupting tunnel connectivity.

The evolution of the IKEv2 Security Association was verified using the `show crypto ikev2 sa detailed` command. Figure 29 shows the Security Association state before and after rekeying.

```
R2#show crypto ikev2 sa detailed
 IPv4 Crypto IKEv2  SA

Tunnel-id Local                 Remote                fvrf/ivrf            Status
1         2.2.2.2/500           1.1.1.1/500           none/none            READY
      Encr: AES-CBC, keysize: 256, PRF: SHA512, Hash: SHA512, DH Grp:16, Auth sign: PSK, Auth ve
      Life/Active Time: 180/69 sec
      CE id: 1003, Session-id: 2
      Status Description: Negotiation done
      Local spi: C706886AC926B32F       Remote spi: 7328FBE7B3AA94C6
      Local id: 2.2.2.2
      Remote id: 1.1.1.1
      Local req msg id:  2              Remote req msg id:  0
      Local next msg id: 2              Remote next msg id: 0
      Local req queued:  2              Remote req queued:  0
      Local window:      5              Remote window:      5
      DPD configured for 0 seconds, retry 0
      Fragmentation not  configured.
      Dynamic Route Update: disabled
      Extended Authentication not configured.
      NAT-T is not detected
      Cisco Trust Security SGT is disabled
      Initiator of SA : Yes

 IPv6 Crypto IKEv2  SA

R2#show crypto ikev2 sa detailed
 IPv4 Crypto IKEv2  SA

Tunnel-id Local                 Remote                fvrf/ivrf            Status
6         2.2.2.2/500           1.1.1.1/500           none/none            READY
      Encr: AES-CBC, keysize: 256, PRF: SHA512, Hash: SHA512, DH Grp:16, Auth sign: PSK, Auth ve
      Life/Active Time: 180/89 sec
      CE id: 0, Session-id: 2
      Status Description: Negotiation done
      Local spi: E18E02B6B2633345       Remote spi: 96BE8FEFC1B8A5F0
      Local id: 2.2.2.2
      Remote id: 1.1.1.1
      Local req msg id:  0              Remote req msg id:  0
      Local next msg id: 0              Remote next msg id: 0
      Local req queued:  0              Remote req queued:  0
      Local window:      5              Remote window:      5
      DPD configured for 0 seconds, retry 0
      Fragmentation not  configured.
      Dynamic Route Update: disabled
      Extended Authentication not configured.
      NAT-T is not detected
      Cisco Trust Security SGT is disabled
      Initiator of SA : Yes

 IPv6 Crypto IKEv2  SA

R2#
```

Figure 29: Evolution of the IKEv2 Security Association during rekeying

The outputs show that rekeying generated new local and remote SPIs while preserving the negotiated cryptographic algorithms and parameters. Additionally, the SA lifetime was

refreshed after rekeying, confirming the successful establishment of a new IKEv2 Security Association.

The change in Tunnel-id, from 1 to 6, and SPI values demonstrates that the previous Security Association was replaced by a newly negotiated association.

4.7 IPSec Security Association Rekeying

To analyze the IPSec Security Association rekeying process, the IPSec SA lifetime was reduced to 120 seconds on R1 using the following configuration:

```
crypto ipsec profile myIPSecProfile
 set security-association lifetime seconds 120
```

After saving the configuration, router R1 was restarted to ensure that the updated lifetime was applied to newly established IPSec Security Associations.

Wireshark captures were obtained during the rekeying process. Figure 30 presents the exchanged messages associated with IPSec SA rekeying.

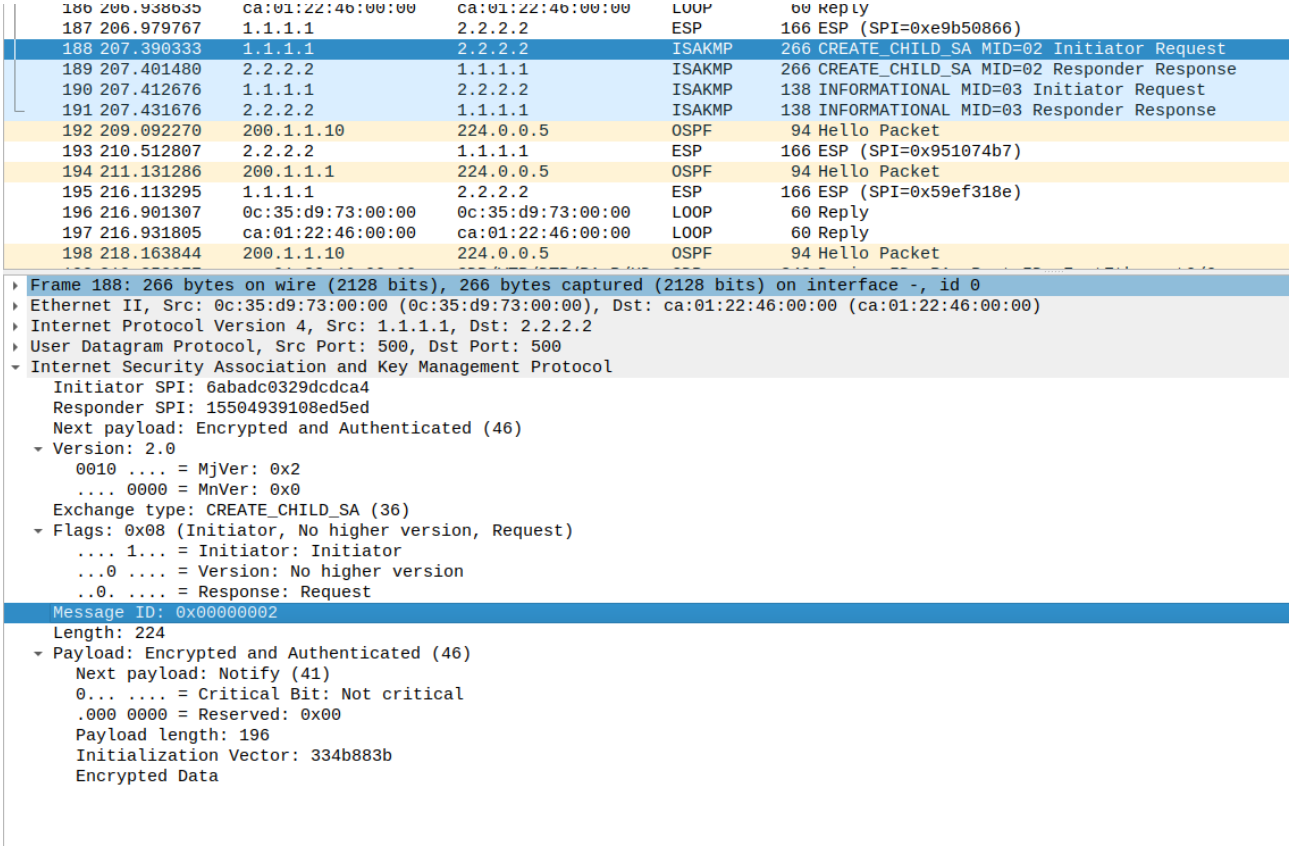
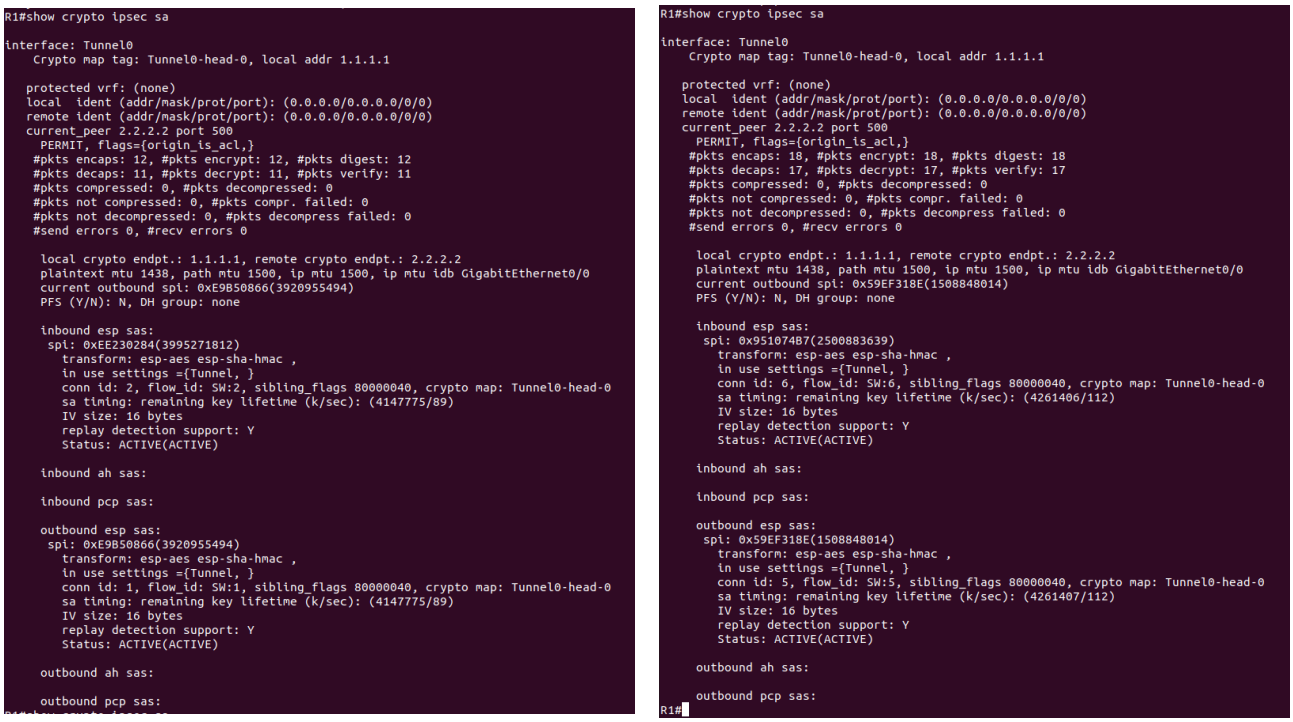


Figure 30: IPSec Security Association rekeying captured in Wireshark

Although both IKEv2 SA rekeying and IPSec SA rekeying use CREATE_CHILD_SA exchanges, the affected Security Associations differ. During IKEv2 SA rekeying, the IKE Security Association itself is replaced and new IKE SPIs are generated. During IPSec SA rekeying, only the ESP CHILD_SA is replaced while the IKEv2 Security Association remains active.

The evolution of the IPSec Security Associations was verified using the `show crypto ipsec sa` command. Figure 31 presents the Security Association state before and after rekeying.



Before rekeying

After rekeying

Figure 31: Evolution of the IPSec Security Associations during rekeying

The outputs show that the inbound and outbound ESP SPIs changed after rekeying, confirming that new IPSec Security Associations were created. Additionally, the Security Association lifetime was refreshed after rekeying while the negotiated cryptographic parameters remained unchanged.

The packet counters continued increasing throughout the process, demonstrating that encrypted traffic continued uninterrupted during the IPSec SA rekeying operation.

Unlike IKEv2 Security Association rekeying, which replaces the IKE SA itself, IPSec rekeying only replaces the CHILD_SA used for ESP traffic protection while preserving the already established IKEv2 Security Association.

5 DMVPN Phase 3

The objective of this experiment was to configure and analyse a DMVPN Phase 3 network using RIP, NHRP redirects, and NHRP shortcuts. The behaviour of DMVPN Phase 3 was compared with DMVPN Phase 1 and Phase 2 using Cisco IOS commands and Wireshark captures.

5.1 Topology

Figure 32 shows the network topology used in the experiment. Router R1 operated as the DMVPN hub, while R2 and R3 operated as spokes. The public underlay uses OSPF, while the overlay private network uses RIPv2.

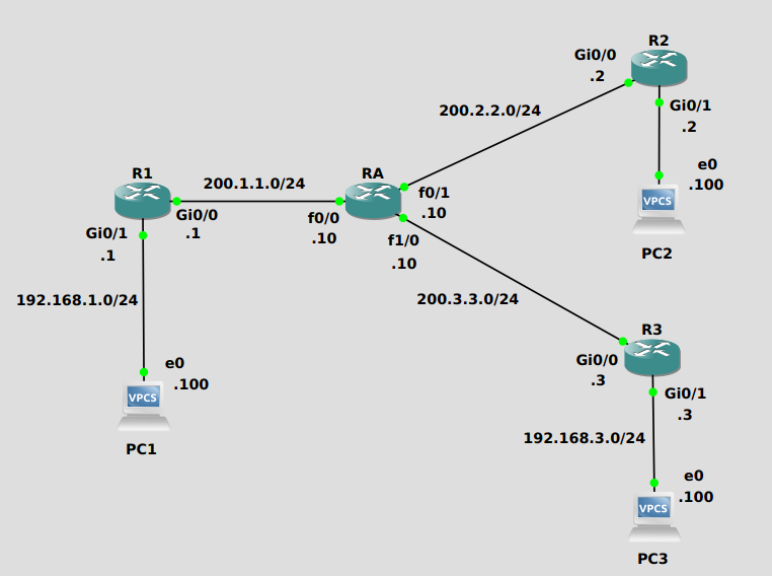


Figure 32: DMVPN Phase 3 topology

5.2 DMVPN Phase 3 Configuration

DMVPN Phase 3 was configured using mGRE tunnel interfaces on all routers, RIPv2 in the overlay, NHRP redirects (`ip nhrp redirect`) on the hub, and NHRP shortcuts (`ip nhrp shortcut`) on the spokes, combined with route summarisation via `ip summary-address rip 0.0.0.0 0.0.0.0` on the hub's tunnel interface.

5.3 Role of `ip summary-address rip`

This command causes R1 to advertise only a default route (`0.0.0.0/0`) to all spokes via RIP, instead of individual spoke subnets. Figure 33 confirms that R2 learns only the default route via `10.10.10.1` and not `192.168.3.0/24` directly.

```
R2#sh ip route rip
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       o - ODR, P - periodic downloaded static route, H - NHRP, l - LISP
       a - application route
       + - replicated route, % - next hop override, p - overrides from PfR

Gateway of last resort is 10.10.10.1 to network 0.0.0.0

R*    0.0.0.0/0 [120/1] via 10.10.10.1, 00:00:12, Tunnel0
R2#sh ip nhrp
10.10.10.1/32 via 10.10.10.1
   Tunnel0 created 00:06:45, never expire
   Type: static, Flags:
   NBMA address: 1.1.1.1
```

Figure 33: Default route learned through RIP at R2

Because spokes only hold a default route toward the hub, all initial spoke-to-spoke traffic transits R1, giving it the opportunity to issue NHRP redirects. When the command was temporarily removed, R1 advertised specific spoke subnets and R2 routed directly to R3 — NHRP redirects were never triggered, confirming the command is a prerequisite for Phase 3 operation.

5.4 Role of RIP Split Horizon

RIP split horizon prevents R1 from re-advertising routes learned from one spoke back out the same mGRE tunnel interface toward other spokes. Without it, R2 and R3 would learn each other's subnets via the hub and route directly, bypassing R1 entirely before NHRP shortcuts could be created. When split horizon was disabled with `no ip split-horizon` on R1's tunnel interface, specific spoke routes propagated to all spokes and no shortcuts were ever established, confirming that split horizon and `ip summary-address rip` must both be active for Phase 3 to work correctly.

5.5 OSPF vs RIP at the Public Network

Two distinct routing protocol traffic types are visible at the public network links. OSPF Hello packets carry outer destination `224.0.0.5` and are sourced from physical underlay interfaces — they are not GRE-encapsulated and maintain adjacencies solely in the underlay. RIP Response packets, by contrast, are GRE-encapsulated and carry outer loopback-to-loopback addresses (e.g., `1.1.1.1` → `2.2.2.2`), travelling inside the mGRE overlay to distribute private subnet reachability between sites. OSPF is never aware of the private subnets, and RIP is never visible on the underlay.

5.6 NHRP Redirects and Shortcuts

Initially R2 only knows the hub's NBMA address. When PC2 pings PC3, R2 forwards the first ICMP Echo Request to R1. R1 simultaneously forwards it to R3 and sends an NHRP

Traffic Indication back to R2, identifying `10.10.10.3` as a destination reachable via a better path. Figure 34 shows these Traffic Indication packets captured at the public network.

169	276.503534	1.1.1.1	2.2.2.2	NHRP	122	NHRP Traffic Indication
172	276.525054	1.1.1.1	3.3.3.3	NHRP	122	NHRP Traffic Indication
173	276.533183	2.2.2.2	1.1.1.1	NHRP	110	NHRP Resolution Request, ID=2
174	276.535825	1.1.1.1	3.3.3.3	NHRP	130	NHRP Resolution Request, ID=2
175	276.553515	3.3.3.3	1.1.1.1	NHRP	110	NHRP Resolution Request, ID=2
176	276.554695	1.1.1.1	2.2.2.2	NHRP	130	NHRP Resolution Request, ID=2

Figure 34: NHRP Traffic Indication packets generated by R1

R2 then sends an NHRP Resolution Request to resolve R3's NBMA address, carrying its own source NBMA (`2.2.2.2`) and protocol (`10.10.10.2`) addresses. R3 replies with its NBMA address (`3.3.3.3`). Figure 35 shows this exchange, and Figure 36 shows the resulting dynamic shortcut entry installed at R2.

79	92.269870	192.168.2.100	192.168.3.100	ICMP	122	Echo (ping) request id=0xb4ae, seq=1/256, ttl=63 (reply in 82)
80	92.284426	1.1.1.1	2.2.2.2	NHRP	122	NHRP Traffic Indication
81	92.295118	2.2.2.2	1.1.1.1	NHRP	110	NHRP Resolution Request, ID=2
82	92.304876	192.168.3.100	192.168.2.100	ICMP	122	Echo (ping) reply id=0xb4ae, seq=1/256, ttl=62 (request in 79)
83	92.325231	3.3.3.3	2.2.2.2	NHRP	158	NHRP Resolution Reply, ID=2, Code=Success
84	92.335360	1.1.1.1	2.2.2.2	NHRP	130	NHRP Resolution Request, ID=2
85	92.338501	2.2.2.2	3.3.3.3	NHRP	158	NHRP Resolution Reply, ID=2, Code=Success

Figure 35: NHRP Resolution Request and Reply

```
R2#sh dmvpn
Legend: Attrb --> S - Static, D - Dynamic, I - Incomplete
        N - NATed, L - Local, X - No Socket
        T1 - Route Installed, T2 - Nexthop-override
        C - CTS Capable, I2 - Temporary
        # Ent --> Number of NHRP entries with same NBMA peer
        NHS Status: E --> Expecting Replies, R --> Responding, W --> Waiting
        UpDn Time --> Up or Down Time for a Tunnel
==========================================================================

Interface: Tunnel0, IPv4 NHRP Details
Type:Spoke, NHRP Peers:2,

 # Ent  Peer NBMA Addr Peer Tunnel Add State  UpDn Tm Attrb
 ----- --------------- --------------- ----- -------- -----
     1 1.1.1.1             10.10.10.1    UP 00:08:35     S
     2 3.3.3.3             10.10.10.3    UP 00:07:57   DT1
                           10.10.10.3    UP 00:07:57   DT1
```

Figure 36: Dynamic DMVPN shortcut entry at R2

All subsequent packets from PC2 to PC3 are sent directly to R3's NBMA address, as confirmed by the traceroute in Figure 37.

5.7 Traffic Analysis

```
PC2> trace 192.168.3.100
trace to 192.168.3.100, 8 hops max, press Ctrl+C to stop
 1   192.168.2.2   0.651 ms  0.613 ms  0.557 ms
 2   10.10.10.3   14.728 ms  19.898 ms  19.887 ms
 3   *192.168.3.100   19.725 ms (ICMP type:3, code:3, Destination port unreachab
le)
```

Figure 37: Traceroute between spokes after shortcut creation

The traceroute confirms that after the NHRP shortcut is established, packets travel directly between R2 and R3 without transiting R1.

5.8 Comparison Between DMVPN Phases

Phase 1 uses point-to-point hub-to-spoke tunnels. All spoke-to-spoke traffic must transit R1 permanently — simple to configure but does not scale.

Phase 2 introduces mGRE on all routers. NHRP resolution allows direct spoke-to-spoke tunnels, but the routing protocol must preserve the original next-hop, requiring per-spoke routing entries on every spoke.

Phase 3 decouples routing from NHRP via summarisation. Spokes need only a single default route toward the hub; NHRP redirects and shortcuts build direct paths on demand. Adding a new spoke requires no routing table changes on existing spokes, making Phase 3 the most scalable of the three.

5.9 AI Comparison

The AI correctly identified that `ip summary-address rip` injects a default route from hub to spokes and that NHRP redirects trigger spoke-to-spoke shortcut creation.

Two discrepancies were found experimentally. First, the AI described RIP split horizon as preventing routing loops in general, without identifying the specific Phase 3 consequence: that disabling it causes spokes to learn direct routes and bypass the hub before shortcuts are created. This was confirmed by the split-horizon experiment above. Second, the AI stated that the hub sends the NHRP redirect *before* forwarding the packet. The Wireshark captures show the redirect and the forwarded packet occurring simultaneously, not sequentially.

6 VRFs and BGP-MPLS L3VPNs

6.1 Introduction

This laboratory exercise explored the implementation of a BGP/MPLS Layer 3 VPN using GNS3 and Cisco IOS routers. Two customer organizations (Blue and Red), each with two geographically separated sites, were interconnected through a provider MPLS backbone composed of PE and P routers. Virtual Routing and Forwarding (VRF) instances were used to isolate customer routing information, while MP-BGP distributed VPN routes and MPLS labels

between PE routers. The objective of the exercise was to analyze how MPLS labels, VRFs, and MP-BGP cooperate to provide scalable and isolated Layer 3 VPN connectivity across a shared provider infrastructure.

6.2 Ping Between Sites and Loopback Sourcing

Connectivity between the sites of each organization was verified by executing sourced pings between the CE routers. Figure 38 presents the successful ping from Blue1 to Blue2, and Figure 39 presents the successful ping from Red1 to Red2.

```
B1#ping 192.168.102.1 source l0

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.102.1, timeout is 2 seconds:
Packet sent with a source address of 192.168.101.1
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 52/56/60 ms
B1#
```

Figure 38: Successful ping from Blue1 to Blue2

```
R1#ping 192.168.112.1 source l0

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.112.1, timeout is 2 seconds:
Packet sent with a source address of 192.168.111.1
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 40/42/48 ms
R1#
```

Figure 39: Successful ping from Red1 to Red2

The pings are sourced from the loopback interfaces because these are the prefixes advertised into the VPN through the static routes redistributed into MP-BGP. The physical CE interface addresses (e.g. `10.10.1.100`) are local to the PE–CE segment and are never injected into the VRF routing table of the remote PE. A ping sourced from the physical interface would reach the destination but the reply would be dropped at the remote PE for lack of a return route. Sourcing from the loopback tests the actual end-to-end VPN path across the advertised prefixes.

6.3 PE1 Routing Tables

The routing architecture of PE1 was analyzed by inspecting the global routing table and the per-VRF routing tables. Figure 40 presents the global table, and Figures 41 and 42 present the VRF tables for the blue and red organizations respectively.

```
PE1#sh ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       o - ODR, P - periodic downloaded static route, H - NHRP, l - LISP
       + - replicated route, % - next hop override

Gateway of last resort is not set

      1.0.0.0/32 is subnetted, 1 subnets
C        1.1.1.1 is directly connected, Loopback0
      2.0.0.0/32 is subnetted, 1 subnets
O        2.2.2.2 [110/12] via 222.222.1.10, 00:22:13, FastEthernet0/0
      222.222.1.0/24 is variably subnetted, 2 subnets, 2 masks
C        222.222.1.0/24 is directly connected, FastEthernet0/0
L        222.222.1.1/32 is directly connected, FastEthernet0/0
O     222.222.2.0/24 [110/11] via 222.222.1.10, 00:22:13, FastEthernet0/0
PE1#
```

Figure 40: Global routing table of PE1, containing only provider infrastructure routes

The global routing table contains exclusively provider infrastructure routes: the directly connected provider links (`222.222.x.x`), the PE and P loopbacks learned via OSPF, and no customer prefixes whatsoever. This reflects the fundamental VRF isolation principle, customer routes exist only within their respective VRF instances and are completely invisible to the global forwarding plane.

```
PE1#sh ip route vrf blue

Routing Table: blue
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       o - ODR, P - periodic downloaded static route, H - NHRP, l - LISP
       + - replicated route, % - next hop override

Gateway of last resort is not set

      10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C        10.10.1.0/24 is directly connected, FastEthernet1/1
L        10.10.1.1/32 is directly connected, FastEthernet1/1
S     192.168.101.0/24 [1/0] via 10.10.1.100
B     192.168.102.0/24 [200/0] via 2.2.2.2, 00:21:26
PE1#
```

Figure 41: Blue vrf routing table in PE1

```
PE1#sh ip route vrf red

Routing Table: red
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       o - ODR, P - periodic downloaded static route, H - NHRP, l - LISP
       + - replicated route, % - next hop override

Gateway of last resort is not set

      11.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C        11.11.1.0/24 is directly connected, FastEthernet1/0
L        11.11.1.1/32 is directly connected, FastEthernet1/0
S     192.168.111.0/24 [1/0] via 11.11.1.100
B     192.168.112.0/24 [200/0] via 2.2.2.2, 00:21:32
PE1#
```

Figure 42: Red vrf routing table in PE1

The blue VRF routing table contains three categories of entries. The directly connected entry for `10.10.1.0/24` corresponds to the PE1–Blue1 segment. The static route for `192.168.101.0/24` via `10.10.1.100` was manually configured to reach Blue1's loopback and is redistributed into MP-BGP. The BGP route for `192.168.102.0/24` via `2.2.2.2` was learned from PE2 through the MP-BGP session, with PE2's loopback serving as the next hop resolved through the MPLS core. The red VRF table is symmetric, containing the equivalent entries for the red organization's address space.

The two organizations are isolated from each other because each VRF constitutes an independent routing and forwarding instance. Interfaces are explicitly bound to a single VRF, so incoming packets are looked up exclusively within that VRF's FIB. Furthermore, route import and export is governed by the Route Target extended community: the blue VRF is configured with `route-target both 1:1` and the red VRF with `route-target both 2:2`, meaning a route advertised by one organization is never imported into the other's VRF regardless of address overlap.

6.4 MPLS Labels During Ping Analysis

To analyze the MPLS label stack used within the provider network, Wireshark captures were obtained on both the PE1–P and P–PE2 links while executing `ping 192.168.102.1 source 10` from Blue1. Figures 43 and 44 present the captured packets on each segment respectively.

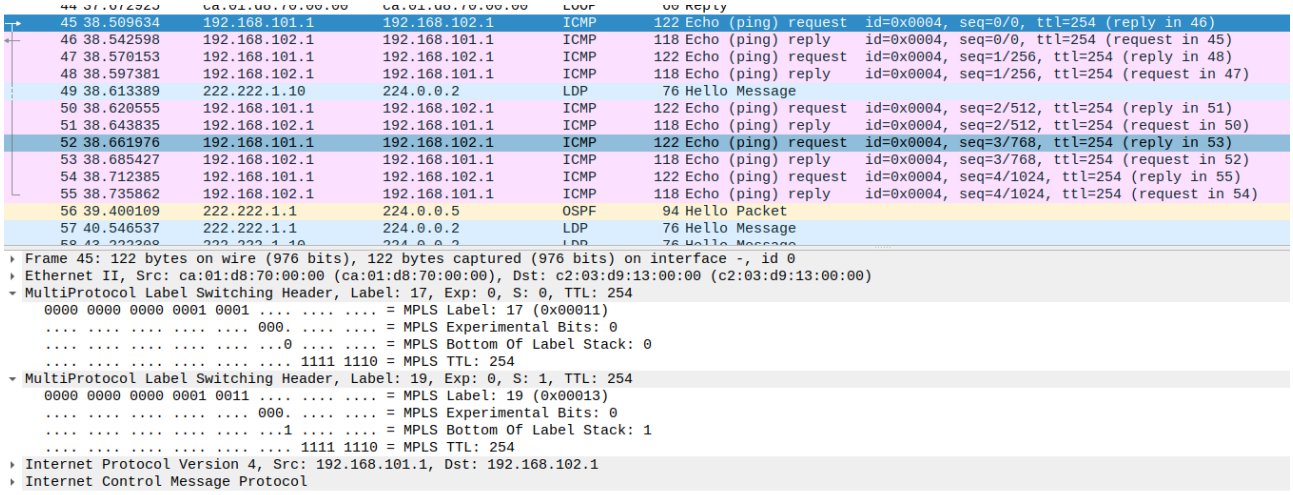


Figure 43: Message captured between PE1 and P, displaying 2 labels

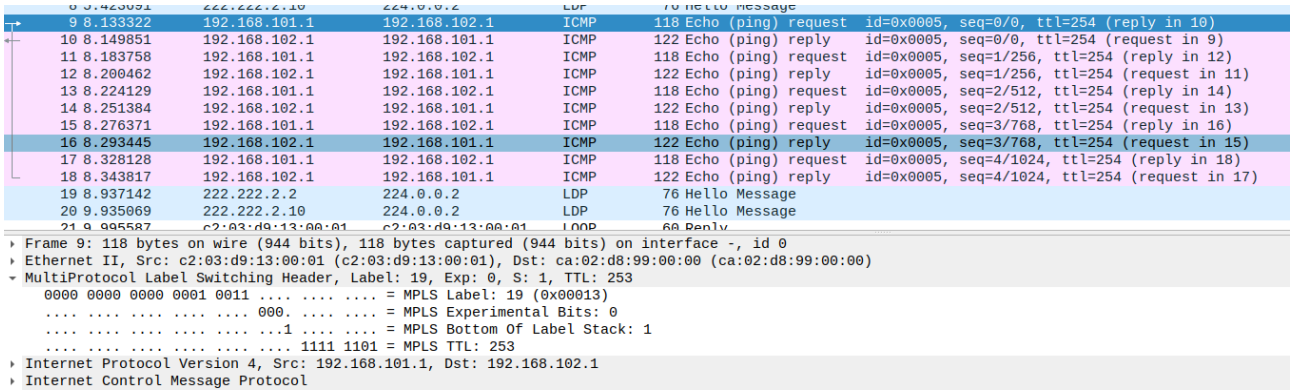


Figure 44: Message captured between P-PE2 displaying only 1 label

On the PE1–P segment, each ICMP packet carries a two-label MPLS stack. The outer label (value 17) is the transport label, distributed by LDP, and is used exclusively to switch the packet across the provider core toward PE2's loopback `2.2.2.2`. Its Bottom of Stack (S) bit is set to 0, indicating that additional labels follow beneath it in the stack. The inner label (value 19) is the VPN label, distributed by MP-BGP, and identifies the destination VRF at the egress PE. Its S bit is set to 1, marking it as the last label in the stack, after which the original IP packet begins.

It is important to note that the VPN label value 19 was not assigned by PE1, the ingress router, but rather by PE2, the egress router. In MPLS VPN, each PE assigns labels for the prefixes it advertises and communicates them to the remote PE via MP-BGP Update messages. PE2 assigned label 19 for prefix `192.168.102.0/24` and advertised it to PE1, which then imposes that label on all traffic destined for that prefix. This is confirmed by `show bgp vpnv4 unicast vrf blue 192.168.102.0` on PE1 shown in Figure 45, where the outgoing label field shows the value received from PE2. The ingress PE is therefore always responsible for imposing a label that was assigned and advertised by the egress PE.

```
PE1#show bgp vpnv4 unicast vrf blue 192.168.102.0
BGP routing table entry for 1:1:192.168.102.0/24, version 6
Paths: (1 available, best #1, table blue)
  Not advertised to any peer
  Local
    2.2.2.2 (metric 12) from 2.2.2.2 (2.2.2.2)
      Origin incomplete, metric 0, localpref 100, valid, internal, best
      Extended Community: RT:1:1
      mpls labels in/out nolabel/19
PE1#
```

Figure 45: BGP VPNv4 entry for prefix 192.168.102.0/24 at PE1, showing outgoing VPN label assigned by PE2

These label values are directly correlated with the MPLS forwarding table of PE1, shown in Figure 46. The entry for `2.2.2.2/32` shows outgoing label 17 via `FastEthernet0/0`, confirming that PE1 imposes the LDP transport label 17 to reach PE2's loopback through the P router. The VPN entry for `192.168.102.0/24[V]` confirms that traffic destined for the blue VRF prefix is forwarded with the label stack observed in the capture.

```
PE1#sh mpls forwarding-table
Local      Outgoing   Prefix           Bytes Label   Outgoing   Next Hop
Label      Label      or Tunnel Id     Switched      interface
17         17         2.2.2.2/32       0             Fa0/0      222.222.1.10
18         Pop Label  222.222.2.0/24   0             Fa0/0      222.222.1.10
19         No Label   192.168.101.0/24[V]   \
                                       2280          Fa1/1      10.10.1.100
20         No Label   192.168.111.0/24[V]   \
                                       1140          Fa1/0      11.11.1.100
PE1#
```

Figure 46: MPLS forwarding table of PE1, showing transport label 17 toward PE2 and VPN label entries

```
PE2#sh mpls forwarding-table
Local      Outgoing   Prefix           Bytes Label   Outgoing   Next Hop
Label      Label      or Tunnel Id     Switched      interface
17         16         1.1.1.1/32       0             Fa0/0      222.222.2.10
18         Pop Label  222.222.1.0/24   0             Fa0/0      222.222.2.10
19         No Label   192.168.102.0/24[V]   \
                                       2280          Fa1/0      10.10.2.100
20         No Label   192.168.112.0/24[V]   \
                                       1140          Fa1/1      11.11.2.100
PE2#
```

Figure 47: MPLS forwarding table of PE2, confirming PHP behavior and VPN label 19 mapping

On the P–PE2 segment, the packet carries only the VPN label (value 19), as shown in Figure 44. This results from Penultimate Hop Popping (PHP): the P router removes the outer

transport label before forwarding the packet to PE2. PHP is implemented through LDP by advertising the implicit-null label for directly connected prefixes. PE2's LDP bindings confirm this behavior, as the local binding for `2.2.2.2/32` is `imp-null`, instructing the P router to pop the transport label before delivery. Consequently, PE2 receives the packet with only the inner VPN label, avoiding an additional label lookup.

PE2 uses the VPN label to identify the destination VRF and forward the packet to the correct CE. This is confirmed by PE2's MPLS forwarding table in Figure 47, where label 19 maps to `192.168.102.0/24[V]`. The `[V]` annotation indicates a VRF route, and the outgoing label field shows `No Label`, meaning PE2 removes the VPN label and forwards the packet natively into the blue VRF toward `10.10.2.100`. Similarly, the outer transport label value 17 observed on the PE1–P link was assigned by the P router and advertised to PE1 through LDP.

The VPN label ensures isolation between organizations, even when overlapping IP address spaces are used. Since forwarding at the egress PE is based on the VPN label rather than the destination IP address, packets with identical IP destinations can be delivered to different VRFs without ambiguity. The outer transport label is swapped at each LSR hop across the provider core, while the inner VPN label remains unchanged from ingress PE to egress PE, where it is finally removed.

6.5 BGP Session Establishment and VPNv4 Route Advertisement

To analyze the BGP Open and Update messages exchanged between PE1 and PE2, interface `f0/0` of PE2 was shut down and re-enabled while a Wireshark capture was running on the PE1–P link, forcing a new BGP session to be established.

Figure 48 presents the BGP Open messages sent by PE1.

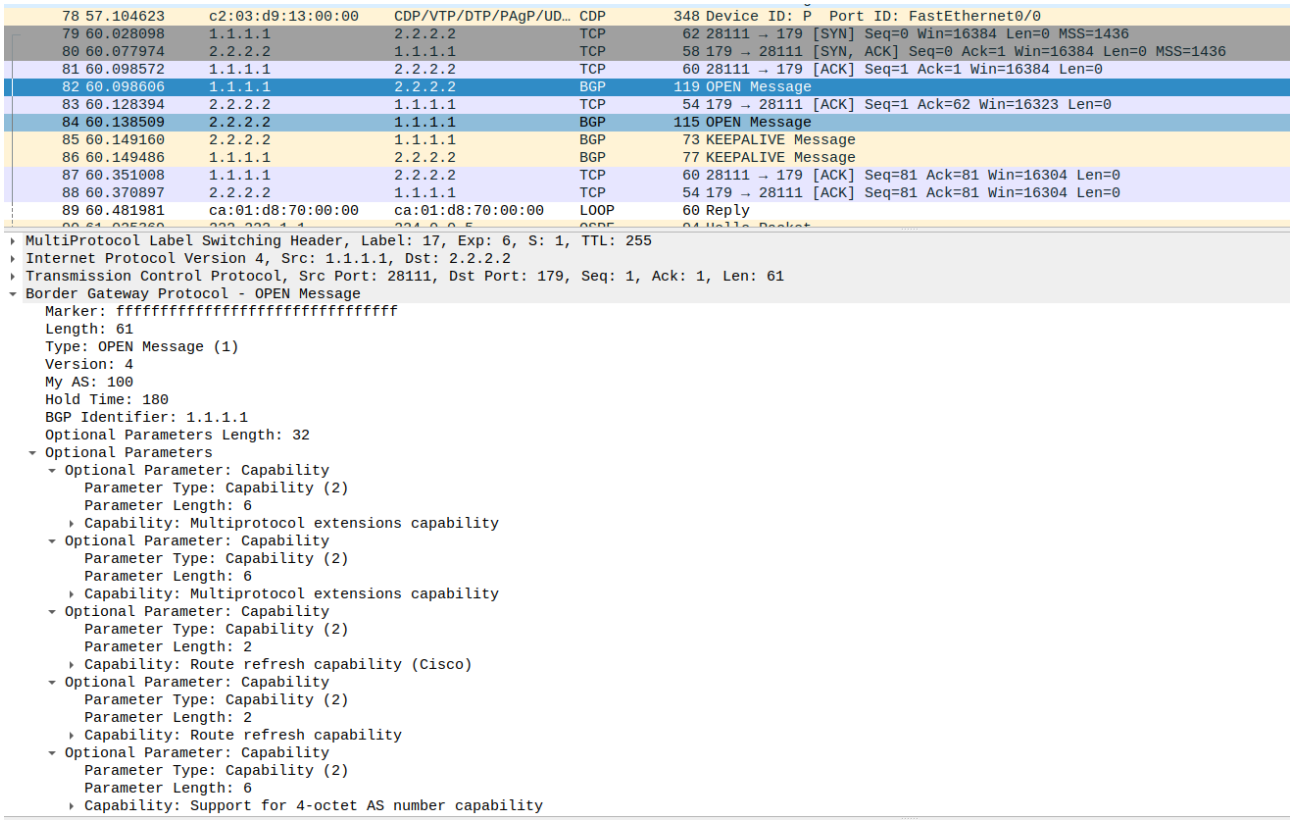


Figure 48: Open messages exchanged

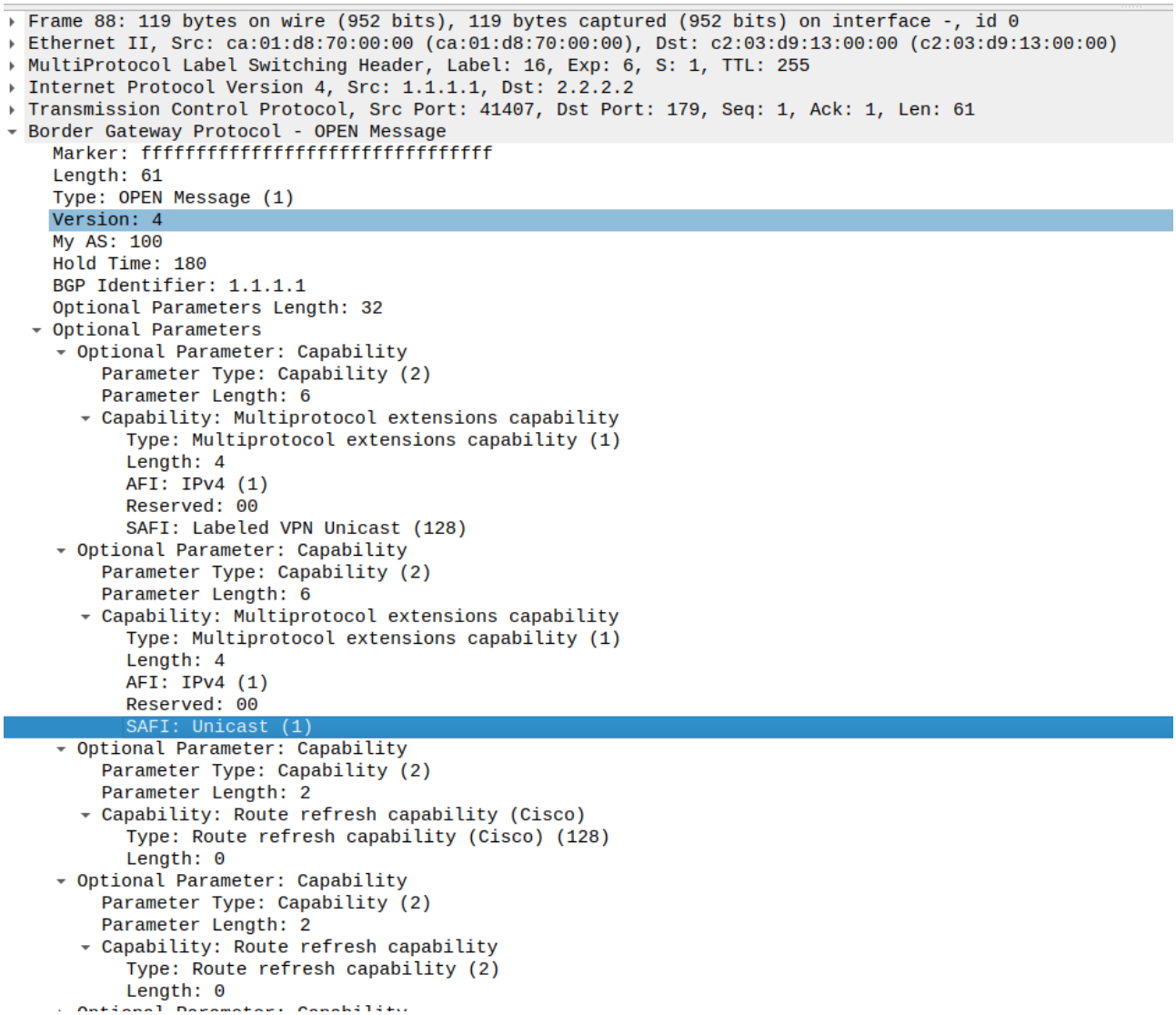


Figure 49: Content of the open message

The Open messages advertise the Multiprotocol Extensions capability (code 1) with `AFI=1` (IPv4) and `SAFI=128` (Labeled VPN Unicast). This is the critical capability that enables VPNv4 route exchange, standard BGP only supports IPv4 unicast (`SAFI=1`) and cannot carry prefixes with an attached Route Distinguisher or MPLS label. Without this capability being confirmed by both peers, no VPN routes can be signaled.

Figure 50 presents the BGP Update message sent by PE2 to PE1 advertising the blue organization's remote prefix. Two path attributes are of particular relevance, as shown in Figure 51. The `MP_REACH_NLRI` attribute (type 14) confirms the negotiated address family (`AFI=1, SAFI=128`) and carries the NLRI containing three elements: the Route Distinguisher `1:1` prepended to prefix `192.168.102.0`, forming the 96-bit VPNv4 address, and VPN label `19` at the bottom of the stack. This label value directly matches the inner label observed in the previous section's Wireshark captures and PE2's MPLS forwarding table entry for `192.168.102.0/24[V]`, confirming that VPN labels are distributed via MP-BGP and not LDP. The next hop is encoded as `RD=0:0, IPv4=2.2.2.2`, identifying PE2's loopback as the

reachability next hop.

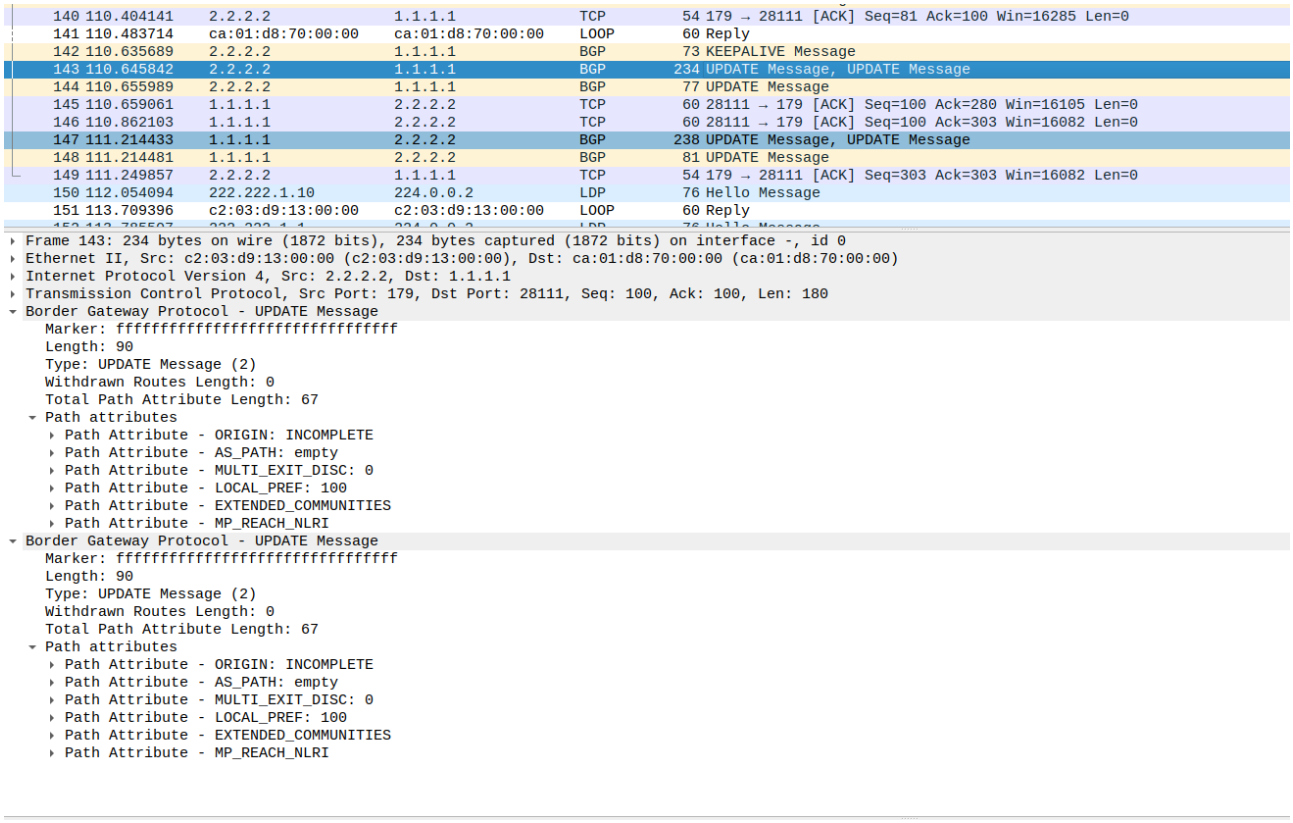


Figure 50: BGP Update messages exchanged between PE2 and PE1 advertising blue organization prefixes

Two additional capabilities are also negotiated: Route Refresh (type 2), which allows peers to request a full re-advertisement of the BGP table without resetting the session, and 4-octet AS number support (type 65), which permits AS numbers above 65535. Both are visible in Figure 49.

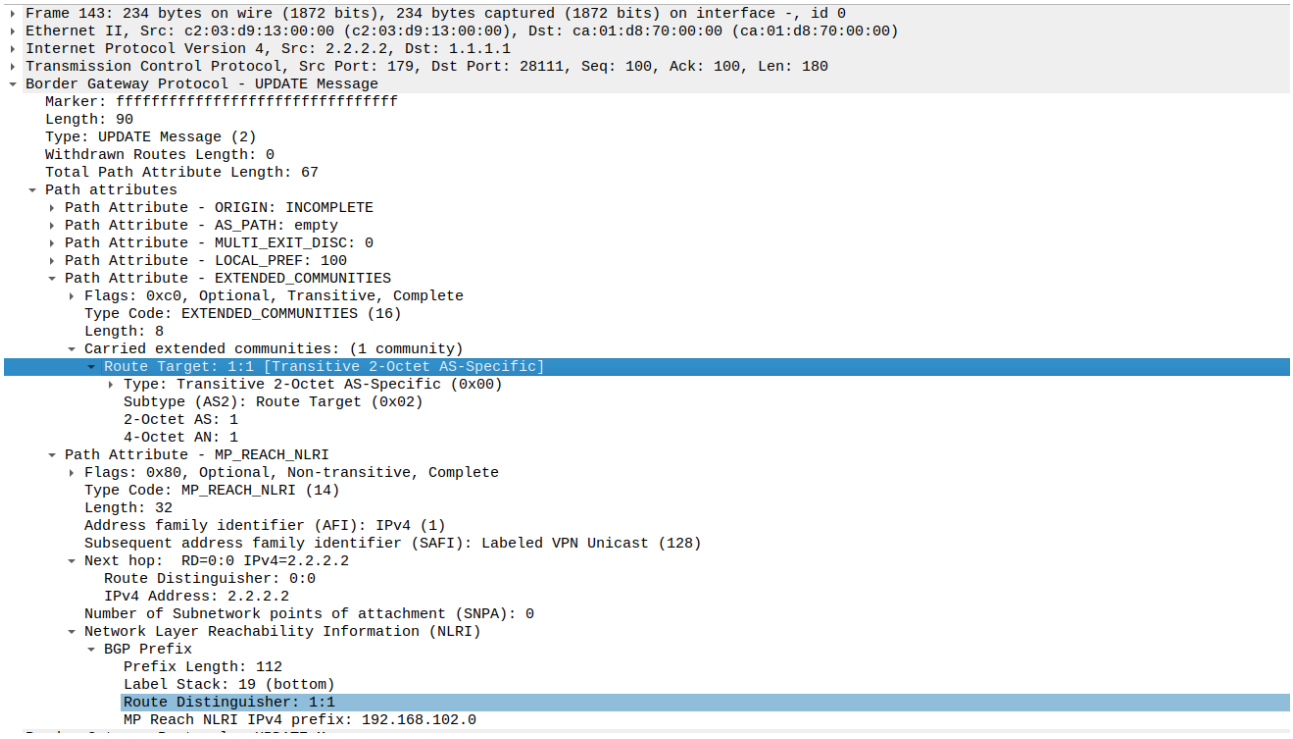


Figure 51: Contents of the BGP Update message, showing MP_REACH_NLRI with Route Distinguisher, VPN label, and EXTENDED_COMMUNITIES with Route Target RT:1:1

The `EXTENDED_COMMUNITIES` attribute (type 16) carries Route Target `RT:1:1`. Upon receiving this update, PE1 imports the prefix into the blue VRF because its import policy matches `RT:1:1`, while the red VRF, configured with `route-target import 2:2`, ignores it entirely. This enforces inter-organization isolation at the control plane, complementing the data plane isolation provided by VRFs.

A VPNv4 route is an IPv4 prefix extended with a 64-bit Route Distinguisher, producing a globally unique 96-bit BGP prefix. The RD ensures overlapping customer address spaces remain distinguishable — `1:1:192.168.102.0/24` and `2:2:192.168.102.0/24` are distinct BGP entries despite identical IPv4 portions. Crucially, the RD carries no policy semantics; routing policy is determined solely by the Route Target. Standard BGP cannot support this VPN solution because it simultaneously lacks the 64-bit RD prefix format, MPLS label encoding within the NLRI, and Extended Communities support for Route Target signaling.

7 Connecting network namespaces with IPIP tunnel

7.1 Introduction

The objective of this exercise was to establish communication between two Linux network namespaces located on different nodes using an IPIP tunnel. Two Ubuntu nodes were used: `ub1` and `ub2`. Each node contained one network namespace connected to the host through a bridge interface and a virtual Ethernet pair (veth). Communication between the namespaces was achieved by creating an IPIP tunnel between the two hosts and configuring appropriate routing rules.

A Linux network namespace is an isolated instance of the kernel's network stack, with its own interfaces, routing table, ARP table, and firewall rules, completely independent from the host and from other namespaces. It is the fundamental isolation primitive underlying Docker containers and other container runtimes.

7.2 Topology

Figure **??** shows the setup used in this exercise. Node `ub1` hosts namespace `blue` and node `ub2` hosts namespace `red`. Each namespace is connected to its host via a bridge interface and a veth pair. An IPIP tunnel (`tun0`) is established between the physical interfaces of the two hosts (`192.168.50.1` and `192.168.50.2`), and static routes on each host direct namespace traffic through the tunnel.

7.3 Configuration of the Namespaces and Tunnel

Two network namespaces were created, one on each node. Namespace `blue` was created on `ub1`, while namespace `red` was created on `ub2`.

7.3.1 Namespace Configuration

On `ub1`, the following commands were used:

```
ip netns add blue

ip link add br2 type bridge
ip link set br2 up

ip link add iblue type veth peer name ibluebr

ip link set iblue netns blue
ip link set ibluebr master br2
ip link set ibluebr up

ip -n blue addr add 10.0.2.2/24 dev iblue
ip -n blue link set iblue up
ip -n blue link set lo up

ip addr add 10.0.2.1/24 dev br2

ip netns exec blue ip route add default via 10.0.2.1
```

On `ub2`, the following commands were executed:

```
ip netns add red

ip link add br1 type bridge
ip link set br1 up
```

```
ip link add ired type veth peer name iredbr

ip link set ired netns red
ip link set iredbr master br1
ip link set iredbr up

ip -n red addr add 10.0.1.2/24 dev ired
ip -n red link set ired up
ip -n red link set lo up

ip addr add 10.0.1.1/24 dev br1

ip netns exec red ip route add default via 10.0.1.1
```

The bridge interfaces acted as gateways between the namespaces and the host operating systems.

7.3.2 Physical Interface Configuration

The physical interfaces connecting the two Ubuntu nodes were configured with IPv4 addresses:

- `ub1`: `192.168.50.1/24`
- `ub2`: `192.168.50.2/24`

The following commands were used:

```
ip addr add 192.168.50.1/24 dev ens3
```

on `ub1`, and

```
ip addr add 192.168.50.2/24 dev ens3
```

on `ub2`.

7.3.3 IPIP Tunnel Configuration

An IPIP tunnel was created between the two hosts.
On `ub1`:

```
ip tunnel add tun0 mode ipip \
local 192.168.50.1 remote 192.168.50.2

ip addr add 192.168.100.1/30 dev tun0
ip link set tun0 up
```

On ub2:

```
ip tunnel add tun0 mode ipip \
local 192.168.50.2 remote 192.168.50.1

ip addr add 192.168.100.2/30 dev tun0
ip link set tun0 up
```

The tunnel endpoints used the physical IP addresses of the hosts, while the tunnel interfaces themselves used the network 192.168.100.0/30.

7.3.4 Routing and IP Forwarding

IP forwarding was enabled on both hosts using:

```
sysctl -w net.ipv4.ip_forward=1
```

Static routes were then added so that traffic destined for the remote namespace network would be forwarded through the tunnel.

On ub1:

```
ip route add 10.0.1.0/24 via 192.168.100.2 dev tun0
```

On ub2:

```
ip route add 10.0.2.0/24 via 192.168.100.1 dev tun0
```

With these configurations, communication between the two namespaces was successfully established through the IPIP tunnel.

7.4 Verification of Connectivity

After configuring the namespaces, bridge interfaces, tunnel interfaces and routing tables, connectivity between the two namespaces was tested using ICMP echo requests.

The following command was executed from the namespace located on ub1:

```
ip netns exec blue ping 10.0.1.2
```

The ping was successful, confirming that packets were correctly routed through the IPIP tunnel between the two hosts.

Figure 52 shows the successful communication between the namespaces.

```
root@ubuntu-cloud:/home/ubuntu# ip netns exec blue ping 10.0.1.2
PING 10.0.1.2 (10.0.1.2) 56(84) bytes of data.
64 bytes from 10.0.1.2: icmp_seq=1 ttl=62 time=2.03 ms
64 bytes from 10.0.1.2: icmp_seq=2 ttl=62 time=1.44 ms
64 bytes from 10.0.1.2: icmp_seq=3 ttl=62 time=1.58 ms
64 bytes from 10.0.1.2: icmp_seq=4 ttl=62 time=1.43 ms
^C
--- 10.0.1.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 1.432/1.619/2.029/0.243 ms
root@ubuntu-cloud:/home/ubuntu#
```

Figure 52: Successful ping between namespaces

The routing tables were also verified to ensure that traffic destined for the remote namespace network was forwarded through the tunnel interface `tun0`.

Figure 53 shows the routing table and network interfaces configured on `ub2`.

```
root@ubuntu-cloud:/home/ubuntu# ip route
10.0.1.0/24 dev br1 proto kernel scope link src 10.0.1.1
10.0.2.0/24 via 192.168.100.1 dev tun0
192.168.50.0/24 dev ens3 proto kernel scope link src 192.168.50.2
192.168.100.0/30 dev tun0 proto kernel scope link src 192.168.100.2
root@ubuntu-cloud:/home/ubuntu# ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 0c:82:2a:71:00:00 brd ff:ff:ff:ff:ff:ff
    altname enp0s3
3: br1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DEFAULT group default qlen 1000
    link/ether c6:93:49:c3:c3:bf brd ff:ff:ff:ff:ff:ff
4: iredbr@if5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br1 state UP mode DEFAULT group default qlen 1000
    link/ether 0a:cf:6e:fe:ee:da brd ff:ff:ff:ff:ff:ff link-netns red
6: tunl0@NONE: <NOARP> mtu 1480 qdisc noop state DOWN mode DEFAULT group default qlen 1000
    link/ipip 0.0.0.0 brd 0.0.0.0
7: tun0@NONE: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1480 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/ipip 192.168.50.2 peer 192.168.50.1
```

Figure 53: Routing table and interfaces on ub2

7.5 Verification of IPIP Encapsulation

To verify that the communication between namespaces was effectively using IPIP tunneling, packets were captured on the physical interface connecting the two hosts using Wireshark.

The capture revealed that the packets contained two IP headers:

- an outer IP header containing the physical addresses of the hosts (`192.168.50.1` and `192.168.50.2`);
- an inner IP header containing the original namespace communication addresses (`10.0.2.2` and `10.0.1.2`).

This confirms that the original packets were encapsulated inside another IP packet before being transmitted through the physical network.

Figure 54 shows the Wireshark capture illustrating the double IP header generated by the IPIP tunnel.

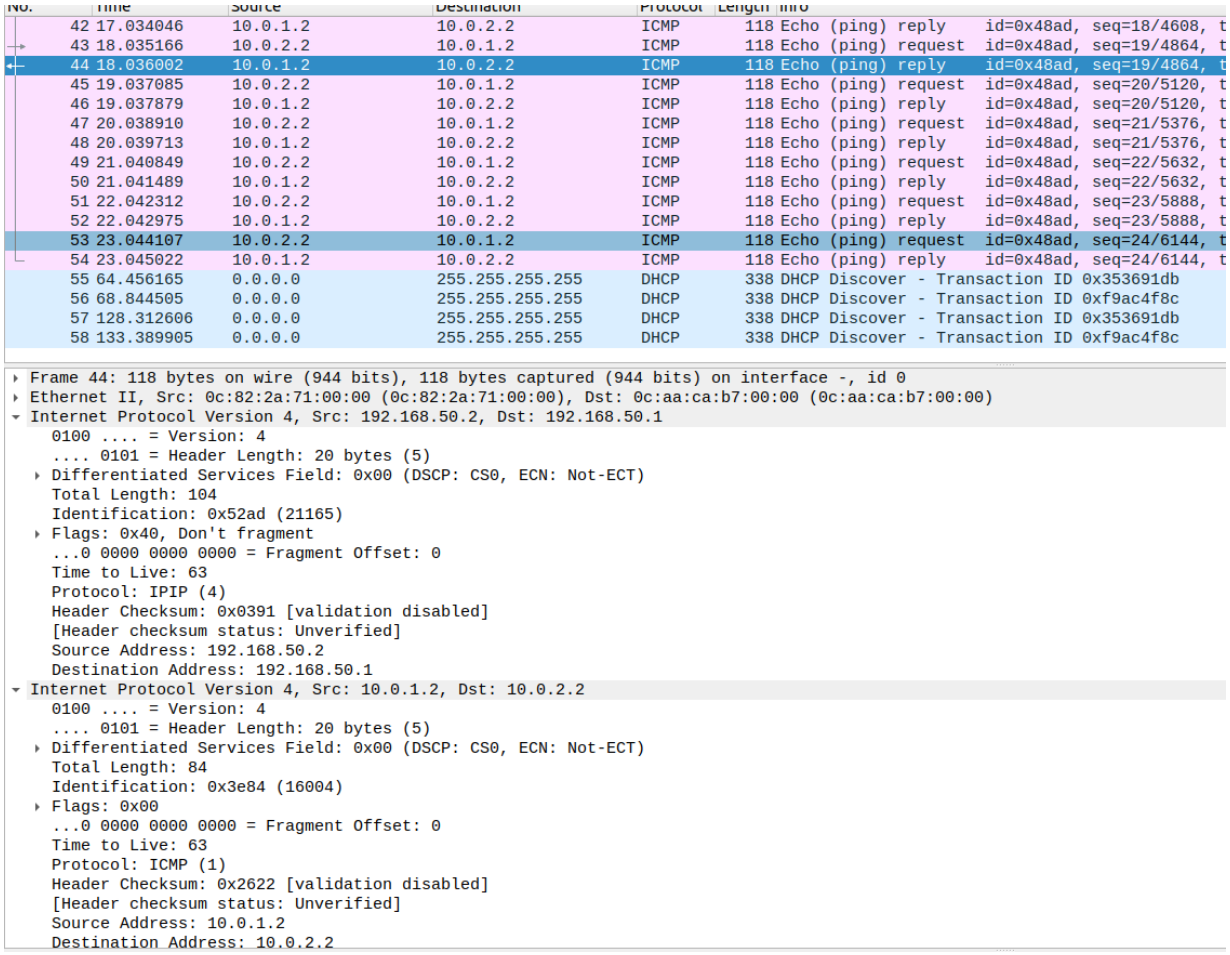


Figure 54: Wireshark capture showing IPIP encapsulation

7.6 Tunnel Verification

The tunnel configuration was verified using the command:

```
ip tunnel show
```

The output confirmed that the tunnel interface `tun0` was active and correctly configured with the local and remote endpoints.

Figure 55 shows the tunnel configuration on `ub2`.

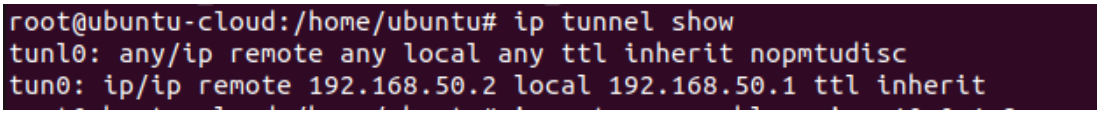


Figure 55: IPIP tunnel configuration

8 Macvlan Networks with VLAN Isolation

8.1 Introduction

The objective of this exercise was to configure isolated Docker macvlan networks using IEEE 802.1Q VLAN tagging. Two Ubuntu nodes, `ub1` and `ub2`, were connected through a Layer 2 Ethernet switch and configured with VLAN subinterfaces. Docker macvlan networks were then attached to these VLAN interfaces, allowing containers connected to the same VLAN to communicate while isolating traffic between different VLANs.

Two VLANs were configured:

- VLAN 10 using subnet `20.10.0.0/24`;
- VLAN 20 using subnet `20.20.0.0/24`.

Each Ubuntu node hosted one container connected to VLAN 10 and one container connected to VLAN 20. This allowed communication between containers attached to the same VLAN across different nodes, while preventing communication between containers attached to different VLANs.

The experiment demonstrates how Docker macvlan networks can be combined with IEEE 802.1Q VLAN tagging to provide Layer 2 isolation between groups of containers distributed across multiple hosts.

8.2 VLAN Interface Configuration

IEEE 802.1Q support was enabled on both Ubuntu nodes using the Linux `8021q` kernel module.

The following commands were executed on both `ub1` and `ub2`:

```
modprobe 8021q

ip link add link ens3 name ens3.10 type vlan id 10
ip link add link ens3 name ens3.20 type vlan id 20

ip link set ens3.10 up
ip link set ens3.20 up
```

These commands created two VLAN subinterfaces associated with the physical interface `ens3`. Interface `ens3.10` was associated with VLAN ID 10, while interface `ens3.20` was associated with VLAN ID 20.

No IP addresses were assigned to the VLAN subinterfaces, since they were used exclusively as Layer 2 parent interfaces for the Docker macvlan networks.

Figure 56 shows the interfaces configured on `ub1`.

```
root@ub1:/home/ubuntu# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 0c:e3:7e:a9:00:00 brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 20.0.0.1/24 brd 20.0.0.255 scope global ens3
       valid_lft forever preferred_lft forever
    inet6 fe80::ee3:7eff:fea9:0/64 scope link
       valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether a2:9b:68:e9:76:9b brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
4: ens3.20@ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 0c:e3:7e:a9:00:00 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::ee3:7eff:fea9:0/64 scope link
       valid_lft forever preferred_lft forever
5: ens3.10@ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 0c:e3:7e:a9:00:00 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::ee3:7eff:fea9:0/64 scope link
       valid_lft forever preferred_lft forever
root@ub1:/home/ubuntu#
```

Figure 56: Physical and VLAN interfaces configured on ub1

8.3 Macvlan Network Configuration

Two Docker macvlan networks were created on each node.

The following commands were executed:

```
docker network create -d macvlan \
--subnet=20.10.0.0/24 \
-o parent=ens3.10 mynet10

docker network create -d macvlan \
--subnet=20.20.0.0/24 \
-o parent=ens3.20 mynet20
```

The network `mynet10` was attached to VLAN interface `ens3.10`, while network `mynet20` was attached to VLAN interface `ens3.20`.

Figure 57 shows the Docker networks configured on `ub1`.

```
root@ub1:/home/ubuntu# docker network ls | grep -E "mynet"
5aaea801955d   mynet     macvlan   local
53cef27ae3f4   mynet10   macvlan   local
054a74190a0d   mynet20   macvlan   local
root@ub1:/home/ubuntu#
```

Figure 57: Docker macvlan networks configured on ub1

The detailed configuration of network `mynet10` was also verified using Docker inspection commands.

Figure 58 shows the configuration of `mynet10`.

```
root@ub1:/home/ubuntu# docker network inspect mynet10
[
    {
        "Name": "mynet10",
        "Id": "53cef27ae3f47f91b2f05aafe878a09658545171eacb7784368987926ae5f40e",
        "Created": "2026-05-27T18:44:23.734574626Z",
        "Scope": "local",
        "Driver": "macvlan",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "20.10.0.0/24"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {
            "parent": "ens3.10"
        },
        "Labels": {},
        "Containers": {},
        "Status": {
            "IPAM": {
                "Subnets": {
                    "20.10.0.0/24": {
                        "IPsInUse": 2,
                        "DynamicIPsAvailable": 254
                    }
                }
            }
        }
    }
]
root@ub1:/home/ubuntu#
```

Figure 58: Inspection of Docker macvlan network mynet10

8.4 Container Deployment

Containers were then attached to the corresponding VLAN networks.
On `ub1`, the following containers were created:

```
docker run --privileged --net=mynet10 \
--ip=20.10.0.10 --name=cont0 -itd alpine /bin/sh

docker run --privileged --net=mynet20 \
--ip=20.20.0.10 --name=cont1 -itd alpine /bin/sh
```

On ub2, the following containers were created:

```
docker run --privileged --net=mynet10 \
--ip=20.10.0.11 --name=cont2 -itd alpine /bin/sh

docker run --privileged --net=mynet20 \
--ip=20.20.0.11 --name=cont3 -itd alpine /bin/sh
```

Each node therefore hosted one container in VLAN 10 and one container in VLAN 20.
Figure 59 shows the running containers on ub1.

```
root@ub1:/home/ubuntu# docker ps
CONTAINER ID   IMAGE     COMMAND     CREATED       STATUS         PORTS     NAMES
9463018d7be9   alpine    "/bin/sh"   2 hours ago   Up 2 seconds             cont1
23be60b1707c   alpine    "/bin/sh"   2 hours ago   Up 2 seconds             cont0
root@ub1:/home/ubuntu#
```

Figure 59: Running containers on ub1

8.5 Verification of Connectivity and Isolation

Connectivity tests were performed using ICMP echo requests between containers attached to the same VLAN.

The following command was executed on ub1:

```
docker exec cont0 ping 20.10.0.11
```

The ping was successful, confirming communication between containers connected to VLAN 10 across different nodes.

Similarly, communication between containers connected to VLAN 20 was verified using:

```
docker exec cont1 ping 20.20.0.11
```

Additional tests were then performed between containers attached to different VLANs. These communication attempts failed, confirming that VLAN isolation was correctly enforced.

Communication between mynet10 and mynet20 was also tested.

To test that the communication fails the following command was executed on ub1:

```
docker exec cont0 ping 20.20.0.11
```

and:

```
docker exec cont1 ping 20.10.0.11
```

Figure 60 shows both successful communication between containers attached to the same VLAN and failed communication attempts between containers attached to different VLANs.

```
root@ub1:/home/ubuntu# docker exec cont1 ping 20.20.0.11
PING 20.20.0.11 (20.20.0.11): 56 data bytes
64 bytes from 20.20.0.11: seq=0 ttl=64 time=2.552 ms
64 bytes from 20.20.0.11: seq=1 ttl=64 time=0.377 ms
64 bytes from 20.20.0.11: seq=2 ttl=64 time=0.624 ms
^Croot@ub1:/home/ubuntudocker exec cont0 ping 20.20.0.11
PING 20.20.0.11 (20.20.0.11): 56 data bytes
ping: sendto: Network unreachable
root@ub1:/home/ubuntu# docker exec cont0 ping 20.10.0.11
PING 20.10.0.11 (20.10.0.11): 56 data bytes
64 bytes from 20.10.0.11: seq=0 ttl=64 time=1.112 ms
64 bytes from 20.10.0.11: seq=1 ttl=64 time=0.564 ms
64 bytes from 20.10.0.11: seq=2 ttl=64 time=0.650 ms
64 bytes from 20.10.0.11: seq=3 ttl=64 time=0.597 ms
^Croot@ub1:/home/ubuntudocker exec cont1 ping 20.10.0.11
PING 20.10.0.11 (20.10.0.11): 56 data bytes
ping: sendto: Network unreachable
root@ub1:/home/ubuntu#
```

Figure 60: Connectivity tests between containers in the same and different VLANs

8.6 Traffic Analysis

Packet captures were performed using Wireshark on the GNS3 links connecting the Ubuntu nodes to the Ethernet switch.

The captures revealed Ethernet frames containing IEEE 802.1Q headers with VLAN identifiers 10 and 20, confirming that VLAN tagging was correctly applied to the transmitted traffic.

Containers attached to VLAN 10 exchanged packets containing VLAN ID 10, while containers attached to VLAN 20 exchanged packets containing VLAN ID 20.

Figures 61 and 62 show captured frames containing IEEE 802.1Q VLAN headers for VLANs 10 and 20, respectively.

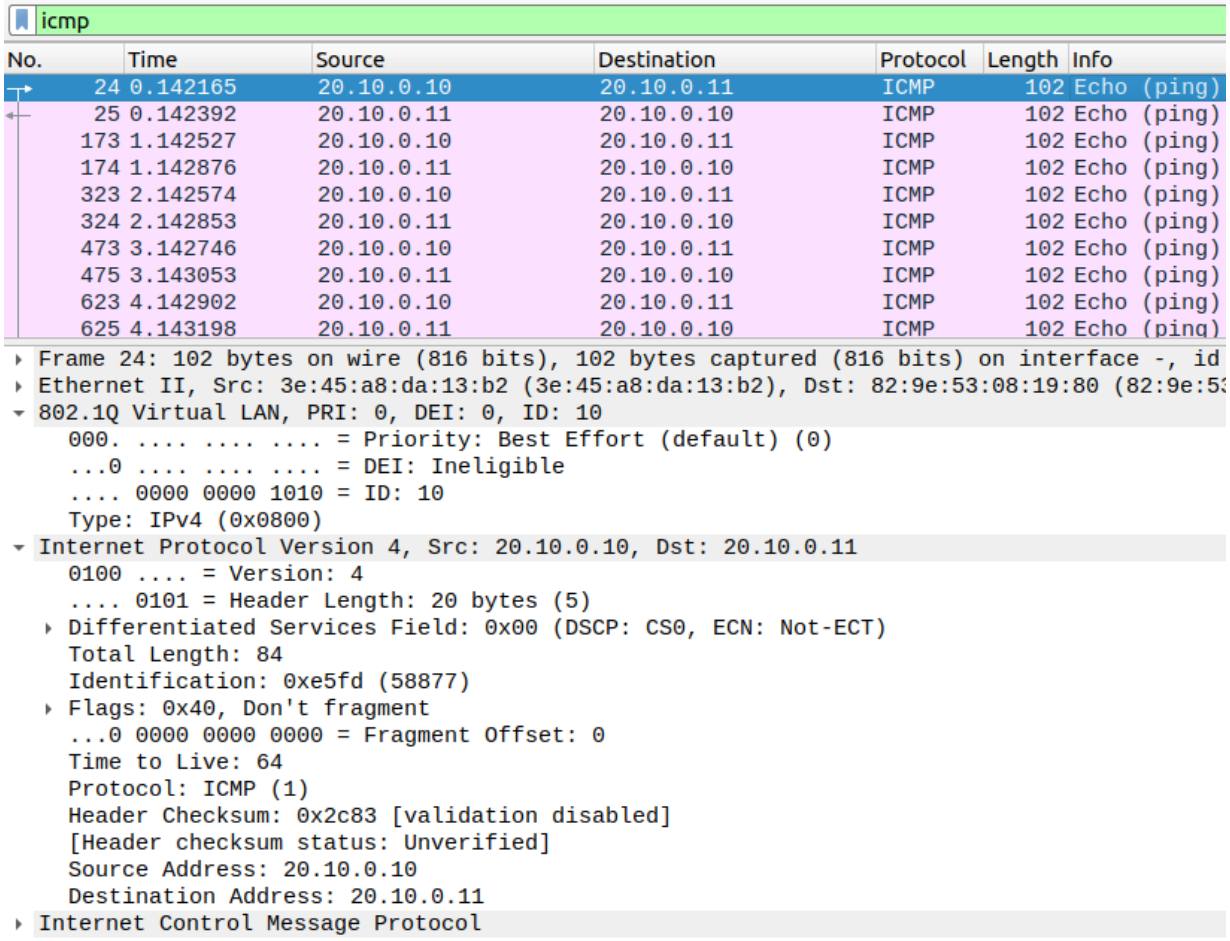


Figure 61: Wireshark capture showing IEEE 802.1Q VLAN tagging with VLAN ID 10

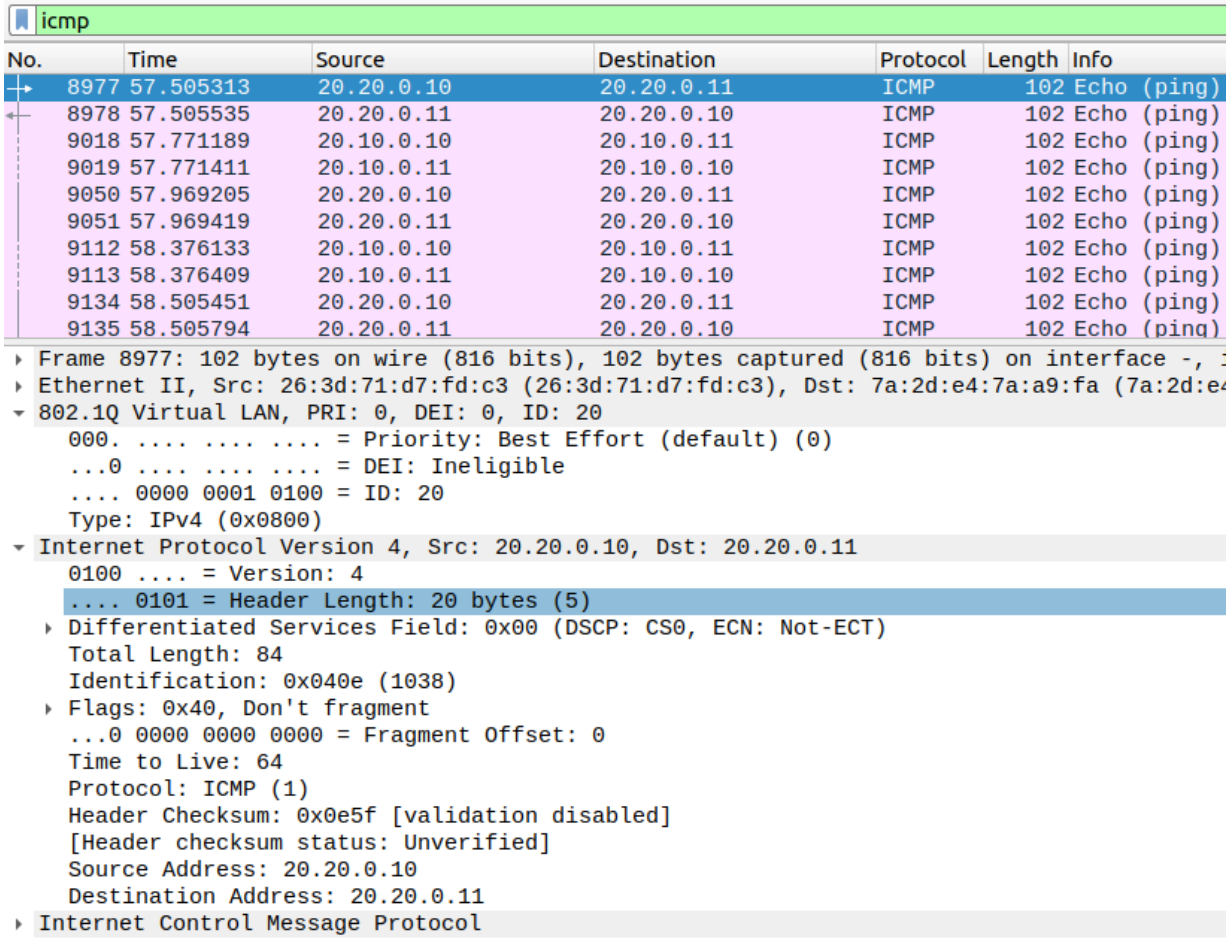


Figure 62: Wireshark capture showing IEEE 802.1Q VLAN tagging with VLAN ID 20

The source and destination MAC addresses observed in the captured frames corresponded directly to the MAC addresses assigned to the communicating containers. The physical interfaces of the Ubuntu hosts acted as transport interfaces responsible for transmitting VLAN-tagged Ethernet frames between the two nodes through the Ethernet switch.

8.7 Discussion and Limitations

The experiment demonstrated that Docker macvlan networks can be combined with IEEE 802.1Q VLAN tagging to provide Layer 2 isolation between containers distributed across multiple hosts.

Containers attached to the same VLAN were able to communicate directly through the Ethernet switch, while communication between different VLANs was blocked.

One limitation of macvlan networks is that the host system cannot directly communicate with containers attached to the macvlan interface unless additional routing or bridge configurations are introduced. Another limitation is that troubleshooting and packet captures may be more complex because traffic generated by macvlan interfaces is not always visible through standard host interface captures depending on the Linux networking path used internally by Docker and the kernel.

The physical interfaces of the Ubuntu nodes acted as transport media for VLAN-tagged Ethernet frames exchanged between containers located on different hosts, while the VLAN subinterfaces provided logical Layer 2 separation between the different macvlan networks.

9 Linux VXLAN with Multicast Routing

9.1 Introduction

The objective of this exercise was to create two overlay networks using VXLAN technology combined with IP multicast routing. All three Ubuntu nodes from the network scenario were used: `ub1` and `ub2`, connected to router `R1` via `Switch1` on the `20.0.0.0/24` subnet, and `ub3`, connected directly to `R1` on the `21.0.0.0/24` subnet.

Two overlay networks were configured:

- `bluenet`, using subnet `10.0.0.0/24` and VXLAN Network Identifier (VNI) 33, associated with multicast group `239.1.1.3`;
- `rednet`, using subnet `11.0.0.0/24` and VNI 66, associated with multicast group `239.1.1.6`.

A total of ten containers were deployed across the three nodes. The distribution of containers per node and network is shown in Table 1.

	bluenet	**rednet**
`ub1`	b1: 10.0.0.1, b2: 10.0.0.2	r1: 11.0.0.1, r2: 11.0.0.2
`ub2`	b3: 10.0.0.3, b4: 10.0.0.4	r3: 11.0.0.3, r4: 11.0.0.4
`ub3`	b5: 10.0.0.5, b6: 10.0.0.6	—

Table 1: Distribution of containers per node and network

Node `ub3` was only connected to `bluenet`, as it does not participate in `rednet`.

9.2 VXLAN Interface Configuration

The configuration procedure at each node consisted of five steps: creating the VXLAN interfaces, activating them, assigning IP addresses, creating Docker macvlan networks, and deploying containers.

On `ub1`, two VXLAN interfaces were created and configured as follows:

```
ip link add blue type vxlan id 33 group 239.1.1.3 ttl 5 dev ens3 dstport 4789
ip link add red  type vxlan id 66 group 239.1.1.6 ttl 5 dev ens3 dstport 4789

ip link set blue up
ip link set red  up

ip addr add 10.0.0.101/24 dev blue
ip addr add 11.0.0.101/24 dev red
```

Each VXLAN interface was bound to the physical interface `ens3` and configured to use UDP destination port 4789, the IANA-assigned port for VXLAN. The `group` parameter associates each interface with a distinct IP multicast group, so that broadcast and unknown unicast traffic within each overlay network is distributed via multicast rather than flooding all endpoints. The TTL was set to 5 to limit multicast propagation scope.

The IP addresses assigned to the VXLAN interfaces serve as the gateway addresses for the Docker macvlan networks created on that node. Each node was assigned a distinct gateway: `.101` on `ub1`, `.102` on `ub2`, and `.103` on `ub3`.

The same procedure was repeated on `ub2` and `ub3`, with the respective addresses and, in the case of `ub3`, only the `blue` interface being created.

Figure 63 shows the VXLAN interfaces configured on `ub1`.

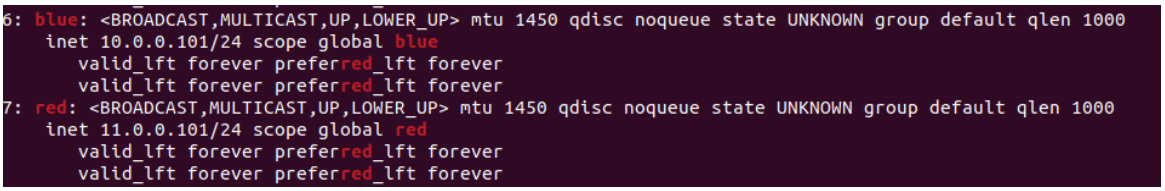


Figure 63: VXLAN interfaces configured on ub1

9.3 Docker Network and Container Deployment

Two Docker macvlan networks were created on `ub1`, each attached to the corresponding VXLAN interface:

```
docker network create -d macvlan \
  --subnet=10.0.0.0/24 --gateway=10.0.0.101 \
  -o parent=blue bluenet

docker network create -d macvlan \
  --subnet=11.0.0.0/24 --gateway=11.0.0.101 \
  -o parent=red rednet
```

Containers were then created and attached to the appropriate networks:

```
docker run --privileged --net=bluenet --name=b1 --ip=10.0.0.1 -itd alpine /bin/sh
docker run --privileged --net=bluenet --name=b2 --ip=10.0.0.2 -itd alpine /bin/sh
docker run --privileged --net=rednet  --name=r1 --ip=11.0.0.1 -itd alpine /bin/sh
docker run --privileged --net=rednet  --name=r2 --ip=11.0.0.2 -itd alpine /bin/sh
```

The same procedure was followed on `ub2` and `ub3`, adjusting container names, IP addresses, and gateway addresses accordingly. On `ub3`, only the `bluenet` network and containers `b5` and `b6` were created.

Figure 64 shows the Docker networks created on `ub1`, and Figure 65 shows all running containers.

```
root@ub1:/home/ubuntu# docker network ls | grep -E "bluenet|rednet"
2e61085cddbb   bluenet   macvlan   local
afae2b4ed98d   rednet    macvlan   local
root@ub1:/home/ubuntu#
```

Figure 64: Docker macvlan networks on ub1

```
root@ub1:/home/ubuntu# docker container ls
CONTAINER ID   IMAGE     COMMAND     CREATED          STATUS          PORTS     NAMES
276f60239c5a   alpine    "/bin/sh"   29 minutes ago   Up 29 minutes             r2
9787ed319393   alpine    "/bin/sh"   29 minutes ago   Up 29 minutes             r1
5e19936e8e89   alpine    "/bin/sh"   29 minutes ago   Up 29 minutes             b2
6c2365153c85   alpine    "/bin/sh"   29 minutes ago   Up 29 minutes             b1
root@ub1:/home/ubuntu#
```

Figure 65: Running containers on ub1

9.4 Multicast Routing Configuration

Protocol Independent Multicast in Dense Mode (PIM-DM) was enabled on router `R1` to support the distribution of multicast traffic between the two network segments. The following configuration was applied:

```
ip multicast-routing

interface f0/0
 ip pim dense-mode

interface f0/1
 ip pim dense-mode
```

PIM Dense Mode was chosen as it floods multicast traffic on all interfaces and subsequently prunes branches where no listeners are present. Both interfaces were configured since `f0/0` connects to the segment containing `ub1` and `ub2`, while `f0/1` connects to the segment containing `ub3`.

9.5 Connectivity Verification

After completing the configuration on all nodes, connectivity between containers of the same overlay network was verified using ICMP echo requests.

The following tests were performed from `ub1`:

```
docker exec b1 ping -c 4 10.0.0.2   # b1 -> b2, same node
docker exec b1 ping -c 4 10.0.0.3   # b1 -> b3, ub1 -> ub2
docker exec b2 ping -c 4 10.0.0.6   # b2 -> b6, ub1 -> ub3
docker exec r1 ping -c 4 11.0.0.3   # r1 -> r3, rednet ub1 -> ub2
```

All pings were successful, confirming that the overlay networks were correctly established across all three nodes and that multicast routing at `R1` was forwarding traffic between the two physical segments.

Figure 66 shows the results of the connectivity tests.

```
root@ub1:/home/ubuntu# docker exec b1 ping -c 4 10.0.0.2
PING 10.0.0.2 (10.0.0.2): 56 data bytes
64 bytes from 10.0.0.2: seq=0 ttl=64 time=0.484 ms
64 bytes from 10.0.0.2: seq=1 ttl=64 time=0.075 ms
64 bytes from 10.0.0.2: seq=2 ttl=64 time=0.072 ms
64 bytes from 10.0.0.2: seq=3 ttl=64 time=0.106 ms

--- 10.0.0.2 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.072/0.184/0.484 ms
root@ub1:/home/ubuntu# docker exec b1 ping -c 4 10.0.0.3
PING 10.0.0.3 (10.0.0.3): 56 data bytes
64 bytes from 10.0.0.3: seq=0 ttl=64 time=2.005 ms
64 bytes from 10.0.0.3: seq=1 ttl=64 time=0.836 ms
64 bytes from 10.0.0.3: seq=2 ttl=64 time=0.696 ms
64 bytes from 10.0.0.3: seq=3 ttl=64 time=0.823 ms

--- 10.0.0.3 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.696/1.090/2.005 ms
root@ub1:/home/ubuntu# docker exec b2 ping -c 4 10.0.0.6
PING 10.0.0.6 (10.0.0.6): 56 data bytes
64 bytes from 10.0.0.6: seq=0 ttl=64 time=16.179 ms
64 bytes from 10.0.0.6: seq=1 ttl=64 time=13.641 ms
64 bytes from 10.0.0.6: seq=2 ttl=64 time=10.861 ms
64 bytes from 10.0.0.6: seq=3 ttl=64 time=19.838 ms

--- 10.0.0.6 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 10.861/15.129/19.838 ms
root@ub1:/home/ubuntu# docker exec r1 ping -c 4 11.0.0.3
PING 11.0.0.3 (11.0.0.3): 56 data bytes
64 bytes from 11.0.0.3: seq=0 ttl=64 time=1.482 ms
64 bytes from 11.0.0.3: seq=1 ttl=64 time=0.603 ms
64 bytes from 11.0.0.3: seq=2 ttl=64 time=0.491 ms
64 bytes from 11.0.0.3: seq=3 ttl=64 time=0.657 ms

--- 11.0.0.3 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.491/0.808/1.482 ms
```

Figure 66: Successful ping tests between containers across nodes

9.6 IGMP Analysis

Wireshark captures performed on the `ens3` interfaces of each node revealed IGMP Membership Report messages being transmitted when the VXLAN interfaces were brought up. These messages inform `R1` that a node wishes to receive traffic destined for a given multicast group.

Nodes `ub1` and `ub2` transmitted membership reports for both multicast groups `239.1.1.3` and `239.1.1.6`, corresponding to `bluenet` and `rednet` respectively. Node `ub3`, having only a

`blue` interface, transmitted a membership report only for group `239.1.1.3`.

This was confirmed at `R1` using the command `show ip igmp groups`, the output of which is shown in Figure 67.

```
R1#sh ip igmp groups
IGMP Connected Group Membership
Group Address    Interface                Uptime    Expires   Last Reporter   Group Accounted
239.1.1.3        FastEthernet0/1          00:03:41  00:02:06  21.0.0.1
239.1.1.3        FastEthernet0/0          00:18:32  00:02:09  20.0.0.2
239.1.1.6        FastEthernet0/0          00:18:32  00:02:06  20.0.0.1
224.0.1.40       FastEthernet0/0          00:21:55  00:02:05  20.0.0.254
```

Figure 67: IGMP group membership table at R1

Group `239.1.1.3` appeared on both `FastEthernet0/0` and `FastEthernet0/1`, reflecting that all three nodes joined `bluenet`. Group `239.1.1.6` appeared only on `FastEthernet0/0`, confirming that `ub3` did not join `rednet`.

Figure 68 shows a Wireshark capture of an IGMP Membership Report transmitted by `ub3`.

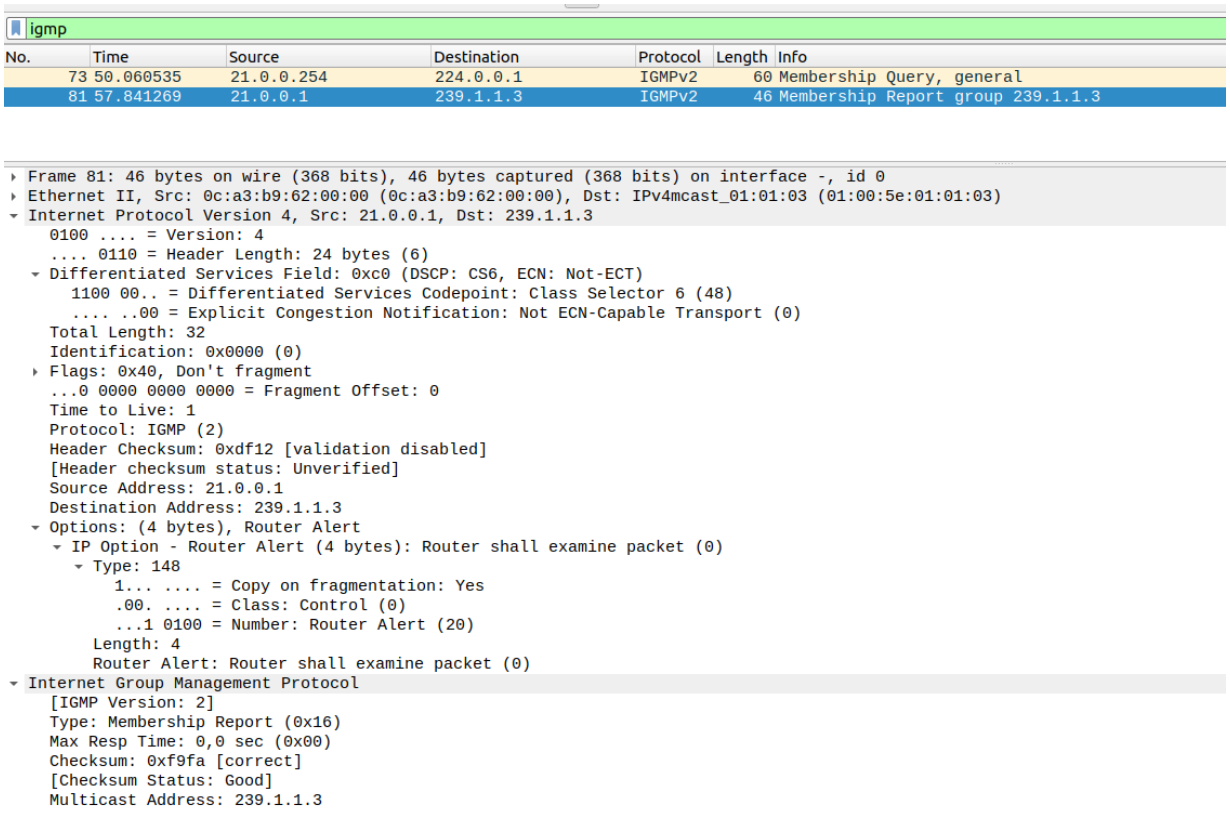


Figure 68: IGMP Membership Report captured on ub3

9.7 ARP and ICMP Encapsulation Analysis

To trigger ARP traffic for analysis, the ARP cache of container `b1` was cleared before initiating a ping to `b6` on `ub3`:

```
docker exec b1 /bin/sh -c "arp -d 10.0.0.6; ping -c 4 10.0.0.6"
```

Wireshark captures were performed on the `ens3` interface of `ub1` to observe the full encapsulation of the resulting packets.

9.7.1 Packet Encapsulation

All traffic between containers located on different nodes was encapsulated using the VXLAN protocol. Each packet consisted of two layers of IP addressing: an *inner* IP header carrying the original container-to-container traffic, and an *outer* IP header used for routing across the physical underlay network.

The inner IP header contained the container source and destination addresses (e.g. `10.0.0.1` → `10.0.0.6`), while the outer IP header contained the physical node addresses (e.g. `20.0.0.1` → `21.0.0.1`). The inner frame was encapsulated in a UDP datagram with destination port 4789, preceded by an 8-byte VXLAN header containing the VNI field.

Figure 69 shows the full encapsulation of a captured ICMP echo request.

```
▸ Frame 44: 148 bytes on wire (1184 bits), 148 bytes captured (1184 bits) on interface -, id 0
▸ Ethernet II, Src: c2:01:18:c6:00:01 (c2:01:18:c6:00:01), Dst: 0c:a3:b9:62:00:00 (0c:a3:b9:62:00:00)
▾ Internet Protocol Version 4, Src: 20.0.0.1, Dst: 21.0.0.1
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▾ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
      0000 00.. = Differentiated Services Codepoint: Default (0)
      .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
    Total Length: 134
    Identification: 0xf0a4 (61604)
  ▸ Flags: 0x00
    ...0 0000 0000 0000 = Fragment Offset: 0
  ▸ Time to Live: 4
    Protocol: UDP (17)
    Header Checksum: 0x9cc1 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 20.0.0.1
    Destination Address: 21.0.0.1
▸ User Datagram Protocol, Src Port: 43806, Dst Port: 4789
▾ Virtual eXtensible Local Area Network
  ▸ Flags: 0x0800, VXLAN Network ID (VNI)
    Group Policy ID: 0
    VXLAN Network Identifier (VNI): 33
    Reserved: 0
▸ Ethernet II, Src: ea:56:05:86:60:b3 (ea:56:05:86:60:b3), Dst: 96:22:9e:e3:95:c7 (96:22:9e:e3:95:c7)
▾ Internet Protocol Version 4, Src: 10.0.0.2, Dst: 10.0.0.6
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▾ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
      0000 00.. = Differentiated Services Codepoint: Default (0)
      .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
    Total Length: 84
    Identification: 0x1d01 (7425)
  ▸ Flags: 0x40, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: ICMP (1)
    Header Checksum: 0x09a1 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.0.0.2
    Destination Address: 10.0.0.6
▸ Internet Control Message Protocol
```

Figure 69: VXLAN encapsulation of an ICMP echo request between b2 and b6, captured on the R1–ub3 link (outer IP TTL decremented to 4 by R1)

9.7.2 ARP Request and Reply

ARP Request packets were sent with a multicast outer IP destination address corresponding to the VXLAN group of the overlay network (`239.1.1.3` for `bluenet`). This ensures that all nodes participating in the overlay receive the broadcast ARP Request, since VXLAN has no mechanism to natively broadcast at the underlay level. ARP Replies, being unicast, were encapsulated with the specific physical IP of the responding node as the outer destination.

The multicast destination address used in ARP Requests differs between networks: `bluenet` uses `239.1.1.3` while `rednet` uses `239.1.1.6`, as each overlay is associated with a distinct multicast group.

Figure 70 shows a captured ARP Request and Figure 71 shows the corresponding ARP Reply.

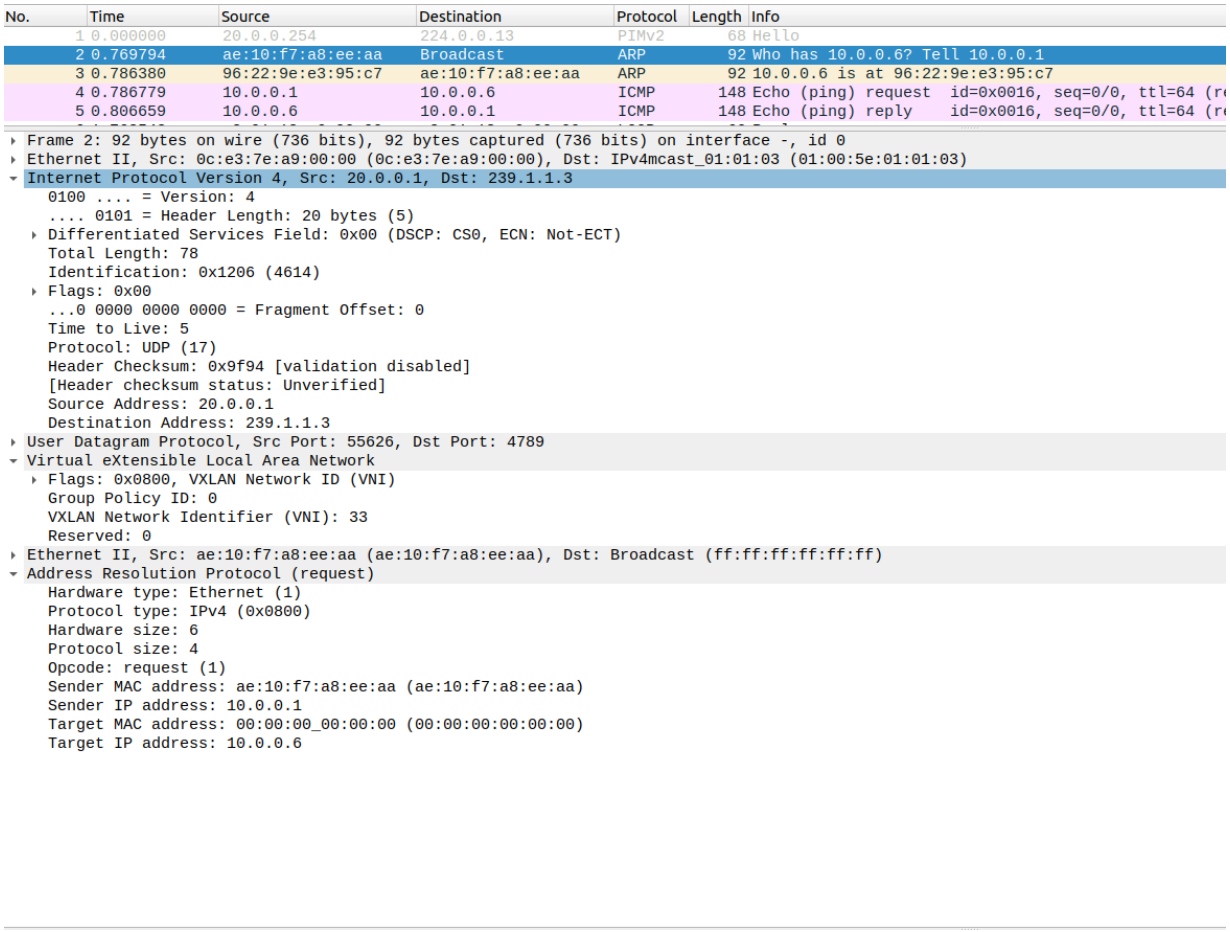


Figure 70: ARP Request encapsulated in VXLAN with multicast outer destination

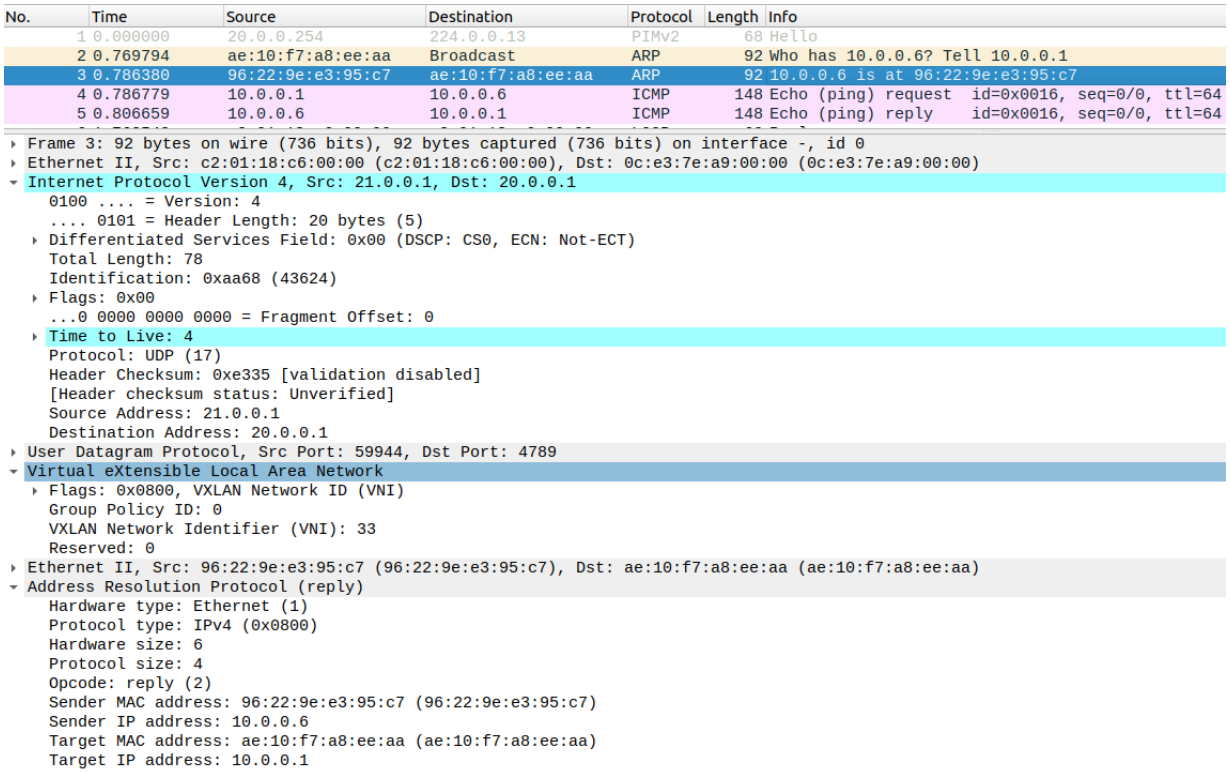


Figure 71: ARP Reply encapsulated in VXLAN with unicast outer destination

9.7.3 ARP Cache

After the ping completed, the ARP cache of container `b1` was inspected using `arp -n`. The cache contained an entry mapping `10.0.0.6` to the MAC address of container `b6`'s network interface, matching exactly the inner Ethernet source address observed in the ARP Reply capture. This confirms that the ARP mechanism operates entirely at the overlay level, with containers unaware of the underlying VXLAN encapsulation.

Figure 72 shows the ARP cache of container `b1` after the ping.

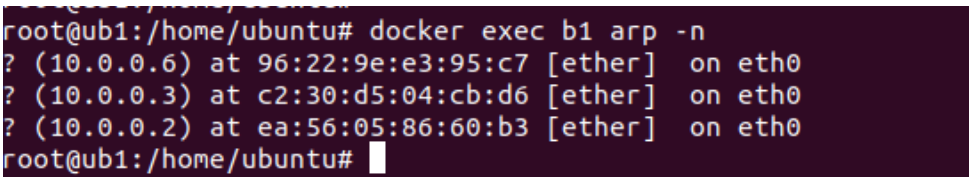


Figure 72: ARP cache of container b1 after pinging b6

9.7.4 VNI Field

The VXLAN header carries a 24-bit VNI field that identifies the overlay network to which the encapsulated frame belongs. In this exercise, VNI 33 was used for `bluenet` and VNI 66 for `rednet`. Upon receiving a VXLAN packet, the destination node uses the VNI to demultiplex the inner frame into the correct overlay network, in the same way that a VLAN tag identifies a broadcast domain on a trunk link. This allows multiple isolated overlay networks to coexist on

the same physical infrastructure and share the same UDP port without frames leaking between networks.

9.8 Comparison with AI Tool Explanation

The AI tool was asked to explain the role of IGMP and multicast routing in this setup, and to predict how ARP and ICMP packets would be encapsulated across nodes.

The AI correctly described the general role of IGMP: each node signals membership in the multicast group associated with its VXLAN interface, so that ARP broadcasts — which have no native VXLAN broadcast mechanism — can be distributed to all VTEP endpoints in the overlay. It also correctly predicted the double-encapsulation structure: inner Ethernet/IP carrying container addresses, outer UDP/IP carrying physical node addresses, with destination port 4789.

Two discrepancies emerged from the experimental results. First, the AI stated that ARP Replies would be sent to the multicast group address, like ARP Requests. The captures in Figures 70 and 71 show otherwise: ARP Replies use a unicast outer destination — the physical IP of the node hosting the requesting container — not the multicast group address. This is consistent with the fact that by the time a reply is generated, the responding VTEP already knows the requester's physical address from the incoming ARP Request, making multicast unnecessary.

Second, the AI described VXLAN isolation in general terms but did not explain the specific role of the VNI field in demultiplexing multiple overlay networks sharing the same physical infrastructure and UDP port. As confirmed by the captures, packets between `bluenet` containers always carry VNI 33 and packets between `rednet` containers always carry VNI 66, regardless of the physical nodes involved. The VNI is what prevents frames from leaking between the two overlay networks — analogous to a VLAN tag on a trunk link — a detail the AI omitted.

10 Docker Swarm Services

10.1 Introduction

The objective of this exercise was to deploy and manage a replicated service using Docker Swarm, and to observe its self-healing behaviour when individual containers or entire nodes fail. The exercise builds on the Swarm cluster configured in Section 7.1, consisting of `ub1` as the manager node and `ub2` and `ub3` as worker nodes, with the `blue` and `red` overlay networks already created.

10.2 Service Creation

A replicated service named `bluesvc` was created on the `blue` overlay network using the `ruivaladas/ws-simple-2` image, with five replicas and external access exposed on port 3000:

```
docker service create --network blue --name bluesvc \
    --replicas 5 --publish 3000:80 ruivaladas/ws-simple-2
```

The `ws-simple-2` image runs a lightweight HTTP server written in Go that listens on port 80 and responds with an HTTP 200 OK message containing its hostname. After creation, `docker service ls` confirmed that the service was running with 5/5 replicas, and `docker service ps bluesvc` revealed the distribution of replicas across nodes, as shown in Figure 73.

```
root@ub1:/home/ubuntu# docker node ls
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
oxcaegbb1uhxku22nbmw51dk5 *   ub1        Ready     Active         Leader           29.5.2
34kg8v82u7x7womjsnvf9f9zu     ub2        Ready     Active                          29.5.2
sjv17sbdgdzan02ko7wqonw0g     ub3        Ready     Active                          29.5.2
root@ub1:/home/ubuntu#
```

Figure 73: Initial replica distribution of bluesvc across the three nodes

Swarm distributed the five replicas unevenly: two were placed on `ub1`, two on `ub3`, and one on `ub2`. This distribution reflects Swarm's scheduling algorithm, which attempts to spread replicas across available nodes based on resource availability rather than strict round-robin placement.

10.3 Self-Healing: Single Container Failure

To evaluate Swarm's self-healing capability, the container running `bluesvc.4` on `ub2` was stopped manually to simulate a container crash:

```
docker container stop <CONTAINER_ID>
```

Swarm detected the failure and immediately scheduled a replacement replica on `ub2`, as shown in Figure 74. The total replica count remained at five throughout, confirming that Swarm continuously reconciles the actual state with the desired state declared at service creation.

```
root@ub1:/home/ubuntu# docker service ps bluesvc
ID             NAME            IMAGE                              NODE      DESIRED STATE   CURRENT STATE            ERROR                       PORTS
b3lcjnbme4qp   bluesvc.1       ruivaladas/ws-simple-2:latest      ub3       Running         Running 4 minutes ago
a65nr99qc8av   bluesvc.2       ruivaladas/ws-simple-2:latest      ub1       Running         Running 4 minutes ago
qn665v847cxu   bluesvc.3       ruivaladas/ws-simple-2:latest      ub3       Running         Running 4 minutes ago
2i3x7va32iwp   bluesvc.4       ruivaladas/ws-simple-2:latest      ub2       Running         Running 31 seconds ago
bmolhol96u56    \_ bluesvc.4   ruivaladas/ws-simple-2:latest      ub2       Shutdown        Failed 37 seconds ago    "task: non-zero exit (2)"
sgtpv348ma0q   bluesvc.5       ruivaladas/ws-simple-2:latest      ub1       Running         Running 4 minutes ago
root@ub1:/home/ubuntu#
```

Figure 74: Swarm scheduling a replacement for the stopped container on ub2

The stopped container was then restarted manually using `docker container start`. Inspecting the service with `docker service ps bluesvc` confirmed that the restarted container was not reabsorbed into the service: Swarm had already replaced it with a new task and tracks replicas by task identity rather than container ID. The manually restarted container continued running as an orphan outside the service.

The same test was repeated by stopping a container on `ub3`, which held two replicas. The replacement was scheduled on `ub2` rather than `ub3`, as shown in Figure 75. This confirms that Swarm does not apply node affinity when rescheduling failed tasks: the replacement is placed on whichever node has available capacity, independently of where the original task was running.

```
root@ub1:/home/ubuntu# docker service ps bluesvc
ID             NAME            IMAGE                           NODE      DESIRED STATE   CURRENT STATE           ERROR                       PORTS
b3lcjnbme4qp   bluesvc.1       ruivaladas/ws-simple-2:latest   ub3       Running         Running 9 minutes ago
a65nr99qc8av   bluesvc.2       ruivaladas/ws-simple-2:latest   ub1       Running         Running 9 minutes ago
qn665v847cxu   bluesvc.3       ruivaladas/ws-simple-2:latest   ub3       Running         Running 9 minutes ago
2i3x7va32iwp   bluesvc.4       ruivaladas/ws-simple-2:latest   ub2       Running         Running 5 minutes ago
bmolhol96u56    \_ bluesvc.4   ruivaladas/ws-simple-2:latest   ub2       Shutdown        Failed 5 minutes ago    "task: non-zero exit (2)"
sgtpv348ma0q   bluesvc.5       ruivaladas/ws-simple-2:latest   ub1       Running         Running 9 minutes ago
root@ub1:/home/ubuntu#
```

Figure 75: Replacement container scheduled on a different node (ub2) after stopping a replica on ub3

10.4 Self-Healing: Node Failure

To simulate a complete node failure, `ub3` was forced to leave the Swarm:

```
docker swarm leave
```

As shown in Figure 76, `ub3` transitioned to a `Down` status in `docker node ls`, and all replicas previously assigned to it were rescheduled onto the remaining nodes (`ub1` and `ub2`). The total replica count was maintained at five. The behaviour is consistent with single container failure: in both cases Swarm detects the loss of running tasks and immediately schedules replacements to restore the desired state.

```
root@ub1:/home/ubuntu# docker node ls
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
oxcaegbb1uhxku22nbmw51dk5 *   ub1        Ready     Active         Leader           29.5.2
34kg8v82u7x7womjsnvf9f9zu     ub2        Ready     Active                          29.5.2
sjv17sbdgdzan02ko7wqonw0g     ub3        Down      Active                          29.5.2
root@ub1:/home/ubuntu# docker service ps bluesvc
ID             NAME            IMAGE                           NODE      DESIRED STATE   CURRENT STATE            ERROR                       PORTS
2yssjdrjnonm   bluesvc.1       ruivaladas/ws-simple-2:latest   ub2       Running         Running 26 seconds ago
b3lcjnbme4qp    \_ bluesvc.1   ruivaladas/ws-simple-2:latest   ub3       Shutdown        Running 29 minutes ago
a65nr99qc8av   bluesvc.2       ruivaladas/ws-simple-2:latest   ub1       Running         Running 29 minutes ago
8ex2dfq8ddve   bluesvc.3       ruivaladas/ws-simple-2:latest   ub2       Running         Running 12 minutes ago
qn665v847cxu    \_ bluesvc.3   ruivaladas/ws-simple-2:latest   ub3       Shutdown        Failed 12 minutes ago    "task: non-zero exit (2)"
2i3x7va32iwp   bluesvc.4       ruivaladas/ws-simple-2:latest   ub2       Running         Running 25 minutes ago
bmolhol96u56    \_ bluesvc.4   ruivaladas/ws-simple-2:latest   ub2       Shutdown        Failed 25 minutes ago    "task: non-zero exit (2)"
sgtpv348ma0q   bluesvc.5       ruivaladas/ws-simple-2:latest   ub1       Running         Running 29 minutes ago
root@ub1:/home/ubuntu#
```

Figure 76: Node ub3 shown as Down after leaving the Swarm, with its replicas rescheduled to ub1 and ub2

10.5 Node Recovery and Rebalancing

After `ub3` left the Swarm, it was rejoined as a worker node using the token obtained from the manager:

```
docker swarm join-token worker   # run at ub1
docker swarm join --token <TOKEN> 20.0.0.1:2377   # run at ub3
```

As shown in Figure 77, `ub3` rejoined the cluster as a new node entry (the previous `Down` entry remained visible). However, no replicas were automatically redistributed to it: all five tasks continued running on `ub1` and `ub2`. This confirms that Docker Swarm does not perform automatic rebalancing after a node recovery, in order to avoid unnecessarily disrupting running services.

```
root@ub1:/home/ubuntu# docker node ls
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
oxcaegbb1uhxku22nbmw51dk5 *   ub1        Ready     Active         Leader           29.5.2
34kg8v82u7x7womjsnvf9f9zu     ub2        Ready     Active                          29.5.2
mqyuxkmgtyl85xq4k45rbenh4     ub3        Ready     Active                          29.5.2
sjv17sbdgdzan02ko7wqonw0g     ub3        Down      Active                          29.5.2
root@ub1:/home/ubuntu# docker service ps bluesvc
ID             NAME            IMAGE                              NODE      DESIRED STATE   CURRENT STATE            ERROR                         PORTS
2yssjdrjnonm   bluesvc.1       ruivaladas/ws-simple-2:latest      ub2       Running         Running 2 minutes ago
b3lcjnbme4qp    \_ bluesvc.1   ruivaladas/ws-simple-2:latest      ub3       Shutdown        Running 31 minutes ago
a65nr99qc8av   bluesvc.2       ruivaladas/ws-simple-2:latest      ub1       Running         Running 31 minutes ago
8ex2dfq8ddve   bluesvc.3       ruivaladas/ws-simple-2:latest      ub2       Running         Running 14 minutes ago
qn665v847cxu    \_ bluesvc.3   ruivaladas/ws-simple-2:latest      ub3       Shutdown        Failed 14 minutes ago    "task: non-zero exit (2)"
2i3x7va32iwp   bluesvc.4       ruivaladas/ws-simple-2:latest      ub2       Running         Running 26 minutes ago
bmolhol96u56    \_ bluesvc.4   ruivaladas/ws-simple-2:latest      ub2       Shutdown        Failed 26 minutes ago    "task: non-zero exit (2)"
sgtpv348ma0q   bluesvc.5       ruivaladas/ws-simple-2:latest      ub1       Running         Running 31 minutes ago
root@ub1:/home/ubuntu#
```

Figure 77: ub3 rejoined as a new node, with no automatic redistribution of replicas

To redistribute replicas across all nodes including the recovered `ub3`, a forced service update was issued at the manager:

```
docker service update --force bluesvc
```

This command triggers a rolling restart of all service tasks, replacing each one sequentially. As shown in Figure 78, Swarm used the opportunity to reschedule tasks more evenly, and `ub3` received one replica after the update completed. The output `verify: Service bluesvc converged` confirmed that all five tasks reached the running state successfully. The `--force` flag is therefore the standard mechanism to manually trigger rebalancing in Docker Swarm following node recovery.

```
root@ub1:/home/ubuntu# docker service update --force bluesvc
bluesvc
overall progress: 5 out of 5 tasks
1/5: running
2/5: running
3/5: running
4/5: running
5/5: running
verify: Service bluesvc converged
root@ub1:/home/ubuntu# docker service ps bluesvc
ID             NAME            IMAGE                              NODE      DESIRED STATE   CURRENT STATE            ERROR                         PORTS
s2ugq2v1vv8v   bluesvc.1       ruivaladas/ws-simple-2:latest      ub2       Running         Running 25 seconds ago
2yssjdrjnonm    \_ bluesvc.1   ruivaladas/ws-simple-2:latest      ub2       Shutdown        Shutdown 26 seconds ago
b3lcjnbme4qp    \_ bluesvc.1   ruivaladas/ws-simple-2:latest      ub3       Shutdown        Running 40 minutes ago
0ylqc5d89dg9   bluesvc.2       ruivaladas/ws-simple-2:latest      ub1       Running         Running 17 seconds ago
a65nr99qc8av    \_ bluesvc.2   ruivaladas/ws-simple-2:latest      ub1       Shutdown        Shutdown 18 seconds ago
ikg96xtfha9l   bluesvc.3       ruivaladas/ws-simple-2:latest      ub2       Running         Running 21 seconds ago
8ex2dfq8ddve    \_ bluesvc.3   ruivaladas/ws-simple-2:latest      ub2       Shutdown        Shutdown 22 seconds ago
qn665v847cxu    \_ bluesvc.3   ruivaladas/ws-simple-2:latest      ub3       Shutdown        Failed 24 minutes ago    "task: non-zero exit (2)"
gdy2fokjomfo   bluesvc.4       ruivaladas/ws-simple-2:latest      ub3       Running         Running 30 seconds ago
2i3x7va32iwp    \_ bluesvc.4   ruivaladas/ws-simple-2:latest      ub2       Shutdown        Shutdown 31 seconds ago
bmolhol96u56    \_ bluesvc.4   ruivaladas/ws-simple-2:latest      ub2       Shutdown        Failed 36 minutes ago    "task: non-zero exit (2)"
xu9900s8jnt7   bluesvc.5       ruivaladas/ws-simple-2:latest      ub1       Running         Running 12 seconds ago
sgtpv348ma0q    \_ bluesvc.5   ruivaladas/ws-simple-2:latest      ub1       Shutdown        Shutdown 13 seconds ago
root@ub1:/home/ubuntu#
```

Figure 78: Forced update redistributing replicas across all three nodes including the recovered ub3

10.6 Load Balancing

To test the load balancing behaviour of the service, a Python client was deployed on the `saar-tools` appliance, connected to `Switch1` on the `20.0.0.0/24` subnet. The client sends a configurable number of sequential HTTP requests to the service, with a one-second interval between each, and prints the hostname returned by the server. Ten requests were directed to `ub1` on port 3000:

```
python3 /tmp/client.py 20.0.0.1 3000 10
```

As shown in Figure 79, the five container hostnames appeared in a repeating cycle of the same fixed order across all ten responses, confirming that Docker Swarm uses a **round-robin** load balancing algorithm. Each of the five replicas was contacted exactly twice, with no deviation from the sequence.

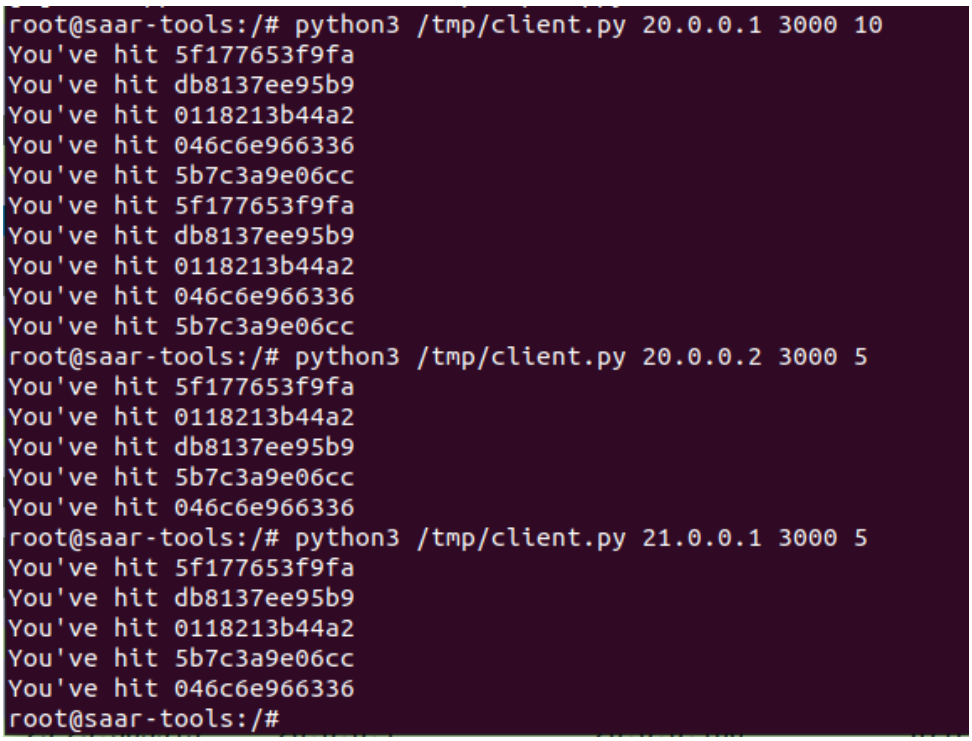


Figure 79: Round-robin load balancing observed across 10 HTTP requests to bluesvc

The test was repeated directing requests to `ub2` (20.0.0.2) and `ub3` (21.0.0.1) on the same port. In all cases the service responded correctly, confirming that Docker Swarm's **routing mesh** allows the service to be reached at any node's IP address, regardless of whether that node is running a replica of the service. Incoming connections are intercepted by the ingress network and forwarded to an available container transparently.

A Wireshark capture was performed on the link between `saar-tools` and `Switch1` to inspect the individual HTTP messages. Figure 80 shows the HTTP GET request sent by the client to `20.0.0.1` on port 3000, and Figure 81 shows the corresponding HTTP 200 OK response.

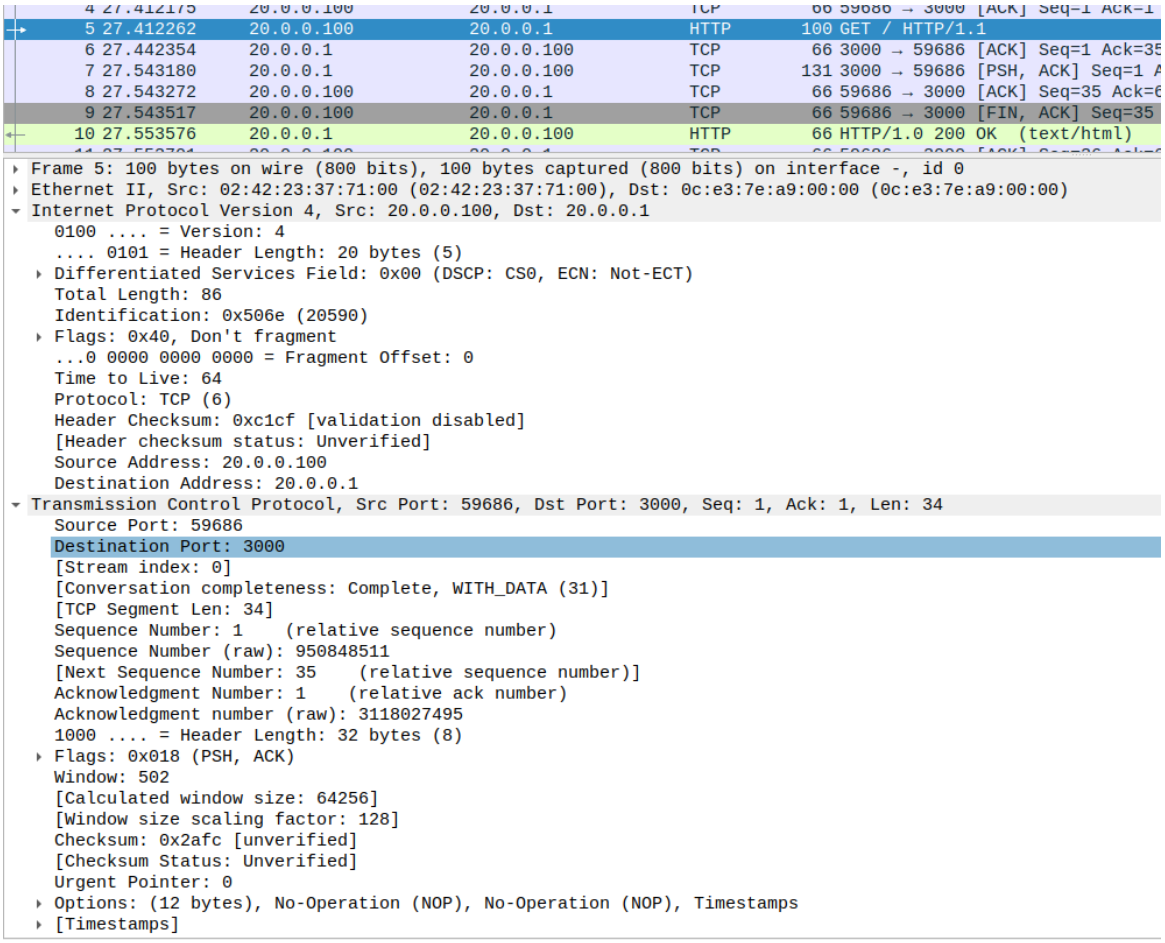


Figure 80: HTTP GET request from saar-tools to bluesvc on port 3000

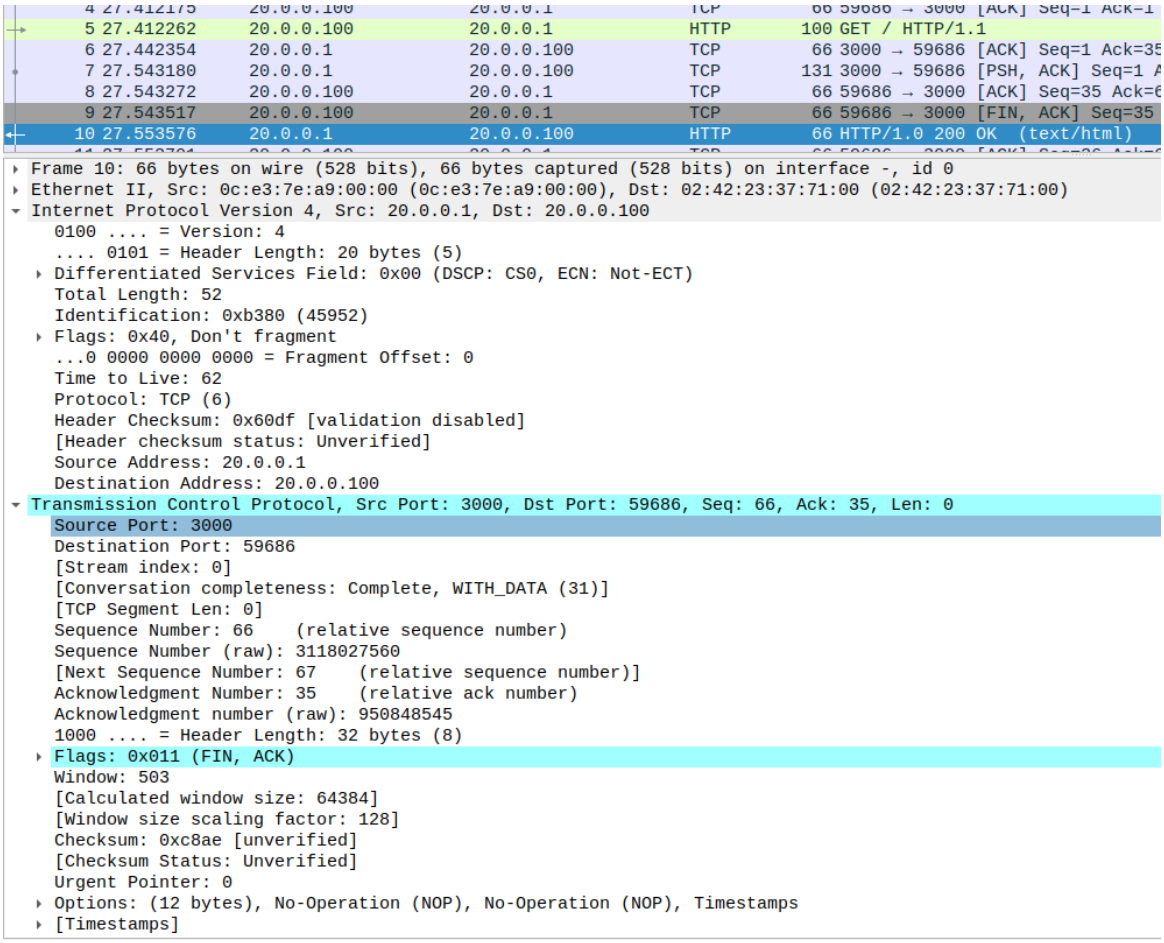


Figure 81: HTTP 200 OK response from bluesvc to saar-tools

The capture shown in Figure 82 confirmed that a new TCP connection was established for every HTTP request. Each request followed a complete SYN, SYN-ACK, ACK, GET, PSH/ACK, HTTP 200 OK, FIN sequence, with successive connections opening approximately one second apart.

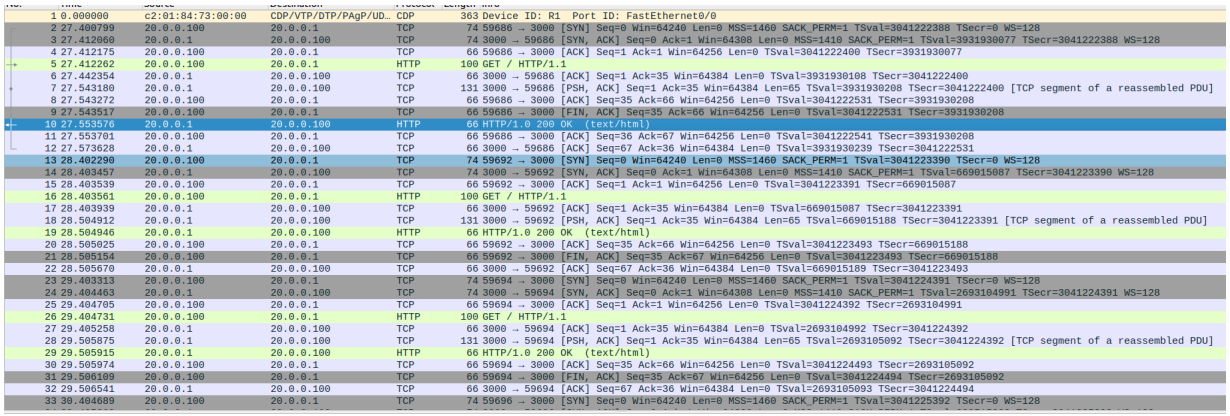


Figure 82: Wireshark capture on the saar-tools link showing repeated TCP connection cycles

10.7 Service Rescaling

The service was rescaled from five to three replicas using:

```
docker service scale bluesvc=3
```

As shown in Figure 83, Swarm immediately shut down two of the excess replicas and converged to the new desired state of three running tasks, one per node. The output `verify: Service bluesvc converged` confirmed successful rescaling.

```
root@ub1:/home/ubuntu# docker service scale bluesvc=3
bluesvc scaled to 3
overall progress: 3 out of 3 tasks
1/3: task: non-zero exit (255)
2/3: running
3/3: task: non-zero exit (255)
verify: Service bluesvc converged
root@ub1:/home/ubuntu# docker service ps bluesvc
ID             NAME            IMAGE                              NODE      DESIRED STATE   CURRENT STATE            ERROR                         PORTS
qd6f6xo7havg   bluesvc.1       ruivaladas/ws-simple-2:latest      ub2       Running         Running 13 minutes ago
s2ugq2v1vv8v    \_ bluesvc.1   ruivaladas/ws-simple-2:latest      ub2       Shutdown        Failed 13 minutes ago    "task: non-zero exit (255)"
2yssjdrjnonm    \_ bluesvc.1   ruivaladas/ws-simple-2:latest      ub2       Shutdown        Shutdown 3 hours ago
b3lcjnbme4qp    \_ bluesvc.1   ruivaladas/ws-simple-2:latest      ub3       Shutdown        Running 4 hours ago
qtea4dzg9302   bluesvc.2       ruivaladas/ws-simple-2:latest      ub1       Running         Running 13 minutes ago
0ylqc5d89dg9    \_ bluesvc.2   ruivaladas/ws-simple-2:latest      ub1       Shutdown        Failed 13 minutes ago    "task: non-zero exit (255)"
a65nr99qc8av    \_ bluesvc.2   ruivaladas/ws-simple-2:latest      ub1       Shutdown        Shutdown 3 hours ago
ikg96xtfha9l   bluesvc.3       ruivaladas/ws-simple-2:latest      ub2       Shutdown        Failed 13 minutes ago    "task: non-zero exit (255)"
8ex2dfq8ddve    \_ bluesvc.3   ruivaladas/ws-simple-2:latest      ub2       Shutdown        Shutdown 3 hours ago
qn665v847cxu    \_ bluesvc.3   ruivaladas/ws-simple-2:latest      ub3       Shutdown        Failed 3 hours ago       "task: non-zero exit (2)"
t1mzpsegoo6f   bluesvc.4       ruivaladas/ws-simple-2:latest      ub3       Running         Running 13 minutes ago
gdy2fokjomfo    \_ bluesvc.4   ruivaladas/ws-simple-2:latest      ub3       Shutdown        Failed 13 minutes ago    "task: non-zero exit (255)"
2i3x7va32iwp    \_ bluesvc.4   ruivaladas/ws-simple-2:latest      ub2       Shutdown        Shutdown 3 hours ago
bmolhol96u56    \_ bluesvc.4   ruivaladas/ws-simple-2:latest      ub2       Shutdown        Failed 13 minutes ago    "task: non-zero exit (2)"
xu9900s8jnt7   bluesvc.5       ruivaladas/ws-simple-2:latest      ub1       Shutdown        Failed 13 minutes ago    "task: non-zero exit (255)"
sgtpv348ma0q    \_ bluesvc.5   ruivaladas/ws-simple-2:latest      ub1       Shutdown        Shutdown 3 hours ago
```

Figure 83: bluesvc rescaled from five to three replicas, one per node

Docker Swarm does not support automatic scaling based on workload natively. Achieving autoscaling would require an external solution, such as a monitoring script that periodically measures resource utilisation and issues `docker service scale` commands accordingly, or a third-party orchestration platform. Kubernetes addresses this natively through its Horizontal Pod Autoscaler (HPA), which scales deployments automatically based on observed CPU or memory metrics.

10.8 Container Network Interfaces

With three replicas running, one per node, the network interfaces of each container were inspected using `docker exec <CONTAINER_ID> ip addr`. As shown in Figures 84, 85, and 86, each container presented three non-loopback interfaces, one more than the standalone containers of Section 7.1.

```
root@ub1:/home/ubuntu# docker container ls
CONTAINER ID   IMAGE                              COMMAND          CREATED          STATUS          PORTS     NAMES
5b7c3a9e06cc   ruivaladas/ws-simple-2:latest      "./ws_simple"    35 minutes ago   Up 35 minutes             bluesvc.2.qtea4dzg9302qr863r0rwf985
root@ub1:/home/ubuntu# docker exec 5b7c3a9e06cc ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
15: eth0@if16: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue state UP group default
    link/ether 02:42:0a:00:01:03 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.0.1.3/24 brd 10.0.1.255 scope global eth0
       valid_lft forever preferred_lft forever
17: eth1@if18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 5a:55:cd:1c:7a:d2 brd ff:ff:ff:ff:ff:ff link-netnsid 1
    inet 172.18.0.3/16 brd 172.18.255.255 scope global eth1
       valid_lft forever preferred_lft forever
21: eth2@if22: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue state UP group default
    link/ether 02:42:0a:00:00:06 brd ff:ff:ff:ff:ff:ff link-netnsid 2
    inet 10.0.0.6/24 brd 10.0.0.255 scope global eth2
       valid_lft forever preferred_lft forever
root@ub1:/home/ubuntu#
```

Figure 84: Network interfaces of the bluesvc container on ub1

```
root@ub2:/home/ubuntu# docker container ls
CONTAINER ID   IMAGE                          COMMAND          CREATED          STATUS          PORTS     NAMES
db8137ee95b9   ruivaladas/ws-simple-2:latest   "./ws_simple"   35 minutes ago   Up 35 minutes             bluesvc.1.qd6f6xo7havgwynqxbzgmrvpb
root@ub2:/home/ubuntu# docker exec db8137ee95b9 ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
13: eth0@if14: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue state UP group default
    link/ether 02:42:0a:00:00:08 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.0.0.8/24 brd 10.0.0.255 scope global eth0
       valid_lft forever preferred_lft forever
17: eth1@if18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 42:af:73:c6:9f:3f brd ff:ff:ff:ff:ff:ff link-netnsid 1
    inet 172.18.0.3/16 brd 172.18.255.255 scope global eth1
       valid_lft forever preferred_lft forever
21: eth2@if22: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue state UP group default
    link/ether 02:42:0a:00:01:06 brd ff:ff:ff:ff:ff:ff link-netnsid 2
    inet 10.0.1.6/24 brd 10.0.1.255 scope global eth2
       valid_lft forever preferred_lft forever
root@ub2:/home/ubuntu#
```

Figure 85: Network interfaces of the bluesvc container on ub2

```
root@ub3:/home/ubuntu# docker container ls
CONTAINER ID   IMAGE                          COMMAND          CREATED          STATUS          PORTS     NAMES
5f177653f9fa   ruivaladas/ws-simple-2:latest   "./ws_simple"   35 minutes ago   Up 35 minutes             bluesvc.4.t1mzpsegoo6fshm5b7imy302u
root@ub3:/home/ubuntu# docker exec 5f177653f9fa ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
13: eth0@if14: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue state UP group default
    link/ether 02:42:0a:00:00:09 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.0.0.9/24 brd 10.0.0.255 scope global eth0
       valid_lft forever preferred_lft forever
15: eth1@if16: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 3a:9c:42:6a:98:d0 brd ff:ff:ff:ff:ff:ff link-netnsid 1
    inet 172.18.0.3/16 brd 172.18.255.255 scope global eth1
       valid_lft forever preferred_lft forever
17: eth2@if18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue state UP group default
    link/ether 02:42:0a:00:01:08 brd ff:ff:ff:ff:ff:ff link-netnsid 2
    inet 10.0.1.8/24 brd 10.0.1.255 scope global eth2
       valid_lft forever preferred_lft forever
root@ub3:/home/ubuntu#
```

Figure 86: Network interfaces of the bluesvc container on ub3

The three interfaces and their corresponding networks are summarised in Table 2.

Interface	Network	VNI	Subnet	Purpose
eth0	blue	4097	10.0.1.0/24	Service overlay network
eth1	docker_gwbridge	—	172.18.0.0/16	Gateway to host
eth2	ingress	4096	10.0.0.0/24	Swarm routing mesh

Table 2: Network interfaces of bluesvc service containers

The `eth0` interface connects the container to the `blue` overlay network (VNI 4097), used for direct container-to-container communication within the service. The `eth1` interface connects to the `docker_gwbridge` local bridge (172.18.0.0/16), which provides the container with a gateway to the host for outbound traffic. The `eth2` interface connects to the `ingress` overlay network (VNI 4096), which implements the Swarm routing mesh and handles externally published traffic arriving on port 3000.

This additional interface, absent in the standalone containers of Section 7.1, is the key structural difference between service containers and standalone containers in Docker Swarm: service containers are always attached to the shared ingress network to participate in the routing mesh.

10.9 HTTP Request Path and VXLAN Encapsulation

To trace the path followed by HTTP requests through the Swarm infrastructure, a Wireshark capture was started on the `ens3` interface of `ub1` and three HTTP requests were sent from `saar-tools` to `ub1` on port 3000. The capture, filtered by HTTP, is shown in Figure 87.

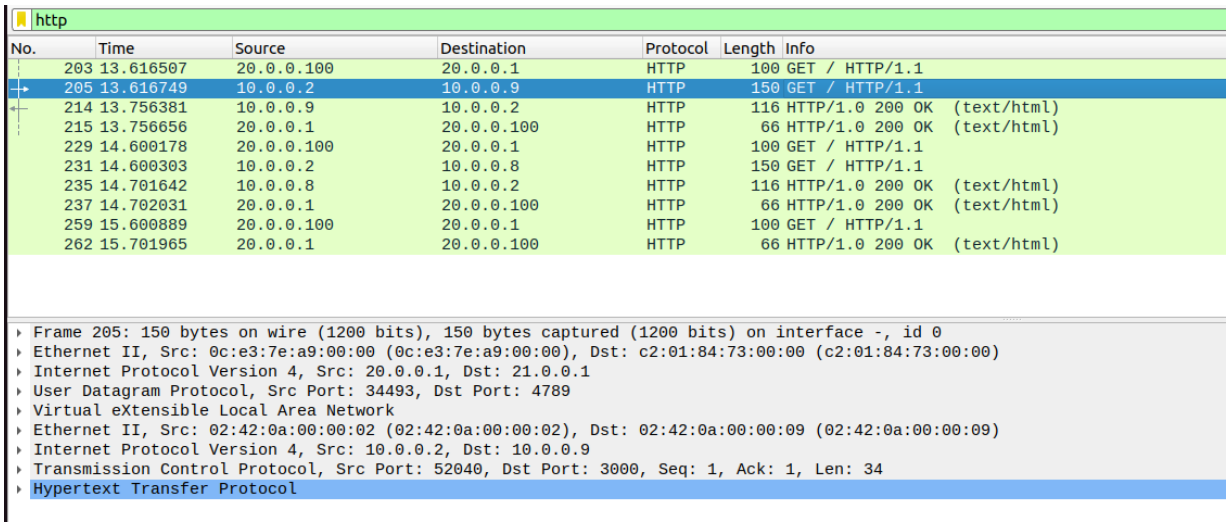


Figure 87: HTTP traffic captured on ub1's ens3 interface during three client requests

Two distinct forwarding paths were observed depending on whether the selected container was local or remote.

Container on ub1 (local). When the routing mesh selected the local replica, the HTTP GET arrived directly from `saar-tools` (20.0.0.100) to `ub1` (20.0.0.1) at the physical layer, as shown in Figure 88. The response was returned directly from `ub1` to `saar-tools` without any tunnelling. No VXLAN packets were generated.

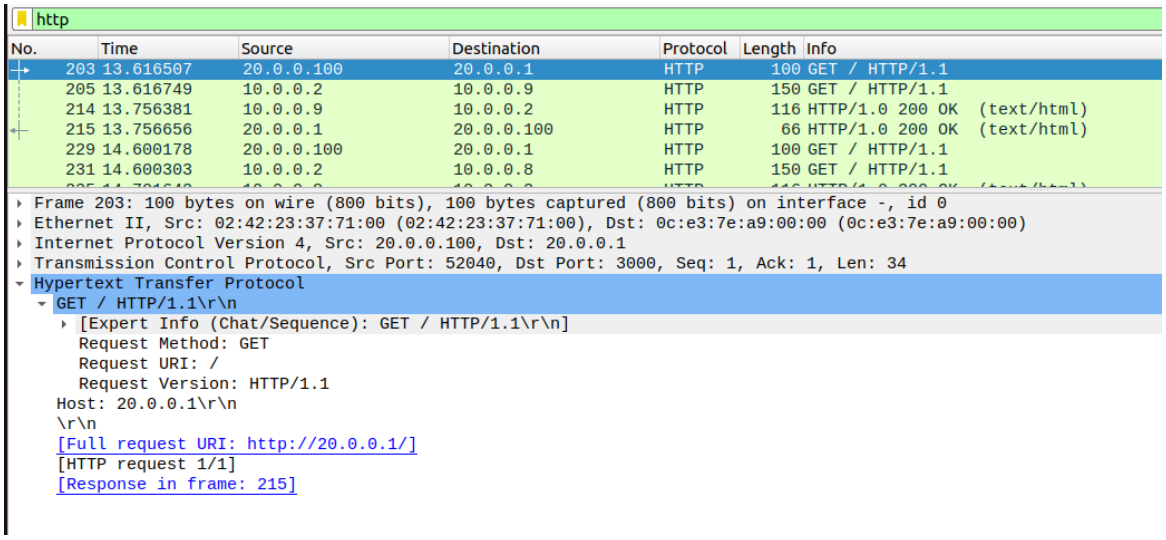


Figure 88: Direct HTTP exchange when the selected container is on the contacted node (ub1)

Container on a remote node (ub2 or ub3). When the routing mesh selected a replica on a different node, `ub1` encapsulated the request in a VXLAN tunnel and forwarded it to the appropriate remote node. The packet detail for a request forwarded to `ub3` is shown in Figure 89.

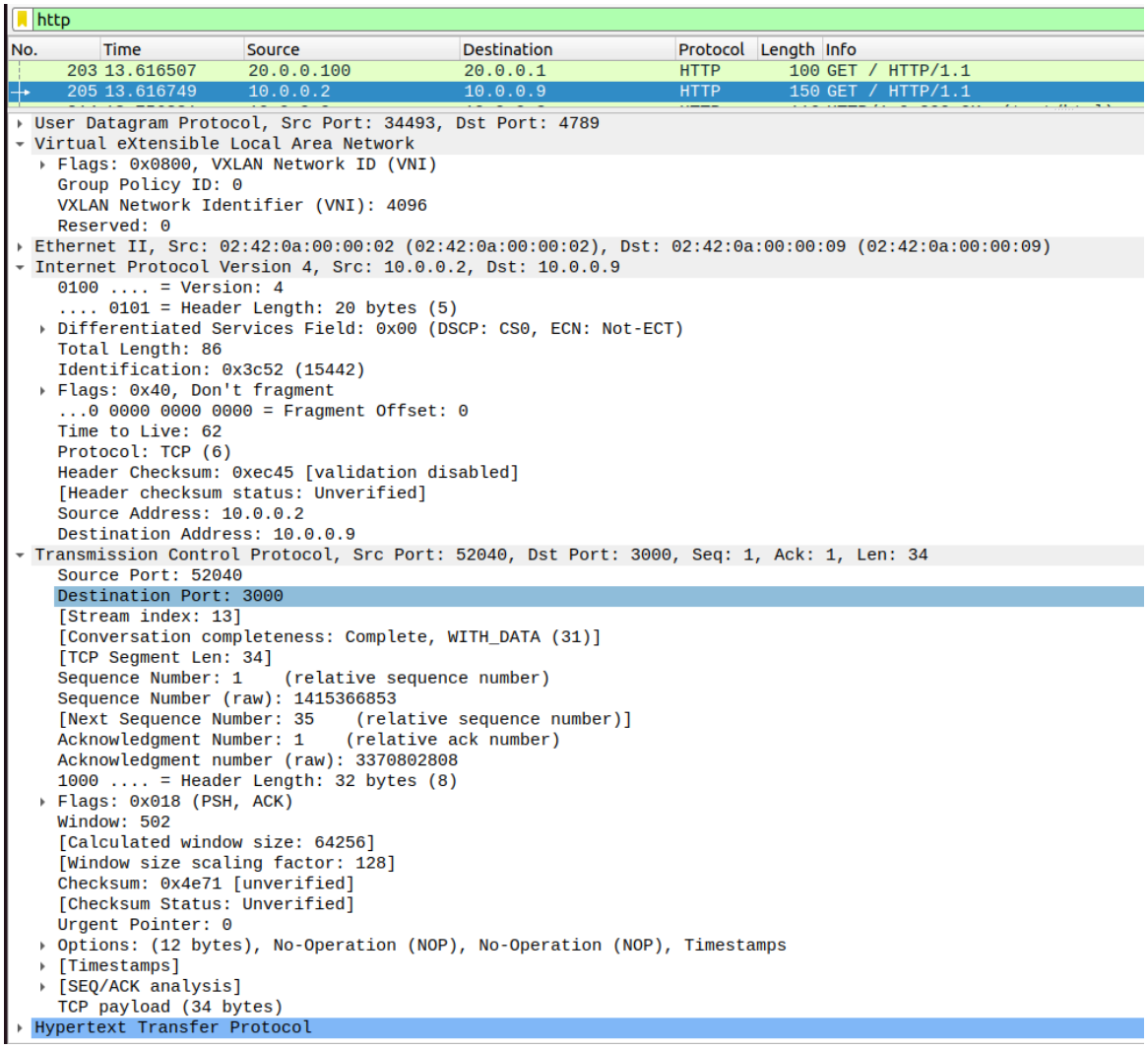


Figure 89: VXLAN encapsulation of an HTTP request forwarded from ub1 to ub3

The encapsulation structure was as follows. The outer IP header carried the physical node addresses (`20.0.0.1` → `21.0.0.1`), routing the packet across the underlay network via `R1`. The UDP destination port was 4789 (the standard VXLAN port). The VXLAN header identified the tunnel with **VNI 4096**, corresponding to the ingress network. The inner Ethernet frame carried the ingress network MAC addresses (`02:42:0a:00:00:02` → `02:42:0a:00:00:09`), and the inner IP header carried the ingress network addresses (`10.0.0.2` → `10.0.0.9`). The inner TCP segment was addressed to port 3000.

The response from the remote container followed the reverse path: the remote node encapsulated the HTTP 200 OK in a VXLAN packet back to `ub1`, which decapsulated it and forwarded the response to `saar-tools`. From the client's perspective, the entire exchange appeared as a direct conversation with `ub1`.

The VXLAN filter view in Figure 90 shows the full sequence of tunnelled exchanges, confirming that two of the three requests required inter-node tunnelling (to `ub3` and `ub2` respectively), while the third was served locally.

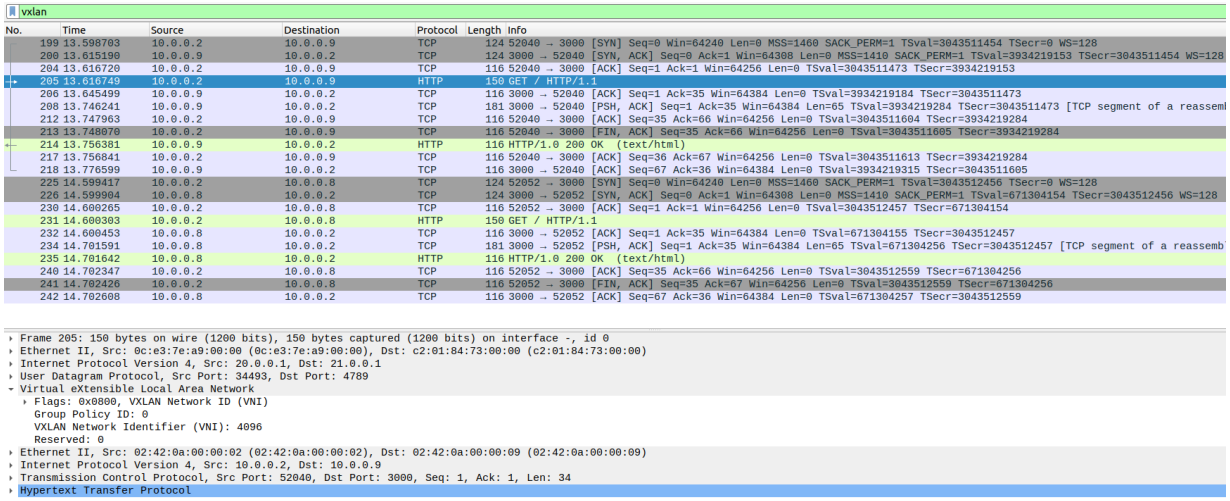


Figure 90: VXLAN-filtered capture showing tunnelled HTTP exchanges to remote nodes

10.10 Comparison with a Second Service: redsvc

A second service, `redsvc`, was created on the `red` overlay network with three replicas and external access on port 3001:

```
docker service create --network red --name redsvc \
    --replicas 3 --publish 3001:80 ruivaladas/ws-simple-2
```

As shown in Figure 91, Swarm distributed one replica per node, mirroring the `bluesvc` distribution.

```
root@ub1:/home/ubuntu# docker service create --network red --name redsvc \
    --replicas 3 --publish 3001:80 ruivaladas/ws-simple-2
kpd3tqbow83c4ib69q24f20qb
overall progress: 3 out of 3 tasks
1/3: running
2/3: running
3/3: running
verify: Service kpd3tqbow83c4ib69q24f20qb converged
root@ub1:/home/ubuntu# docker service ps redsvc
ID             NAME       IMAGE                              NODE      DESIRED STATE   CURRENT STATE            ERROR     PORTS
bcy8p6ftce4l   redsvc.1   ruivaladas/ws-simple-2:latest   ub3       Running         Running 11 seconds ago
wzf9i9a5uosx   redsvc.2   ruivaladas/ws-simple-2:latest   ub1       Running         Running 11 seconds ago
qj7tmg0nfp1u   redsvc.3   ruivaladas/ws-simple-2:latest   ub2       Running         Running 11 seconds ago
root@ub1:/home/ubuntu#
```

Figure 91: Replica distribution of redsvc across the three nodes

A new Wireshark capture was performed while sending three HTTP requests to `ub1` on port 3001. The packet detail of a tunnelled request, shown in Figure 92, revealed that the VXLAN VNI used was again **4096** — identical to `bluesvc`.

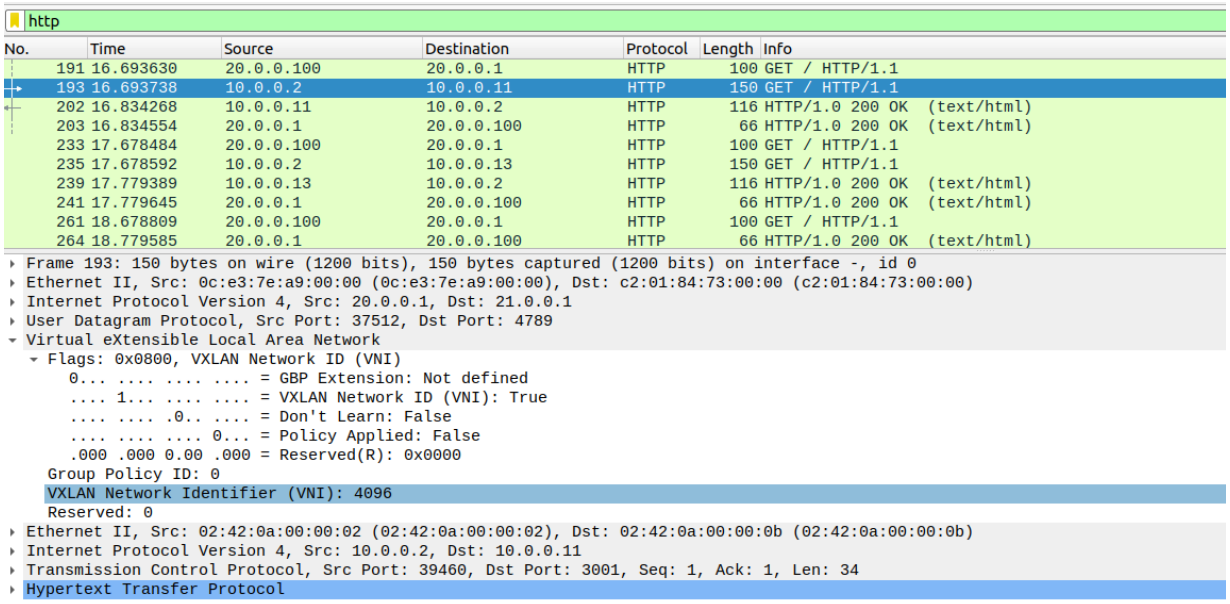


Figure 92: VXLAN encapsulation of an HTTP request for redsvc, showing VNI 4096

This confirms that the ingress network is **shared across all Swarm services**, regardless of which user-defined overlay network (blue or red) the service is attached to. The VNI 4096 tunnel is used for all externally published traffic in the Swarm, and the destination port in the inner TCP header (3000 or 3001) is what distinguishes traffic belonging to different services within the shared ingress tunnel. The `blue` and `red` overlay networks (VNI 4097 and 4098 respectively) are used exclusively for direct container-to-container communication within each service, and are not involved in handling external client requests.

10.11 Comparison with AI Tool Explanation

The behaviour observed experimentally is consistent with the explanation provided by the AI tool consulted during the preparation of this exercise. The AI correctly described Docker Swarm services as a declarative abstraction over replicated containers, identified the ingress network and routing mesh as the mechanism behind load balancing and external accessibility, and explained that self-healing operates by reconciling actual state with desired state without node affinity.

Two points of nuance emerged from the experiments that the AI description addressed only partially. First, the AI noted that Swarm does not rebalance automatically after node recovery, which was confirmed: after `ub3` rejoined the cluster, no replicas were redistributed until a forced update was issued with `docker service update --force`. Second, the AI correctly identified round-robin as the load balancing algorithm, which the Wireshark and client output confirmed precisely. The experimental captures added concrete detail to the AI explanation by showing the actual VNI values, inner and outer IP and MAC address pairs, and the exact packet structure of the VXLAN tunnels used by the ingress network.